Entregable 1 · Detección de intrusiones en redes de drones con datos tabulares

Contenido

- 1. Definición del problema y dataset
- 2. Análisis exploratorio de datos
- 3. Revisión del estado del arte y extracción de los tres modelos base
- 4. Implementación y evaluación de los tres modelos base
- 5. Benchmark de los modelos base y definición del baseline
- 6. Guía de procedimiento para clasificación con datos tabulares
- 7. Limitaciones inferenciales de trabajar con tres modelos
- 8. Conclusiones
- 9. Bibliografía

Estado del arte, análisis exploratorio e implementación de los tres modelos base de la literatura

Autor	Donny Torres
Programa	Maestría en Analítica de Datos, Universidad del Norte (Barranquilla)
Asignatura	Deep Learning
Fecha	Noviembre de 2026
Conjunto de datos	*UAVIDS-2025*, versión 1 (`UAVIDS-2025.csv`)
Fuente	Kaggle: https://www.kaggle.com/datasets/qinglizeng1997/uavids-2025; Zenodo, licencia CC BY 4.0, DOI https://doi.org/10.5281/zenodo.15336998
Tipo de problema	Clasificación con datos tabulares, multiclase de cinco categorías
Repositorio	DTL-DA/proyecto-uavids-2025
Jupyter Book	Pendiente de publicación

Este cuaderno desarrolla el primer entregable del proyecto final de Deep Learning. El propósito es establecer, con evidencia estadística, el nivel mínimo de desempeño (*baseline*) que cualquier modelo avanzado deberá superar al detectar intrusiones en redes de enjambres de drones, de modo que una mejora futura se pueda juzgar por su relevancia y no solo por un promedio más alto. Primero se define el problema, luego se audita el conjunto de datos, se extraen de la literatura los tres modelos de Deep Learning más básicos y se implementan bajo un mismo protocolo, para cerrar con el baseline y su inferencia estadística.

El proyecto tiene tres entregables encadenados: este fija el baseline con tres modelos, el segundo lo amplía a un benchmark de diez modelos y el tercero lleva el resultado a producción con un modelo original. Por eso la partición, el pipeline y las decisiones de este cuaderno se guardan para reutilizarlos.

Configuración del entorno

El cuaderno se ejecuta de principio a fin en CPU, porque el conjunto de datos es tabular y pesa unos 20 MB. Se apoya en tres reglas:

- **Los datos se obtienen con un solo paso.** Si el archivo no está en la carpeta del proyecto, se descarga de Kaggle, y en cualquier caso se verifica su suma SHA-256.
- **Cada corrida se guarda al terminar.** Si la ejecución se interrumpe, se vuelve a ejecutar y se salta lo que ya está hecho.
- **Las semillas y la configuración quedan registradas**, junto con las versiones de las librerías, para que cada resultado se pueda reproducir.

▶ **Mostrar el código de la celda**

```
SHA-256: d50d339f68be7b23f0bf089dd438b20a1835c13182d8641220538121440164d0
Forma del archivo: (122171, 23)
```

▶ **Mostrar el código de la celda**

Componente	Versión
Python	3.13.14
torch	2.6.0+cpu
numpy	2.4.6
pandas	3.0.3
scipy	1.17.1
scikit-learn	1.9.0
statsmodels	0.15.0
matplotlib	3.10.9
Dispositivo	CPU Intel64 Family 6 Model 154 Stepping 4, GenuineIntel, 10 hilos

1. Definición del problema y dataset

En esta sección se formaliza el problema en notación matemática, se declaran por anticipado la métrica, el margen de relevancia y la unidad de independencia, y se justifica y describe

técnicamente el conjunto de datos. Esto es primordial porque estas decisiones condicionan todo lo que sigue y se heredan en los Entregables 2 y 3.

1.1 Definición formal del problema

El sistema recibe los atributos de un flujo de red dentro de un enjambre de drones y debe indicar si es tráfico normal o a qué ataque pertenece. Formalmente:

- **Espacio de entrada:** $\mathcal{X} = \mathbb{R}^d$, el vector de atributos de tráfico y desempeño de un flujo, con d fijado en el análisis exploratorio.
- **Espacio de salida:** $\mathcal{Y}$ = {Blackhole, Flooding, Normal, Sybil, Wormhole}.
- **Función objetivo:** $f : \mathcal{X} \to \mathcal{Y}$, aproximada por $f_\theta(x) = \arg\max_{c \in \mathcal{Y}} p_\theta(c \mid x)$.
- **Función de pérdida:** entropía cruzada, $\mathcal{L}(\theta) = -\frac{1}{n} \sum_{i=1}^{n} w_{y_i} \log p_\theta(y_i \mid x_i)$, con los pesos w_c definidos en el análisis exploratorio.

Tipo de problema. El proyecto corresponde al tipo 1 del enunciado, clasificación con datos tabulares, en su versión multiclase. Se opta por las cinco categorías y no por la versión binaria de normal frente a ataque, porque reconocer el tipo de ataque responde una pregunta más exigente y de ella se puede derivar la binaria, mientras que lo contrario no es posible.

Pregunta de investigación. ¿Qué nivel de desempeño, medido con F1 macro, establecen los tres modelos de Deep Learning más básicos de la literatura al clasificar los flujos de un enjambre de drones en tráfico normal y cuatro ataques, cuando la partición respeta la estructura de dependencia del archivo, y qué mejora debe superar un modelo avanzado para considerarse relevante?

Hipótesis operativa. Al menos un par de modelos base difiere en F1 macro en una magnitud igual o mayor que el margen de relevancia práctica δ.

Declaración a priori. Las decisiones siguientes se fijan antes de ver resultados de cualquier modelo de Deep Learning, para evitar la exploración selectiva. La métrica primaria es el F1 macro, $F1_{macro} = \frac{1}{K} \sum_{c=1}^{K} \frac{2P_cR_c}{P_c+R_c}$, que promedia sin ponderar las $K = 5$ clases, porque en seguridad un ataque poco frecuente importa tanto como uno frecuente. El margen δ = 0,01 tiene una lectura directa: por la definición del promedio macro, equivale a que el F1 de una sola clase cambie 0,05 mientras las demás permanecen iguales, y se contrasta con la variabilidad medida en el piloto de la sección 4.3.1. La unidad de independencia es la serie, una de las que propone el enunciado, porque los flujos consecutivos según `FlowID` son muy parecidos entre

sí, de modo que la dependencia sigue a la posición en el archivo y no a una entidad como el cliente, como se demuestra en la sección 2.1.3. Como el archivo no identifica la simulación de origen, cada serie se aproxima con bloques de 1000 flujos contiguos. Esto no convierte el proyecto en uno de series de tiempo: el archivo no trae marca de tiempo, de modo que no se afirma que ese orden sea cronológico, y ningún modelo usa rezagos como entrada, así que sigue siendo clasificación tabular. Estas decisiones y la partición se fijaron el 3 de noviembre de 2026, antes de entrenar. Al día siguiente solo se cambió el nombre de la unidad para usar el término del enunciado; la partición y las corridas son las mismas.

▶ **Mostrar el código de la celda**

Decisión	**Valor**
Fecha	3 de noviembre de 2026
Métrica primaria	F1 macro en el conjunto de prueba
Métricas secundarias	accuracy, balanced accuracy, precision, recall y F1 (macro, ponderado y por clase), AUC-ROC macro, AUC-PR macro, MCC, ECE
Margen δ	0,0100
Nivel α	0,0500
Unidad de independencia	serie: flujos consecutivos según FlowID, aproximada con bloques de 1000 flujos contiguos
Partición	64 / 16 / 20, estratificada por clase y agrupada por bloques, semilla 42

1.2 Justificación del dataset

Los enjambres de drones se comunican por enlaces inalámbricos y enrutan el tráfico entre ellos mismos, sin una infraestructura fija, de modo que un nodo malicioso puede interrumpir la misión desde adentro. Detectar esa intrusión a partir del tráfico es primordial porque no exige modificar el protocolo de los drones. Los cuatro ataques del conjunto de datos representan esa amenaza: en Blackhole, un nodo atrae rutas y descarta los paquetes; en Flooding, satura la red; en Sybil, se presenta con varias identidades, y en Wormhole, dos nodos crean un túnel que altera las rutas.

El conjunto es público, en Kaggle, Zenodo (licencia CC BY 4.0) e IEEE DataPort (DOI 10.21227/j5p4-zt27). Según sus autores, los flujos se generaron por simulación en NS-3.24, con movilidad de enjambre basada en un algoritmo BOID extendido, IEEE 802.11ac, enrutamiento AODV, tráfico UDP y canal Nakagami, con un alcance de transmisión de 100 m y 20 dBm de

potencia (Zeng et al., 2025). Es decir, son datos simulados y no capturas de drones reales, lo que limita su validez externa.

Hay referencias directas para comparar el baseline. Los autores reportan, con partición aleatoria estratificada de 80 y 20 %, un F1 de 82 % con regresión logística, 93 % con DNN, 93 % con CNN, 94 % con LSTM y 96 % con XGBoost (Zeng et al., 2025). Zarkadis y Douligeris (2026) reportan un F1 de prueba de 90,66 % con MLP y de 94,28 % con RealMLP, y Bakiri et al. (2026), con una partición por nodo de origen, una exactitud de 92,50 % en cinco clases.

1.3 Descripción técnica

Cada fila es un flujo de red, con 22 atributos y la etiqueta. Los autores los agrupan así:

Grupo	Atributos	Uso en los modelos
Conexión	`FlowID`, `SrcAddr`, `SrcPort`, `DstAddr`, `DstPort`, `Protocol`	Se excluyen, por las razones que se miden en la sección 2.1.4
Volumen de tráfico	`FlowDuration/s`, `TxPackets`, `RxPackets`, `LostPackets`, `TxBytes`, `RxBytes`, `TxPacketRate/s`, `RxPacketRate/s`, `TxByteRate/s`, `RxByteRate/s`, `MeanPacketSize`	Entrada
Desempeño	`MeanDelay/s`, `MeanJitter/s`, `Throughput/Kbps`, `PacketDropRate`, `AverageHopCount`	Entrada

La etiqueta no proviene de anotadores: el simulador anota cada flujo con la etiqueta de su escenario, normal o de ataque, y una verificación posterior cruza los tiempos de inyección del ataque con los flujos capturados (Zeng et al., 2025). Por tal motivo, puede haber flujos sin relación con el ataque que reciben la etiqueta de su escenario, un ruido de etiqueta que se examina en la sección 2.1.2. En cuanto a la estructura de dependencia, los flujos de una misma simulación comparten nodos, tiempos y condiciones, y el archivo no identifica esa simulación, de modo que la dependencia se audita de forma indirecta en la sección 2.1.3.

▶ **Mostrar el código de la celda**

Propiedad	Valor
Filas	122171
Columnas	23
Tamaño del archivo (MB)	20,1000
Tipos de columna	{'float64': 10, 'int64': 9, 'str': 4}
Versión	Kaggle y Zenodo, v1
SHA-256	d50d339f68be7b23f0bf089dd438b20a1835c13182d86412205381214401640d0

Interpretación. El archivo tiene 122 171 flujos y 23 columnas, y pesa 20,1 MB, de modo que cabe completo en la memoria y se puede entrenar en CPU. La suma SHA-256 de la tabla identifica esta versión, y el cuaderno se detiene si el archivo cambia.

Esto significa que el tamaño no impone ninguna restricción sobre el diseño, y que las repeticiones del estudio se pueden definir por criterios estadísticos y no por el costo de cómputo.

Por lo anterior, se trabaja con el archivo completo, sin muestreo, salvo en los diagnósticos donde se indique.

2. Análisis exploratorio de datos

En esta sección se audita el conjunto de datos para tomar decisiones de modelado. Cada análisis cierra con la decisión de preprocesamiento que deriva, el impacto en la arquitectura y los riesgos, incluidas las posibles fuentes de fuga de datos. El conjunto de prueba se define en la sección 2.1.3, se congela con su suma SHA-256 y, desde ese punto, no se usa su etiqueta ni para diseñar el preprocesamiento ni para ajustar hiperparámetros.

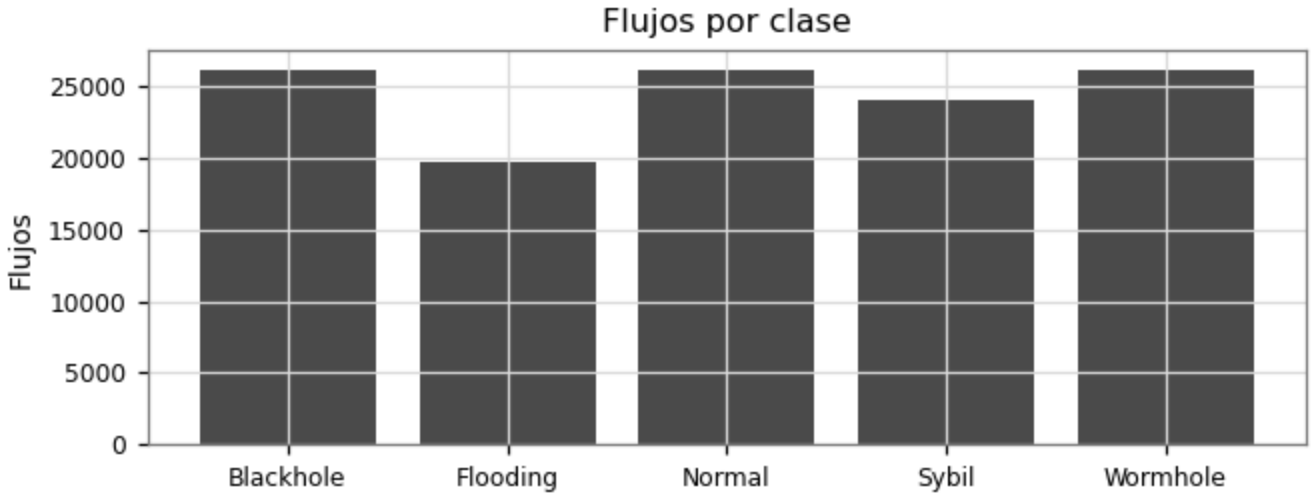
2.1 EDA general

2.1.1 Variable objetivo

Se cuantifica el desbalance con la razón entre la clase mayoritaria y la minoritaria, y con la entropía de la distribución, $H = -\sum_c p_c \log_2 p_c$.

▶ **Mostrar el código de la celda**

Clase	Flujos	%
Blackhole	26110	21,37
Flooding	19726	16,15
Normal	26172	21,42
Sybil	24077	19,71
Wormhole	26086	21,35



Interpretación. Normal es la clase mayoritaria, con 21,4 %, y Flooding la minoritaria, con 16,1 %, para una razón de 1,33 a 1. La entropía es de 2,314 bits frente a un máximo de $\log_2 5$ = 2,322 bits, es decir, el 99,7 % de la incertidumbre de un reparto uniforme.

Esto significa que no hay clases raras y que el desbalance es leve. Un clasificador trivial que responda siempre la clase mayoritaria tendría un F1 macro de 0,071, porque su F1 es cero en las otras cuatro clases, y ese es el piso real de comparación.

Por tal motivo, se conservan todas las clases, sin remuestrear, y se pondera la entropía cruzada, con pesos que se calculan después de eliminar las filas repetidas, porque esa limpieza cambia las proporciones, como muestra la sección 2.1.2.

Decisión de preprocesamiento. No se elimina ni se duplica ninguna muestra por desbalance; los pesos de clase se calculan solo con entrenamiento en la sección 2.2.5.

Impacto en la arquitectura. La capa de salida tiene 5 neuronas con entropía cruzada ponderada, y la selección de modelos se hace con F1 macro.

Riesgos. La proporción de clases es una decisión de quien construyó el archivo, no una frecuencia real de ataques. Por ello, las métricas no anticipan el desempeño en una red donde los ataques sean raros.

2.1.2 Calidad del dataset

Se buscan valores faltantes, explícitos o disfrazados, datos fuera de rango físico, inconsistencias entre atributos y duplicados. Los casi duplicados se buscan redondeando el logaritmo de cada atributo a dos decimales, lo que equivale a una diferencia de cerca de 1 %, y el ruido de etiquetas se estima con los grupos de flujos idénticos o casi idénticos que tienen clases distintas.

▶ **Mostrar el código de la celda**

Revisión	Resultado
Valores nulos	0
Valores no finitos	0
Valores negativos	0
Columnas constantes	Protocol
RxPackets mayor que TxPackets	0
LostPackets distinto de TxPackets - RxPackets	24554
PacketDropRate mayor que 1	81
Flujos sin ningún paquete recibido	30
De ellos, con retardo igual a 0	30

label	Blackhole	Flooding	Normal	Sybil	Wormhole
% de flujos sin recepción	0	0,15	0	0	0

Clase	Flujos	Filas repetidas	% de la clase	% del archivo sin repetidas
Blackhole	26110	0	0,0	22,9
Flooding	19726	7950	40,3	10,3
Normal	26172	0	0,0	23,0
Sybil	24077	269	1,1	20,9
Wormhole	26086	0	0,0	22,9

Revisión	Resultado
Filas repetidas sin contar FlowID	8205
Filas repetidas en los atributos de tráfico, con la misma etiqueta	8219
Grupos idénticos en tráfico con más de una clase	0
Filas en grupos de casi duplicados	33025
Grupos de casi duplicados	7932
Grupos de casi duplicados con más de una clase	187
Filas en esos grupos	547

▶ **Mostrar el código de la celda**

Interpretación. El archivo no tiene valores nulos ni no finitos, y ningún atributo es negativo, así que no hay faltantes explícitos. Sin embargo, 30 flujos, el 0,02 %, no recibieron ningún paquete, y en los 30 el retardo aparece como 0. Ese cero no es un retardo nulo, sino un valor no observado, es decir, un faltante disfrazado. Su mecanismo no es MCAR, porque todos están en Flooding (V de Cramér = 0,036, $p < 0{,}001$), y corresponde a MNAR, ya que el retardo falta justamente porque ningún paquete llegó. Aun así, son tan pocos que su efecto es despreciable.

En cuanto a la consistencia, 81 flujos tienen una tasa de pérdida mayor que 1, imposible por definición, y en 24 554 flujos los paquetes perdidos no coinciden con la diferencia entre enviados y recibidos, lo que indica que el simulador los contabiliza con otro criterio. Así mismo, al ignorar `FlowID` aparecen 8 219 filas repetidas en los atributos de tráfico, todas con la misma etiqueta, concentradas en Flooding, donde son el 40,3 % de la clase. Es decir, al eliminarlas, esa clase pasa de 16,1 % a 10,3 % del archivo. Además, hay 7 932 grupos de casi duplicados, de los cuales 187 mezclan clases.

Hay que tener en cuenta que esos 187 grupos reúnen 547 filas, el 0,4 % del archivo, y que se toman como una cota del ruido de etiquetas: son flujos casi iguales con distinta etiqueta, lo que apunta a tráfico sin relación con el ataque que recibe la etiqueta de su escenario. Esto significa que el F1 macro no puede llegar a 1 aunque el modelo sea perfecto.

Decisión de preprocesamiento. Se eliminan las filas repetidas antes de partir, se recorta `PacketDropRate` a 1 y se conserva el cero del retardo, porque la falta de recepción se expresa también en `RxPackets`. Los casi duplicados se conservan, porque son flujos cortos y parecidos, propios del tráfico, y su efecto sobre la partición se mide en la sección 2.1.3.

Impacto en la arquitectura. Ninguno en la estructura. La reducción de Flooding sí aumenta su peso en la pérdida, como se calcula en la sección 2.2.5.

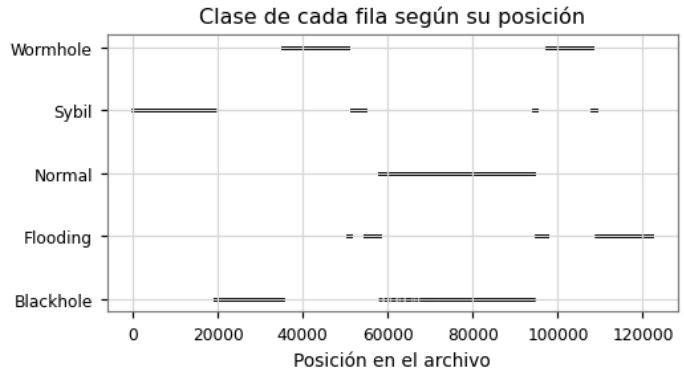
Riesgos. Una fila repetida en entrenamiento y en prueba sería fuga directa, y por eso se elimina. El ruido de etiquetas no se puede corregir, porque el archivo no permite saber qué flujos participan en el ataque.

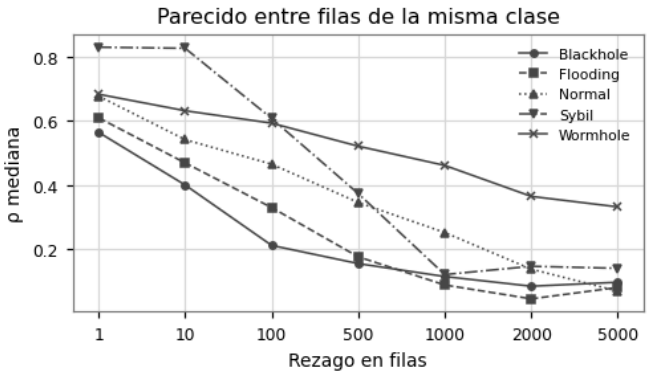
2.1.3 Partición del dataset

La estrategia depende de la estructura de dependencia, así que primero se mide. Si las filas fueran independientes, cada clase ocuparía posiciones al azar en el archivo y dos filas vecinas se parecerían como dos filas cualesquiera. Para saberlo se mide cuántas filas vecinas comparten clase y la correlación de rangos de Spearman entre los atributos de filas de la misma clase separadas por distintos rezagos.

▶ **Mostrar el código de la celda**

Clase	Rezago 1	Rezago 10	Rezago 100	Rezago 500	Rezago 1000	Rezago 2000	Rezago 5000
Blackhole	0,565	0,401	0,212	0,156	0,115	0,085	0,098
Flooding	0,610	0,470	0,330	0,177	0,089	0,046	0,081
Normal	0,678	0,542	0,466	0,346	0,253	0,139	0,070
Sybil	0,830	0,827	0,608	0,374	0,122	0,147	0,141
Wormhole	0,684	0,632	0,594	0,522	0,462	0,365	0,333






```
Mediana entre clases por rezago: {'Rezago 1': 0.678, 'Rezago 10': 0.542, 'Rezago
```

Interpretación. El 93,0 % de las filas vecinas comparte clase, frente a 20,2 % bajo un orden aleatorio, y el tramo más largo de una sola clase tiene 19 180 filas. Además, entre filas de la

misma clase, la ρ de Spearman mediana es de 0,678 en el rezago 1 y de 0,098 en el rezago 5000, frente a 0,000 con un orden aleatorio.

Esto significa que las filas no son independientes: el archivo conserva bloques de flujos muy parecidos, que sugieren un origen común en una misma simulación, aunque no trae el identificador que permitiría comprobarlo. Por eso, una partición aleatoria deja flujos casi gemelos en entrenamiento y en prueba, y mide en parte la capacidad de reconocer esa cercanía y no el ataque. Esa dependencia tampoco sigue a una entidad como el cliente: 125 de las 176 direcciones de origen aparecen en dos o más clases, y los tramos contiguos de una misma dirección tienen un largo mediano de 1 fila, de modo que agrupar por dirección no separaría a los flujos vecinos.

Por tal motivo, se declara la serie como unidad de independencia y, como el archivo no marca dónde empieza y termina cada una, se aproxima con una partición agrupada por bloques de filas contiguas, en la que un bloque completo queda en una sola partición.

Decisión de preprocesamiento. Cada fila se asigna a un bloque según su posición en el archivo, y los bloques se reparten entre las particiones, estratificando por clase.

Impacto en la arquitectura. Ninguno directo. La partición agrupada sí cambia el error con el que se comparan las arquitecturas, porque ahora mide la generalización a bloques no vistos.

Riesgos. El bloque es una aproximación, y no equivale a la simulación: los flujos a ambos lados del borde de un bloque siguen siendo parecidos, y un mismo escenario largo puede aportar bloques a varias particiones. Es decir, la partición agrupada reduce la fuga, pero no la elimina. Se declara como amenaza a la validez interna en la sección 5.5.

▶ **Mostrar el código de la celda**

```
SHA-256 del conjunto de prueba: 2a2dab99c08ed1f19f24add07f5c6b0375e5ce0df7951a990
Filas de prueba con un casi duplicado en entrenamiento: 1.8 % | coincidencias exa
```

	Entrenamiento	Validación	Prueba
Blackhole	23,67	22,07	21,15
Flooding	10,33	10,35	10,34
Normal	22,79	23,46	23,15
Sybil	20,59	21,65	21,27
Wormhole	22,62	22,47	24,09
Filas	73267	17802	22883
% del total	64,3	15,62	20,08
Bloques	78	19	24

Interpretación. Tras eliminar las filas repetidas quedan 113 952 flujos, repartidos en bloques de 1000 filas, porque en ese rezago la ρ mediana entre filas de la misma clase ya cayó a 0,122, sin dejar grupos tan grandes que la estratificación pierda sentido. Ningún bloque aparece en dos particiones. La proporción de clases difiere entre particiones ($p < 0,001$), con un efecto de V de Cramér = 0,019 y una brecha máxima de 1,37 puntos porcentuales frente al promedio. En los atributos, la mayor distancia de Kolmogorov-Smirnov entre entrenamiento y prueba es de 0,128, en `TxPacketRate/s`.

Así mismo, el 1,8 % de las filas de prueba tiene un casi duplicado en entrenamiento y hay 0 coincidencias exactas, de modo que entre particiones casi no quedan flujos gemelos. Hay que tener en cuenta que eso no garantiza que una partición contenga solo simulaciones nuevas, porque el archivo no las identifica.

Por lo anterior, se conserva esta partición y se congela la prueba con su suma SHA-256.

Decisión de preprocesamiento. Partición estratificada por clase y agrupada por bloques, de 64, 16 y 20 %, con semilla 42, que se guarda en un archivo para reutilizarla en los Entregables 2 y 3.

Impacto en la arquitectura. Ninguno directo; todas las arquitecturas se comparan sobre las mismas particiones.

Riesgos. Con pocos bloques por clase, la composición de la prueba depende de qué bloques le tocaron. Por eso, la variabilidad entre semillas no incluye la de la partición, lo que se declara en la sección 7.

▶ **Mostrar el código de la celda**

Partición de validación	F1 macro de referencia (boosting)
Aleatoria estratificada, como en la literatura	0,9555
Agrupada por bloques de 1000 filas	0,8866

Filas por bloque	Bloques	F1 macro medio	Desviación entre particiones
100	948	0,9468	0,0031
500	193	0,8926	0,0577
1000	97	0,8913	0,0669
2000	60	0,8979	0,0561
5000	25	0,7638	0,1682

Interpretación. Con el mismo modelo de referencia y los mismos atributos, el F1 macro de validación es de 0,9555 con partición aleatoria y de 0,8866 con partición agrupada, una diferencia de 0,0689. Al variar el largo del bloque dentro de entrenamiento y validación, con tres particiones por largo, el F1 macro medio va de 0,947 a 0,764, y la desviación entre particiones llega a 0,168. Con los bloques más largos, esa desviación crece porque quedan muy pocos bloques y una partición puede quedarse casi sin una clase, y por eso no se usan.

Esto quiere decir que parte del desempeño de una partición aleatoria proviene de reconocer flujos vecinos en el archivo, y esa magnitud supera ampliamente el margen $\delta = 0{,}01$. Por lo tanto, los valores de la literatura y los de este trabajo no son directamente comparables si la literatura usa partición aleatoria, lo que se examina en la sección 3.4.

En consecuencia, se decide reportar el baseline con la partición agrupada, por ser la que estima mejor la generalización a tráfico no visto.

Decisión de preprocesamiento. No agrega transformaciones: se conserva la partición por bloques definida arriba, y todo ajuste se hace solo con entrenamiento.

Impacto en la arquitectura. Ninguno en la estructura. Sí cambia la selección, porque las arquitecturas y los hiperparámetros se comparan solo con validación agrupada; una partición aleatoria favorecería a los modelos que reconocen flujos vecinos.

Riesgos. Esta diferencia se mide con un solo modelo de referencia y en validación, de modo que no se traslada exacta a las redes. Además, con los bloques más largos una partición puede quedarse casi sin una clase, lo que aumenta la varianza y es otra razón para no usarlos.

2.1.4 Sesgos y desplazamientos de distribución

En esta parte se revisan los sesgos de representación, temporales, espaciales y de dominio; luego se mide si algún atributo funciona como atajo (Geirhos et al., 2020), es decir, si reconoce la clase sin decir nada del comportamiento del tráfico, y finalmente un clasificador adversarial intenta distinguir las particiones entre sí, de modo que un AUC cercano a 0,5 indica que no hay desplazamiento de covariables (Lopez-Paz y Oquab, 2017).

▶ **Mostrar el código de la celda**

Atributos	F1 macro, partición aleatoria	F1 macro, partición agrupada
Solo FlowID	0,9260	0,8593
Solo direcciones IP	0,5920	0,5280
Solo puertos	0,2006	0,2087
Solo atributos de tráfico y desempeño	0,9555	0,8866

SrcPort	9	654
label		
Blackhole	11248	14862
Flooding	19726	0
Normal	11294	14878
Sybil	24077	0
Wormhole	25494	592

Interpretación. En sesgos de representación, las cinco clases tienen entre 16,1 % y 21,4 % de los flujos, sin subpoblaciones que se puedan comparar dentro de cada clase. En sesgos temporales y espaciales, el archivo no trae marcas de tiempo ni coordenadas, de modo que solo se puede auditar el orden de las filas, como se hizo en la sección 2.1.3. En cuanto al dominio, todos los flujos provienen de un solo simulador y de una sola configuración de radio y enrutamiento.

En atajos, `FlowID`, que es solo un índice, alcanza por sí solo un F1 macro de 0,926 con partición aleatoria, y baja a 0,859 con partición agrupada. Las direcciones IP alcanzan 0,592 y 0,528, y 51 direcciones de origen aparecen en una sola clase. Los puertos alcanzan 0,201, porque el puerto 654, el del enrutamiento AODV, no aparece en Flooding ni en Sybil.

Esto significa que estos atributos reconocen el escenario simulado y no el comportamiento del ataque. Un modelo que los use obtendría un resultado alto que no se traslada a otra red, donde los índices, las direcciones y la mezcla de tráfico serían otros.

Decisión de preprocesamiento. Se excluyen `FlowID`, `SrcAddr`, `DstAddr`, `SrcPort` y `DstPort`, además de `Protocol`, que es constante. Quedan 16 atributos de tráfico y desempeño, los mismos que recomiendan los autores del conjunto de datos en su repositorio (Zeng, 2025), aunque su artículo considera útiles las direcciones para detectar ataques de identidad (Zeng et al., 2025).

Impacto en la arquitectura. La entrada es un vector de 16 valores numéricos, sin variables categóricas, de modo que no se necesitan capas de *embedding* para entidades.

Riesgos. Los atributos de tráfico también pueden reflejar la configuración de cada escenario, como la duración de la simulación o la tasa de envío. Esa parte no se puede separar del ataque con estos datos, y se declara como amenaza a la validez externa en la sección 5.5. Además, `FlowID` conserva 0,859 con partición agrupada, a solo 0,027 de la referencia con atributos de tráfico, porque los tramos de una sola clase, de hasta 19 180 filas, son más largos que el bloque, y muchos bloques de validación quedan rodeados de bloques de entrenamiento de la misma clase. Esto quiere decir que la partición no impide reconocer el tramo del archivo, y un modelo cuyos atributos delaten ese tramo conservaría parte de esa ventaja.

▶ **Mostrar el código de la celda**

```
Clasificador adversarial, AUC: entrenamiento frente a validación 0.889 | entrenam
```

Atributo	KS	p	p Holm
TxPacketRate/s	0,128	2,30e-251	3,67e-250
TxPackets	0,118	1,79e-212	2,68e-211
TxByteRate/s	0,107	6,04e-175	8,45e-174
TxBytes	0,104	1,20e-165	1,56e-164
RxPacketRate/s	0,104	4,06e-163	4,87e-162
AverageHopCount	0,101	3,83e-154	4,22e-153

Interpretación. El clasificador adversarial obtiene un AUC de 0,889 al separar entrenamiento de validación y de 0,848 al separar entrenamiento de prueba. La mayor distancia de Kolmogorov-Smirnov es de 0,128.

Esto significa que hay un desplazamiento de covariables entre particiones, esperable al partir por bloques, porque cada partición reúne tramos distintos del archivo. Hay que tener en cuenta que, con particiones de este tamaño, cualquier diferencia pequeña alcanza significancia, de modo que el AUC, como tamaño de efecto, pesa más que el valor p.

Decisión de preprocesamiento. Se conserva la partición, porque ese desplazamiento es justamente lo que un modelo enfrentará con tráfico de tramos no vistos, y se ajustan las transformaciones solo con entrenamiento.

Impacto en la arquitectura. Ninguno en la estructura. Este desplazamiento justifica regularizar y seleccionar con validación agrupada.

Riesgos. Esta prueba solo observa los atributos, no la relación entre atributos y clase, de modo que no descarta un cambio en esa relación.

2.2 EDA específico: clasificación con datos tabulares

En esta parte se aplica el bloque de datos tabulares del enunciado: la estructura de los atributos, su relación con el objetivo y la complejidad de la tarea. Desde aquí se trabaja solo con el conjunto de entrenamiento.

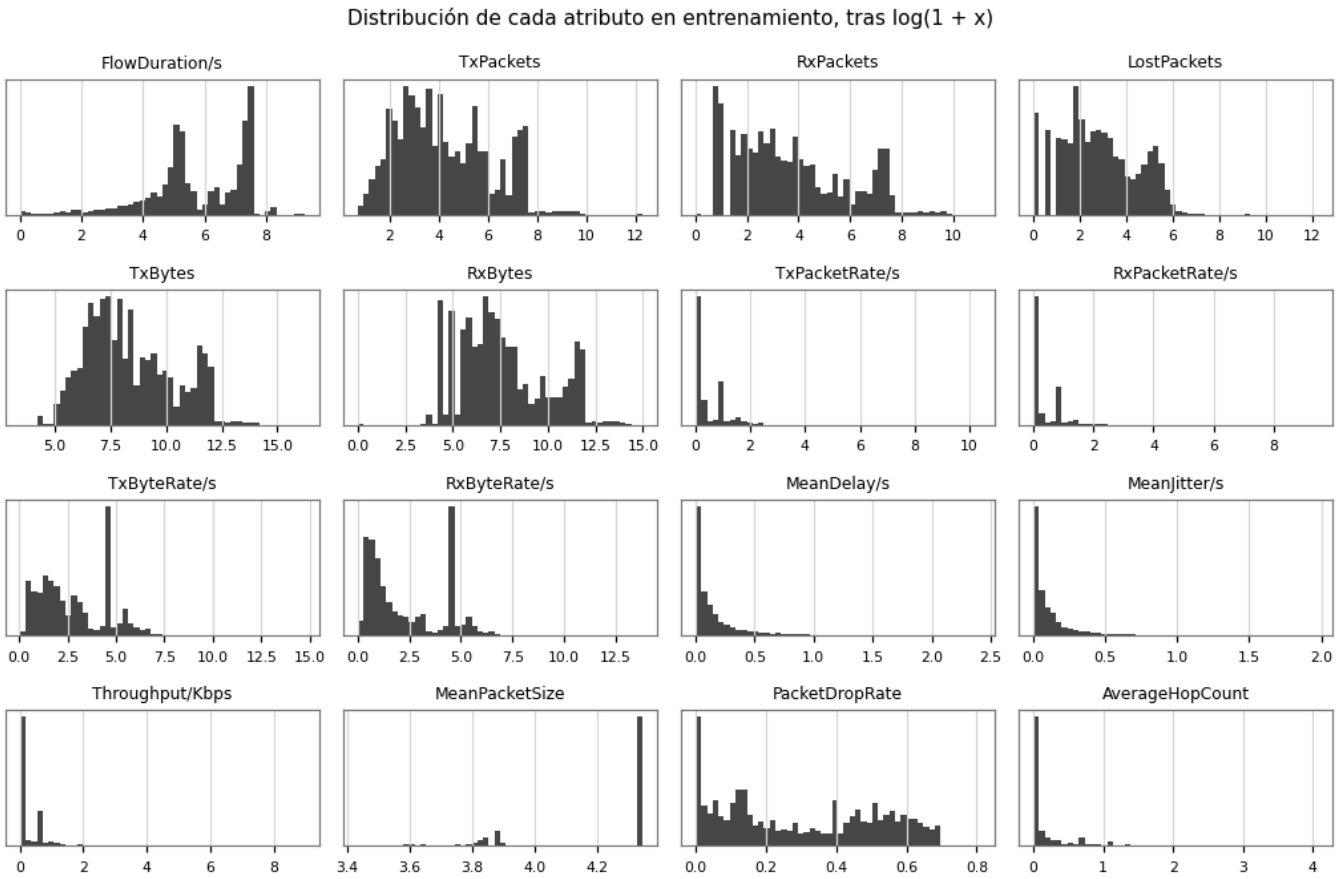
2.2.1 Estructura de atributos

Se tipifica cada columna y se describe su escala, su asimetría y sus valores atípicos, con el criterio del rango intercuartílico: un valor es atípico si queda a más de 1,5 veces ese rango por fuera de los cuartiles.

▶ **Mostrar el código de la celda**

Columna	Tipo	Cardinalidad	Uso
FlowID	Índice	122171	Excluida
SrcAddr	Categórica nominal	176	Excluida
SrcPort	Categórica de puerto	2	Excluida
DstAddr	Categórica nominal	218	Excluida
DstPort	Categórica de puerto	2	Excluida
Protocol	Categórica nominal	1	Excluida

Atributo	Valores distintos	Mediana	Máximo	Asimetría	Curtosis	% atípicos	Asimetría tras log	% a t
FlowDuration/s	42145	246,012	10591,800	1,857	11,829	0,925	-0,739	1
TxPackets	2050	50,000	220886,000	61,415	5337,110	16,889	0,441	0
RxPackets	2605	23,000	63653,000	12,181	285,215	18,785	0,560	0
LostPackets	641	13,000	212557,000	104,356	12029,881	15,938	0,217	0
TxBytes	8030	2888,000	11265075,000	41,199	2953,558	17,646	0,490	0
RxBytes	5762	1334,000	3246018,000	9,980	144,446	19,516	0,512	0
TxPacketRate/s	51405	0,194	33116,600	162,261	29079,142	6,044	2,645	2
RxPacketRate/s	55333	0,059	13115,800	145,827	22492,069	4,621	2,718	1
TxByteRate/s	51464	10,742	2516860,000	163,375	29366,831	5,798	0,475	0
RxByteRate/s	56521	3,436	996800,000	145,908	22509,087	4,399	0,649	0
MeanDelay/s	71378	0,064	10,168	6,924	82,397	11,207	3,086	9
MeanJitter/s	64608	0,048	6,240	6,625	67,939	10,033	3,510	8
Throughput/Kbps	56822	0,027	7974,400	145,908	22509,079	4,399	3,756	1
MeanPacketSize	26	76,000	76,000	-1,148	-0,549	0,000	-1,225	0
PacketDropRate	13910	0,320	1,250	0,349	-1,267	0,000	0,158	0
AverageHopCount	8559	0,025	58,000	17,631	1025,958	16,707	2,449	1



Interpretación. Las 6 columnas de conexión son categóricas o índices, y ya se excluyeron; los 16 atributos restantes son numéricos. Su asimetría es extrema: la mayor es de 163,4, en `TxByteRate/s`, y la mediana de esa asimetría entre atributos es de 29,4, con una mediana de 8,0 % de valores atípicos. Tras aplicar $\log(1 + x)$, la asimetría mediana baja a 0,54 y los atípicos a 0,2 %.

Esto significa que unos pocos flujos muy grandes dominarían el gradiente si se usaran las escalas originales, cuyos máximos llegan a 11 265 075. Además, varios histogramas tienen más de una moda, lo que apunta a mezclas de tipos de flujo.

Por tal motivo, se opta por transformar con $\log(1 + x)$, que conserva los ceros, y estandarizar después con la media y la desviación de entrenamiento.

Decisión de preprocesamiento. Recorte de `PacketDropRate` a 1, $\log(1 + x)$ en todos los atributos y estandarización ajustada solo con entrenamiento. No se eliminan atípicos, porque son flujos válidos y pueden ser la huella del ataque.

Impacto en la arquitectura. Con la entrada en escala comparable, un perceptrón multicapa no necesita capas de normalización para arrancar. No hay variables categóricas que codificar.

Riesgos. Ajustar la transformación con todo el archivo sería una fuga, y por eso solo se usa entrenamiento. Los atípicos pueden seguir pesando en modelos sin regularización.

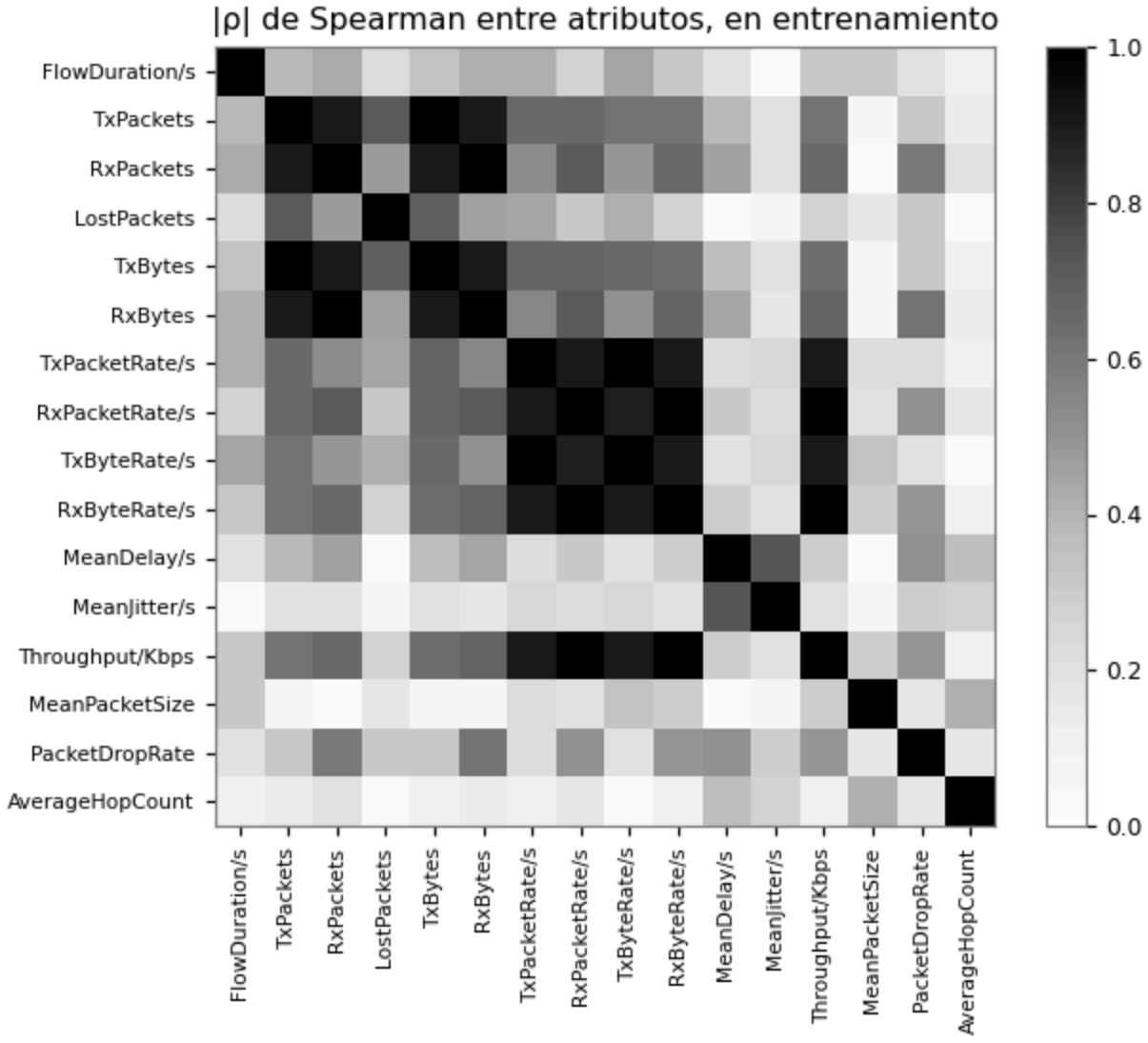
2.2.2 Relación entre atributos y objetivo

Se miden las correlaciones lineales y de rango entre atributos, la información mutua de cada atributo con la clase, para captar relaciones no lineales, y la multicolinealidad, con el factor de inflación de varianza, $VIF_j = 1/(1 - R_j^2)$, y el número de condición de la matriz de correlación.

▶ **Mostrar el código de la celda**

Atributo	Información mutua (nats)	VIF
TxByteRate/s	0,982	449,6
FlowDuration/s	0,934	82,6
TxPacketRate/s	0,932	24,8
RxByteRate/s	0,853	367,7
Throughput/Kbps	0,853	192,0
TxBytes	0,843	2,209,3
RxPacketRate/s	0,825	289,8
PacketDropRate	0,817	9,4
RxBytes	0,725	616,6
TxPackets	0,639	2,156,6
RxPackets	0,524	516,9
AverageHopCount	0,405	1,8
MeanJitter/s	0,343	2,3
LostPackets	0,316	6,3
MeanPacketSize	0,297	25,0
MeanDelay/s	0,231	2,7

|r| de Pearson	|ρ| de Spearman	Atributo	Atributo 2
0,9940	0,9927	RxPackets	RxBytes
0,9934	0,9899	RxPacketRate/s	Throughput/Kbps
0,9928	0,9907	TxPackets	TxBytes
0,9347	0,9112	TxPackets	RxPackets
0,9313	0,9000	RxPackets	TxBytes
0,9286	0,9028	TxBytes	RxBytes
0,9212	0,9059	TxByteRate/s	RxByteRate/s
0,9207	0,8990	TxPackets	RxBytes



Interpretación. El atributo con más información sobre la clase es `TxByteRate/s`, con 0,982 nats, y el de menos es `MeanDelay/s`, con 0,231 nats, frente a una entropía de la clase de 1,604 nats. En multicolinealidad, el VIF llega a 2 209, el número de condición es de 190 y 0 pares tienen $|r|$ mayor que 0,999.

Esto se explica por la construcción de los atributos: las tasas son cocientes de los conteos sobre la duración, y, en la escala original, `Throughput/Kbps` equivale a `RxByteRate/s` en otras unidades. Es decir, varios atributos repiten la misma información, y los coeficientes de un modelo lineal no serían interpretables. Así mismo, ningún atributo alcanza por sí solo la entropía de la clase, de modo que la clasificación requiere combinarlos.

En ese sentido, se conservan los 16 atributos para los modelos de Deep Learning, porque la literatura usa los mismos y porque una red con regularización tolera la colinealidad, aunque sí se usa regularización L2 en el modelo lineal de referencia.

Decisión de preprocesamiento. No se eliminan atributos por colinealidad. No se detectan atributos derivados de la etiqueta entre los 16 que se conservan; los candidatos a fuga, `FlowID`, IP y puertos, ya se excluyeron en la sección 2.1.4.

Impacto en la arquitectura. La redundancia reduce la dimensión efectiva, de modo que no se justifican capas muy anchas y conviene la regularización. Además, el orden de los atributos no tiene significado físico, lo cual cuestiona el sesgo inductivo de los modelos que leen el vector como una secuencia.

Riesgos. La información mutua se estima sobre una muestra de 30 000 flujos y no capta interacciones entre atributos.

2.2.3 Complejidad de la tarea

Se ajusta una regresión logística con regularización L2 para estimar la separabilidad lineal y dimensionar el margen de mejora disponible. Esta es una referencia de contexto, no un modelo base, y el boosting de la sección 2.1.3 se usa como referencia de lo que alcanza un modelo no lineal sin Deep Learning.

▶ **Mostrar el código de la celda**

Clase	F1 logístico	F1 boosting
Blackhole	0,8283	0,8693
Flooding	0,8648	0,8144
Normal	0,9523	0,9901
Sybil	0,9652	0,8939
Wormhole	0,7905	0,8696

Interpretación. En validación, la regresión logística obtiene un F1 macro de 0,8803, con 0,8875 en entrenamiento, y el boosting obtiene 0,8875. Por clase, el modelo lineal es mejor en Flooding y Sybil, mientras que el boosting lo es en Blackhole, Normal y Wormhole. La clase más difícil para el modelo lineal es Wormhole, con F1 de 0,791.

Esto significa que la tarea es en buena parte separable con una frontera lineal sobre los logaritmos, y que, en estos bloques de validación y sin ajustar el boosting, la no linealidad aporta 0,0072, menos que el margen $\delta = 0{,}01$. Es decir, el costo de una red no se justifica de antemano, y hay que comprobarlo sobre la prueba.

Por lo anterior, conviene usar el modelo logístico como piso de comparación de los modelos base en la sección 5.3.

Decisión de preprocesamiento. Se mantiene la transformación definida en la sección 2.2.1.

Impacto en la arquitectura. La distancia de 0,0072 entre el modelo lineal y el boosting indica que basta con pocas capas ocultas para capturar la no linealidad, y no justifica por sí sola una arquitectura profunda.

Riesgos. El boosting es una referencia de contexto; no compite como modelo base y sus hiperparámetros no se ajustaron.

2.2.4 Implicaciones clave

Las decisiones de este bloque condicionan los modelos de la sección 4, y se resumen en cuatro puntos:

Aspecto	Decisión	Sustento
Diseño del perceptrón multicapa	Pocas capas ocultas, de ancho moderado y con regularización	La dimensión efectiva es baja y el margen no lineal es acotado (secciones 2.2.2 y 2.2.3)
Variables categóricas	No se usan	Las columnas categóricas funcionan como atajos (sección 2.1.4)
Función de pérdida	Entropía cruzada con pesos de clase	Al eliminar las filas repetidas, Flooding queda como clase minoritaria (secciones 2.1.2 y 2.2.5)
Partición	Estratificada por clase y agrupada por bloques	Las filas vecinas no son independientes (sección 2.1.3)

2.2.5 Preparación de los datos para entrenar

Con la transformación definida, los datos se guardan listos para entrenar, con la media y la desviación de entrenamiento y los pesos de clase, $w_c = n/(Kn_c)$, que hacen que cada clase aporte lo mismo a la pérdida total, porque $n_c w_c = n/K$.

▶ **Mostrar el código de la celda**

Clase	Entrenamiento	Peso
Blackhole	17342	0,845
Flooding	7569	1,936
Normal	16698	0,878
Sybil	15085	0,971
Wormhole	16573	0,884

```
Datos preparados: 16 atributos | entrenamiento 73,267 | validación 17,802 | prueb
```

3. Revisión del estado del arte y extracción de los tres modelos base

En esta sección se documenta la búsqueda de literatura, se identifican los modelos de Deep Learning más básicos que los estudios emplean como referencia al detectar intrusiones en redes de drones y se justifica la elección de los tres modelos base. Su resultado central es la tabla de frecuencia, de la cual salen esos tres modelos, y esto es primordial porque el baseline de todo el proyecto depende de esa elección.

3.1 Estrategia de búsqueda

Cadenas de búsqueda. Se usaron tres cadenas booleanas sobre título y resumen, o el campo más cercano de cada base, para publicaciones de 2019 a 2026, es decir, los últimos ocho años. La primera es general; la segunda se concentra en los ataques de red de este proyecto, y la tercera busca el uso directo del conjunto de datos.

Cadena	Consulta
C1	(drone OR UAV OR «unmanned aerial» OR FANET) AND («intrusion detection» OR «attack detection») AND («deep learning» OR «neural network» OR CNN OR LSTM OR MLP)
C2	(drone OR UAV OR «unmanned aerial» OR FANET) AND (swarm OR «ad hoc» OR «network traffic») AND (blackhole OR wormhole OR sybil OR flooding OR cyberattack) AND («deep learning» OR «neural network» OR CNN OR LSTM OR MLP)
C3	UAVIDS OR «UAVIDS-2025»

Palabras clave, sinónimos y variantes. Cada cadena combina conceptos con AND y, dentro de cada concepto, sinónimos y variantes terminológicas con OR. Las comillas buscan la frase exacta, y los términos van en inglés porque es el idioma en que las bases indexan títulos y resúmenes.

Concepto	Palabras clave, sinónimos y variantes	Cadenas
Plataforma	drone, UAV, «unmanned aerial», FANET	C1 y C2
Tarea	«intrusion detection», «attack detection»	C1
Contexto de red	swarm, «ad hoc», «network traffic»	C2
Ataques del conjunto de datos	blackhole, wormhole, sybil, flooding, cyberattack	C2
Método	«deep learning», «neural network», CNN, LSTM, MLP	C1 y C2
Conjunto de datos	UAVIDS, «UAVIDS-2025»	C3

Cadena exacta por base. La cadena booleana es la misma en todas las bases, y cambian el campo y el filtro de años, porque cada base los define a su manera. En la tabla, *S* representa la cadena C1, C2 o C3 tal como aparece arriba.

Base	Cadena ejecutada	Campo	Periodo
OpenAlex	*S* en el filtro `title_and_abstract.search`	Título y resumen	`from_publication_date:2019-01-0` hasta el día de la consulta
arXiv	*S*, con el prefijo `abs:` en cada término o frase, por ejemplo `abs:drone OR abs:UAV OR abs:"unmanned aerial"`	Resumen	Se conservan las publicaciones desde 2019
IEEE Xplore	*S* en *Command Search*	Todos los metadatos	Filtro de año de publicación de 2019 2026
Scopus	`TITLE-ABS-KEY(` *S* `) AND PUBYEAR > 2018 AND PUBYEAR < 2027`	Título, resumen y palabras clave	En la misma cadena
Web of Science	`TS=(` *S* `) AND PY=(2019-2026)`, en la colección principal	Tema	En la misma cadena

Bases de datos. Las consultas se ejecutaron en OpenAlex y arXiv el 3 de noviembre de 2026, y en IEEE Xplore, Scopus y Web of Science el 4 de noviembre de 2026, con el acceso de la biblioteca de la Universidad del Norte. Como cada base define sus campos de otra forma, Scopus busca en título, resumen y palabras clave (`TITLE-ABS-KEY`), Web of Science en el campo de tema (`TS`) e IEEE Xplore en todos los metadatos, de modo que sus conteos pueden ser algo más amplios. Las cadenas listas para copiar están en `literatura/consultas_wos_scopus_ieee.md` y, para OpenAlex y arXiv, en `literatura/buscar_literatura.py`.

Base	C1	C2	C3	Estado
OpenAlex	323	40	220	Exportada
arXiv	11	3	29	Exportada; 28 registros de C3 dentro del periodo
IEEE Xplore	481	40	81	Exportada
Web of Science	132	11	16	Exportada
Scopus	523	18	26	Exportada

Verificación de cobertura de sinónimos. Una vez hecha la búsqueda, se comprobó si las cadenas omitían variantes terminológicas. El 5 de noviembre de 2026 se ejecutaron en OpenAlex versiones ampliadas de C1 y C2, que agregan UAS, «unmanned aircraft», «flying ad hoc», «Internet of Drones», IoD y quadcopter en plataforma; «anomaly detection», IDS, «cyberattack detection» y «threat detection» en tarea; «black hole» y DDoS en ataques, y GRU, RNN, transformer y autoencoder en método. C1 ampliada devolvió 851 registros y C2 ampliada 49, y de ellos 523 no estaban entre los 1109 registros depurados. El mismo filtro automático dejó 422, y el cribado por título y resumen halló 23 elegibles o dudosos; la mayoría de los excluidos detecta anomalías en imágenes, video o datos de vuelo, y no intrusiones en la red.

Esto significa que la cadena original sí tenía un vacío: de los 23 registros, 16 se habían perdido por el concepto de método, sobre todo por autoencoder y transformer, y 7 por el de tarea, mientras que los sinónimos de plataforma no aportaron ninguno. Sin embargo, el vacío es acotado, porque equivale al 10 % de los 228 elegibles de las bases.

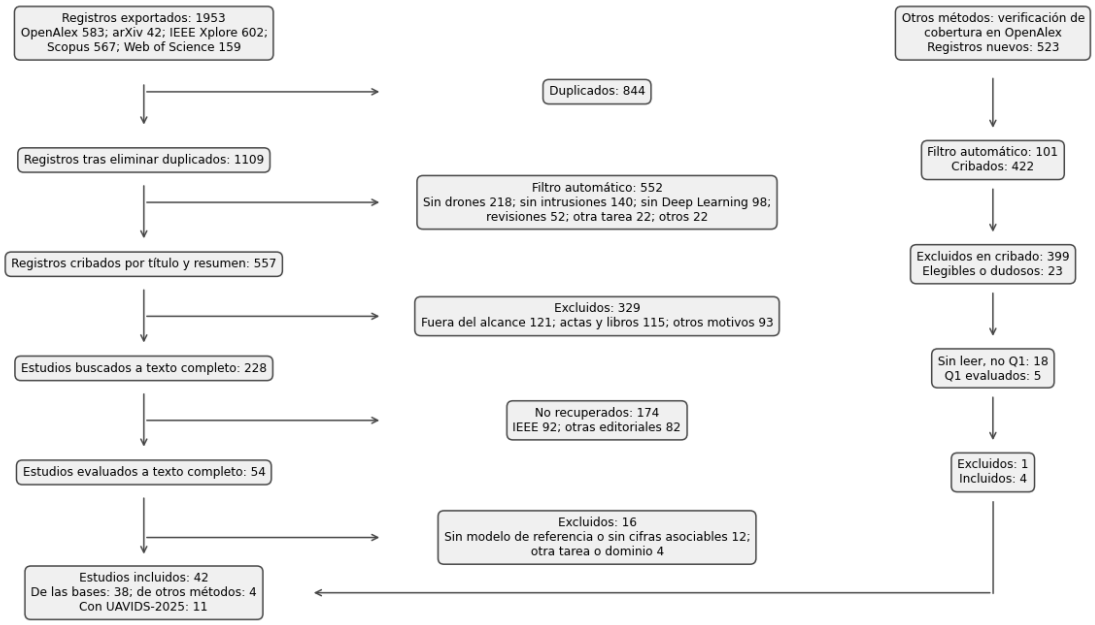
Por tal motivo, se opta por leer a texto completo los 5 que están en revistas Q1, de los cuales se incluyen 4, y los otros 18 quedan sin leer. La verificación no se repitió en Scopus, Web of Science ni IEEE Xplore. Las cadenas ampliadas y sus conteos están en `literatura/consultas_wos_scopus_ieee.md`, y la decisión de cada registro, en `literatura/cribado_cobertura_sinonimos.csv`.

Criterios de inclusión y exclusión. Se incluye un estudio si detecta o clasifica intrusiones o ataques en redes o sistemas de drones a partir de datos tabulares de tráfico o del sistema, si usa Deep Learning, si reporta métricas verificables y si compara contra al menos un modelo de Deep Learning de referencia. Se excluye si no describe lo suficiente el modelo de referencia o el protocolo experimental, si no es un estudio primario, como las revisiones y los artículos solo de

datos, y si trabaja con cámaras, señales de radio o solo suplantación de GPS. La sección 3.2 declara cómo se aplicó en la práctica el criterio de descripción suficiente.

Trazabilidad del proceso. El diagrama de flujo sigue el formato PRISMA (Page et al., 2021). Las decisiones de cada registro, con su motivo, están en `literatura/registros_depurados.csv`, `literatura/cribado_bases_nuevas.csv` y, para la verificación de cobertura, `literatura/cribado_cobertura_sinonimos.csv`. Los estudios de esa verificación entran por la rama de otros métodos del diagrama.

▶ **Mostrar el código de la celda**



Interpretación. De 1953 registros exportados, 844 son duplicados, el filtro automático retira 552 y el cribado por título y resumen retira 329. Quedan 228 estudios elegibles, de los cuales se pudieron evaluar 54, es decir, el 24 %, y se incluyen 38. La verificación de cobertura agrega 4 por la rama de otros métodos, para un total de 42. El artículo que presenta el conjunto de datos no menciona Deep Learning en su resumen y el filtro automático lo retiró; se rescató a mano. Ese artículo reporta los valores, pero no describe las arquitecturas, que están en el código público de los autores (Zeng, 2025).

Hay que tener en cuenta que esta lectura parcial introduce un sesgo: quedan sin evaluar 174 estudios, 92 de ellos de IEEE. Además, la lectura no fue uniforme, porque se priorizaron los de mayor cuartil y los de acceso abierto: se evaluaron 28 de los 68 elegibles Q1 y 26 de los 160 restantes, y en la verificación de cobertura se leyeron los 5 Q1 y ninguno de los 18 restantes. Por tanto, la tabla de frecuencia no se puede leer como un censo de la literatura del tema.

Por lo anterior, conviene leer los conteos como una estimación de la práctica en el área, con mayor peso de las revistas Q1 y del acceso abierto.

Quién realizó la selección. Al ser un trabajo de autor único, hubo un solo revisor y no hubo desacuerdos que resolver, lo cual es una limitación frente a un cribado doble. El cribado y la extracción se hicieron con apoyo de un asistente de inteligencia artificial (Claude, de Anthropic) bajo la supervisión del autor. El autor descargó con el acceso de la biblioteca los textos Q1 que no estaban en acceso abierto. La extracción se apoyó en la búsqueda dirigida de las tablas y los pasajes de comparación de cada texto, no en su lectura íntegra, de modo que un dato que no se halló queda como no reportado o no identificado, aunque podría estar en una figura o en un material suplementario.

Evaluación de la calidad de los estudios. Para cada estudio se valoró la partición, las repeticiones, la dispersión, las pruebas estadísticas y el riesgo de fuga. Solo 2 de los 42 estudios agrupan filas relacionadas, por sesión o por nodo; 31 no reportan repeticiones, solo 8 reportan dispersión y 6 aplican alguna prueba.

En cuanto a la calidad de las fuentes, el enunciado pide preferir literatura Q1. Según el cuartil SJR 2025 de SCImago (2026), que se calcula con datos de Scopus, 26 estudios son Q1, 5 Q2, 2 Q3 y 1 Q4, mientras que 8 son conferencias, preprints o fuentes sin cuartil, entre ellas el artículo de los autores del conjunto de datos. Hay que tener en cuenta que el cuartil describe a la revista en 2025, no a cada artículo ni al año en que se publicó.

Estudio	Cuartil SJR	Partición	Repeticiones	Dispersión	Prueba estadística	Riesgo fuga
Abbas et al. (2026)	Q1	no identificada	no reportadas	no	ninguna	No evalua
Abdulhammed et al. (2026)	Q2	aleatoria 80/20	no reportadas	no	ninguna	Alto: n agrupa relacic
Aburasain (2024)	Q1	aleatoria 70/30	no reportadas	no	ninguna	Alto: n agrupa relacic
Alharthi (2024)	Q2	aleatoria 70/30	no reportadas	no	t de Student entre los dos mejores	Alto: n agrupa relacic
Alissa et al. (2022)	Q1	70/30	no reportadas	no	ninguna	Alto: n agrupa relacic
Alotaibi et al. (2024)	Q1	aleatoria 80/20 y 70/30	no reportadas	no	ninguna	Alto: n agrupa relacic
Alruwaili y Tunc (2026)	no está en SJR	aleatoria 80/20	no reportadas	no	ninguna	Alto: n agrupa relacic
Alsaraireh et al. (2026)	Q1	estratificada 70/15/15	varias ejecuciones	sí	ninguna	Alto: n agrupa relacic
Alshammari et al. (2024)	Q2	aleatoria 80/20	no reportadas	no	ninguna	Alto: n agrupa relacic
Alshamrani y Alghamdi	Q1	un tipo de ataque fuera	no reportadas	sí	t pareada, $p < 0,001$	Alto: n agrupa

Estudio	Cuartil SJR	Partición	Repeticiones	Dispersión	Prueba estadística	Riesgo fuga
(2026)		en cada evaluación				relacio
Arulalan et al. (2023)	no está en SJR	aleatoria 70/30	no reportadas	no	ninguna	Alto: n agrupa relacio
Ashraf et al. (2023)	Q1	aleatoria 70/30	no reportadas	no	ninguna	Alto: n agrupa relacio
Attaullah et al. (2026a)	Q1	entrenamiento y prueba, no identifica agrupación	10	sí	t de Student	Alto: n agrupa relacio
Attaullah et al. (2026c)	Q1	5 pliegues estratificados, con 70/15/15 en cada uno	5 pliegues	no	ninguna	Alto: n agrupa relacio
Attaullah et al. (2026b)	Q1	5 pliegues estratificados	5 pliegues	no	ninguna	Alto: n agrupa relacio
Babu et al. (2024)	Q1	aleatoria 80/20 y 70/30	no reportadas	no	ninguna	Alto: n agrupa relacio
Bakiri et al. (2026)	Q1	por nodo de origen en UAVIDS-2025; estratificada 72/8/20 en UAV-NIDD	no reportadas	no	ninguna	Menor agrupa sesión nodo

Estudio	Cuartil SJR	Partición	Repeticiones	Dispersión	Prueba estadística	Riesg fuga
Balgoon (2026)	no está en SJR	aleatoria estratificada, 72 % de entrenamiento	no reportadas	no	ninguna	Alto: n agrup relacic
Batur Şahin (2026)	Q1	estratificada 70/15/15	5 semillas	sí	ninguna	Alto: n agrup relacic
Biswas et al. (2026)	Q1	estratificada por sesión de captura y dron	5 semillas	sí	ninguna	Menor agrup sesión nodo
Cengiz et al. (2024)	Q1	5 pliegues	5 pliegues	no	ninguna	Alto: n agrup relacic
Gharami y Moni (2025)	no está en SJR	aleatoria 80/20	no reportadas	no	ninguna	Alto: n agrup relacic
Hadi et al. (2024)	Q1	80/20	no reportadas	no	ninguna	Alto: n agrup relacic
Hadi et al. (2026)	Q1	no identificada	no reportadas	no	ninguna	No evalua
Hakeem et al. (2025)	Q2	aleatoria 70/30	no reportadas	no	ninguna	Alto: n agrup relacic

Estudio	Cuartil SJR	Partición	Repeticiones	Dispersión	Prueba estadística	Riesg fuga
Hamad et al. (2025)	Q3	no identificada	no reportadas	no	ninguna	No evalua
Junaid et al. (2026)	Q1	estratificada 70/15/15, con SMOTE solo en entrenamiento	no reportadas	no	ninguna	Alto: n agrupa relacic
Katbar et al. (2024)	Q3	aleatoria 80/20	no reportadas	no	ninguna	Alto: n agrupa relacic
Katib et al. (2025)	Q1	80/20 y 70/30	no reportadas	no	ninguna	Alto: n agrupa relacic
Li et al. (2026a)	Q1	aleatoria 90/10	no reportadas	no	ninguna	Alto: n agrupa relacic
Li et al. (2026b)	Q1	90/10	10	sí	ninguna	Alto: n agrupa relacic
Menssouri et al. (2024)	sin cuartil	aleatoria	no reportadas	no	ninguna	Alto: n agrupa relacic
Mohamed et al. (2026)	Q1	70/15/15; además, un ataque fuera	no reportadas	sí	$p < 0,01$, sin nombrar la prueba en los pasajes revisados	Alto: n agrupa relacic

Estudio	Cuartil SJR	Partición	Repeticiones	Dispersión	Prueba estadística	Riesgo fuga
Negm et al. (2024)	Q1	80/20 y 70/30	no reportadas	no	ninguna	Alto: n agrupa relacio
Ozsoy et al. (2026)	Q4	aleatoria 80/20, con 176 clientes por IP de origen	no reportadas	no	ninguna sobre modelos	Alto: n agrupa relacio
SureshBabu y Geetha (2026)	no está en SJR	no identificada	no reportadas	no	ninguna	No evalua
Tan et al. (2019)	Q1	aleatoria; usa 10 % del archivo	no reportadas	no	ninguna	Alto: n agrupa relacio
Wang et al. (2026)	Q1	aleatoria, con validación cruzada	5 semillas	sí	t de Student pareada	Alto: n agrupa relacio
Wu et al. (2024)	Q1	entrenamiento con parte de los ataques; prueba más de diez veces mayor	no reportadas	no	ninguna	Alto: n agrupa relacio
Zarkadis y Douligeris (2026)	no está en SJR	aleatoria estratificada 80/20, con 10 pliegues en entrenamiento	10 pliegues	no	permutación sobre atributos, no sobre modelos	Alto: n agrupa relacio

Estudio	Cuartil SJR	Partición	Repeticiones	Dispersión	Prueba estadística	Riesg fuga
Zeng et al. (2025)	no está en SJR	aleatoria estratificada 80/20	no reportadas	no	ninguna	Alto: n agrup relacic
Zhou et al. (2026)	Q2	5 pliegues	5 pliegues	no	ninguna	Alto: n agrup relacic

3.2 Identificación de los tres modelos base de Deep Learning

Procedimiento de extracción. En cada estudio incluido se revisó qué modelos de Deep Learning se emplean como referencia, se registró su arquitectura y configuración cuando se reportan, y se normalizó cada modelo por familia. Con esto se construye la tabla de frecuencia, se ordenan los candidatos por frecuencia y complejidad, se seleccionan tres y se verifica que no sean redundantes.

Tabla de síntesis. Los valores se transcriben como los reporta cada estudio, en su propia métrica y con su propio conjunto de datos.

Referencia	Problema	Dataset	Modelos de Deep Learning de referencia	Métricas reportadas y valores
Abbas et al. (2026)	Detección de intrusiones en drones: flujos de red simulados	UAVIDS-2025	FL-CNN, FL-RNN y FL-MLP (CNN; RNN; DNN/MLP)	Precisión en Wormhole: FL-CNN 0,70, FL-RNN 0,78, FL-MLP 0,78 y FL-GCN 0,81; en Blackhole: 0,90, 0,92, 0,94 y 0,92
Abdulhammed et al. (2026)	Detección de intrusiones en drones: flujos de red simulados	UAVIDS-2025 y UAV-GCS	FFN de 128 y 64, LSTM de 64 y CNN 1D (DNN/MLP; LSTM; CNN)	En UAVIDS-2025, el texto da 97,65 % de exactitud para FFN y 100 % para LightGBM
Aburasain (2024)	Detección de intrusiones en drones: flujos de red	ToN-IoT y Edge-IIoTset	LSTM y GRU (LSTM; GRU)	Exactitud: LSTM de 96,20 a 96,95 %, GRU de 96,00 a 96,50 %
Alharthi (2024)	Detección de intrusiones en drones: flujos de red	KDD Cup 99, NSL-KDD, WSN-DS y CICIDS	CNN, LSTM y CNN-LSTM (CNN; LSTM; CNN-LSTM)	Exactitud en una de las tablas: CNN 89,10 %, LSTM 92,25 %, CNN-LSTM 95,11 %; propuesto 99,99 %
Alissa et al. (2022)	Detección de intrusiones en drones: flujos de red	8000 registros con las clases de NSL-KDD	MLP, junto con cinco modelos clásicos, tomados de un trabajo publicado (DNN/MLP)	Exactitud: propuesto 99,12 %, MLP 90,03 %
Alotaibi et al. (2024)	Detección de intrusiones	no identificado	IDBN y DPC-DBN (DBN)	Exactitud: IDBN 95,32 %, DPC-DBN

Referencia	Problema	Dataset	Modelos de Deep Learning de referencia	Métricas reportadas y valores
	en drones: flujos de red			94,39 %
Alruwaili y Tunc (2026)	Detección de intrusiones en drones: flujos de red	UNSW-NB15 y Edge-IIoT	DNN, CNN 1D y CNN 1D con LSTM (DNN/MLP; CNN; CNN-LSTM)	Mejora de exactitud de 0,83 a 4,98 % en UNSW-NB15
Alsaraireh et al. (2026)	Detección de intrusiones en drones: flujos de red	tres conjuntos UAV-IDS	CNN 1D como modelo base del detector (CNN)	Exactitud sin perturbar: 95,27 % en un conjunto y 78,93 % en UAV3-IDS
Alshammari et al. (2024)	Detección de intrusiones en drones: flujos de red	NSL-KDD y TON_IoT	LSTM-RNN y CNN (LSTM; CNN)	Exactitud en NSL-KDD: LSTM-RNN 95,91 %, CNN 86,54 %; propuesto 99,32 %
Alshamrani y Alghamdi (2026)	Detección de intrusiones en drones: flujos de red	UAV-NIDD	LSTM y FlowTransformer supervisados, como cota superior (LSTM; Transformer)	F1 supervisado: LSTM 94,5 ± 0,7 % y FlowTransformer 95,8 ± 0,6 %; propuesto 85,6 ± 0,8 % sin haber visto el ataque
Arulalan et al. (2023)	Detección de intrusiones en drones: flujos de red	CICIDS2017	CNN-LSTM, CNN y EDFFNN (CNN-LSTM; CNN; DNN/MLP)	No se pudieron asociar las cifras a cada modelo en el texto
Ashraf et al. (2023)	Detección de intrusiones	CICIDS2017 y otros	RNN, LSTM y Bi-LSTM de 64	Exactitud: Bi-LSTM 91,36 %, LSTM

Referencia	Problema	Dataset	Modelos de Deep Learning de referencia	Métricas reportadas y valores
	en drones: flujos de red		unidades, GRU y perceptrón (RNN; LSTM; BiLSTM; GRU; DNN/MLP)	91,00 %, RNN 90,16 %, GRU 89,13 %, perceptrón 85,02 %
Attaullah et al. (2026a)	Detección de intrusiones en drones: datos del sistema	T-ITS, ciberfísico de UAV	DNN (DNN/MLP)	Exactitud: DNN 96,43 ± 0,22 %; propuesto 98,74 ± 0,07 %
Attaullah et al. (2026c)	Detección de intrusiones en drones: datos del sistema	T-ITS	FFCNN y MC-DNN, tomados de trabajos publicados (CNN; DNN/MLP)	Exactitud del propuesto: 99,44 %
Attaullah et al. (2026b)	Detección de intrusiones en drones: flujos de red	conjunto multimodal de UAV	GAN-MCNN-DFS, ANN, CNN, XAI-DNN y SEMI-GRU, tomados de trabajos publicados (CNN; DNN/MLP; GRU)	Exactitud del propuesto: 99,49 %
Babu et al. (2024)	Detección de intrusiones en drones: flujos de red	no identificado	IDBN y DPC-DBN (DBN)	Exactitud: IDBN 95,32 %, DPC-DBN 94,39 %; propuesto 99,63 %

Referencia	Problema	Dataset	Modelos de Deep Learning de referencia	Métricas reportadas y valores
Bakiri et al. (2026)	Detección de intrusiones en drones: flujos de red simulados	UAVIDS-2025 y UAV-NIDD	KAN plana; DNN-KDQ y UAV-IDS-ConvNet, de trabajos publicados (KAN; DNN/MLP; CNN)	Exactitud en cinco clases en UAVIDS-2025: 92,50 %; KAN plana 93,35 %
Balgoon (2026)	Detección de intrusiones en drones: flujos de red simulados	UAVIDS-2025 y T-ITS	BiLSTM, GNN y autoencoder (BiLSTM; GNN; Autoencoder)	F1 macro: BiLSTM 77,87 %, GNN 94,60 %; propuesto 98,18 %
Batur Şahin (2026)	Detección de intrusiones en drones: registros de vuelo y tráfico MAVLink	UAV Attack Dataset, de IEEE DataPort	Clasificador feedforward solo, como línea base de la ablación (DNN/MLP)	Exactitud: clasificador 92,23 ± 0,3 % (F1 0,9198); propuesto centralizado 98,67 ± 0,2 % (F1 0,9856
Biswas et al. (2026)	Detección de intrusiones en drones: flujos de red	ISOT Drone Anomaly Detection	CNN 1D, MLP, GRU, LSTM y RNN (CNN; DNN/MLP; GRU; LSTM; RNN)	Exactitud del propuesto: 99,99 %
Cengiz et al. (2024)	Detección de intrusiones en drones: datos del sistema	UAV con ataques de GPS	Red de retropropagación, con y sin una capa adicional (DNN/MLP)	Exactitud: BPN 0,895, BPN con una capa más 0,9324
Gharami y Moni (2025)	Detección de intrusiones	UAV IDS, UKM IDS y otros	MLP-AE, L-MADE y FFCNN (Autoencoder; CNN)	Exactitud en UAV IDS: FFCNN 99,99 %, MLP-AE 98,81

Referencia	Problema	Dataset	Modelos de Deep Learning de referencia	Métricas reportadas y valores
	en drones: flujos de red			%; propuesto 99,99 %
Hadi et al. (2024)	Detección de intrusiones en drones: flujos de red	UAVIDS-2020, NF-UQ-NIDS-v2 y 5G-NIDD	DNN simple y DNN multinivel simple (DNN/MLP)	Exactitud en UAVIDS-2020: fusión 99,95 %, multinivel 98,91 %, DNN simple 94,78 %
Hadi et al. (2026)	Detección de intrusiones en drones: flujos de red	GCS-NIDD	TabNet, junto con LightGBM y BLS (TabNet)	Exactitud por modelo: TabNet 93,10 %, LightGBM 90,20 %, BLS 89,70 %; fusión 99,30 %
Hakeem et al. (2025)	Detección de intrusiones en drones: datos del sistema	datos ciberfísicos de GPS	MLP, CNN, LSTM y ResNet (DNN/MLP; CNN; LSTM; ResNet)	Exactitud: ResNet 90,99 %, MLP 89,49 %, CNN 86,36 %, LSTM 82,45 %
Hamad et al. (2025)	Detección de intrusiones en drones: flujos de red	CIC-IDS 2017	CNN-GRU y LSTM-CNN, tomados de trabajos publicados (CNN-LSTM; GRU)	Exactitud: CNN-GRU 98,73 %, LSTM-CNN 99,21 %; propuesto: F1 99,28 %
Junaid et al. (2026)	Detección de intrusiones en drones: flujos de red simulados	UAVIDS-2025	MLP, junto con cuatro modelos clásicos (DNN/MLP)	Binario: MLP exactitud 99,1 % y F1 99,5 %; propuesto 97,7 % y 98,5 %; XGBoost 99,6 % y 99,7 %

Referencia	Problema	Dataset	Modelos de Deep Learning de referencia	Métricas reportadas y valores
Katbar et al. (2024)	Detección de intrusiones en drones: flujos de red	no identificado	LSTM y CNN (LSTM; CNN)	Exactitud: LSTM 93 %, CNN 93 %; propuesto 95 %
Katib et al. (2025)	Detección de intrusiones en drones: flujos de red	InSDN	AlexNet, ResNet y ShuffleNet, tomados de trabajos publicados (CNN; ResNet)	Exactitud del propuesto: 99,88 %
Li et al. (2026a)	Detección de intrusiones en drones: flujos de red	UAV-IDS-2020	CNN 1D como modelo base del detector (CNN)	Tasa de éxito del ataque sobre la CNN: 93,61 %
Li et al. (2026b)	Detección de intrusiones en drones: flujos de red	UAV-NIDD y CICIoT2022	FS-Net, ET-BERT, YaTC, GraphDAPP, TFE-GNN y FNN (RNN; Transformer; GNN; DNN/MLP)	F1 en UAV-NIDD: TGE-ETD 99,97 ± 0,04 %, YaTC 99,96 ± 0,77 %, FNN 95,20 %, TFE-GNN 93,01 ± 2,14 %, MAML 71,09 %
Menssouri et al. (2024)	Detección de intrusiones en drones: flujos convertidos a matriz	UAV-IDS-2020	CNN 1D y CNN 1D con PCA (CNN)	CNN 1D: exactitud cerca de 75 % en modo BF; propuesto 99,99 %
Mohamed et al. (2026)	Detección de intrusiones en drones:	UAVIDS-2025, UNSW-NB15 y ET-BERT	DAE, CNN-GRU, GCN, GNN-Adv, TodyNet y GTCN-G	F1 en UAVIDS-2025: AD-GNN

Referencia	Problema	Dataset	Modelos de Deep Learning de referencia	Métricas reportadas y valores
	flujos de red simulados		(Autoencoder; CNN-LSTM; GNN)	0,962 ± 0,003, TodyNet 0,941
Negm et al. (2024)	Detección de intrusiones en drones: flujos de red	NSL-KDD	IDBN y DPC-DBN, junto con DRL-BWO, T-SID, un modelo de Deep Learning sin especificar y AK-NN, tomados de trabajos publicados (DBN)	Exactitud: propuesto 99,36 %, IDBN 95,32 %, DPC-DBN 94,39 %
Ozsoy et al. (2026)	Detección de intrusiones en drones: flujos de red simulados	UAVIDS-2025 y T-ITS	MLP compacto federado; DNN, CNN y LSTM de los autores del conjunto de datos (DNN/MLP; CNN; LSTM)	F1: FedAvg 93 %, FedProx 84 %; referencias de los autores: DNN 93 %, CNN 93 %, LSTM 94 %
SureshBabu y Geetha (2026)	Detección de intrusiones en drones: flujos de red simulados	UAVIDS-2025	ANN-GA y ConvLSTM, tomados de trabajos publicados (DNN/MLP; CNN-LSTM)	Exactitud: ANN-GA 91,8 %, ConvLSTM 95,3 %; propuesto 98,7 %
Tan et al. (2019)	Detección de intrusiones en drones: flujos de red	KDD Cup 99	ANN y DNN (DNN/MLP)	Exactitud: ANN 90,79 %, DNN 91,36 %; propuesto 92,44 %

Referencia	Problema	Dataset	Modelos de Deep Learning de referencia	Métricas reportadas y valores
Wang et al. (2026)	Detección de intrusiones en drones: flujos de red simulados	UAVIDS-2025	CNN, RNN, LSTM, ANN y BiLSTM (CNN; LSTM; RNN; DNN/MLP; BiLSTM)	LSTM de dos capas: F1 98,60 %; propuesto: exactitud 99,68 ± 0,12 %
Wu et al. (2024)	Detección de intrusiones en drones: flujos de red	CSE-CIC-IDS2018 y CIC-DDoS2019	VCDL, CNN3D, LSTM, SNDA, SCA+SVM y CGAN (CNN; LSTM; Autoencoder; GAN)	El propuesto alcanza 99 % de exactitud en CIC-DDoS2019; las tablas de comparación no se pudieron extraer
Zarkadis y Douligeris (2026)	Detección de intrusiones en drones: flujos de red simulados	UAVIDS-2025	MLP y RealMLP (DNN/MLP)	F1 de prueba: MLP 90,66 %, RealMLP 94,28 %
Zeng et al. (2025)	Detección de intrusiones en drones: flujos de red simulados	UAVIDS-2025	DNN 16-128-64-32-5; CNN 1D de 32 y 64 filtros; LSTM de 64 unidades; las tres con dropout de 0,2 (DNN/MLP; CNN; LSTM)	F1: DNN 93 %, CNN 93 %, LSTM 94 %; regresión logística 82 %, XGBoost 96 %
Zhou et al. (2026)	Detección de intrusiones en drones: flujos de red	ISCXVPN2016, CICIDS-2018, TON IoT y CIC IoT 2023	CNN, LSTM, Transformer y BPNN de 512, 256 y 128 (CNN; LSTM; Transformer; DNN/MLP)	Exactitud en ISCXVPN2016: LSTM 95,9 %, CNN 94,95 %

Tabla de frecuencia. Un estudio cuenta una vez por familia, aunque use varias variantes.

Modelo	Estudios (n = 42)	%	Estudios con UAVIDS-2025 (n = 11)	% con UAVIDS-2025	Estudios Q1 (n = 26)
DNN/MLP	24	57,1	9	81,8	15
CNN	22	52,4	6	54,5	10
LSTM	14	33,3	4	36,4	6
CNN-LSTM	6	14,3	2	18,2	1
RNN	5	11,9	2	18,2	5
GRU	5	11,9	0	0,0	4
Autoencoder	4	9,5	2	18,2	2
BiLSTM	3	7,1	2	18,2	2
DBN	3	7,1	0	0,0	3
Transformer	3	7,1	0	0,0	2
GNN	3	7,1	2	18,2	2
ResNet	2	4,8	0	0,0	1
GAN	1	2,4	0	0,0	1
TabNet	1	2,4	0	0,0	1
KAN	1	2,4	1	9,1	1

Interpretación. La DNN o MLP y la CNN son las referencias más frecuentes, con 24 y 22 estudios, seguidas de la LSTM, con 14. El cuarto lugar, la combinación de CNN con LSTM, tiene 6 estudios, así que el corte no es ambiguo. El orden se mantiene en los 11 estudios que usan UAVIDS-2025, donde estas tres familias también son las más usadas. Con solo los 26 estudios Q1, DNN/MLP tiene 15, CNN 10 y LSTM 6, por delante de RNN (5).

Esto significa que la selección no depende de un criterio personal, porque en el total los tres primeros lugares están claramente separados del resto. Hay que tener en cuenta que, entre los Q1, LSTM supera a la RNN simple, que tiene menor complejidad, por 1 estudio, gracias a los Q1 de la verificación de cobertura, y en el total la supera por 9. Ese margen es estrecho entre los Q1 y quedan 40 estudios Q1 sin leer, de modo que podría cambiar con más lecturas.

Por tal motivo, se opta por estas tres familias como modelos base.

Arquitectura y configuración reportadas. El paso 2 del procedimiento pide registrar la profundidad, el ancho, la regularización y el entrenamiento de cada modelo de referencia. La tabla reúne lo que se halló en la búsqueda dirigida de cada texto; lo que falta en ella queda como no reportado, aunque podría estar en una figura o en un material suplementario.

Referencia	Modelo	Arquitectura	Regularización	Entrenamiento	Fuente del dato
Zeng (2025)	DNN	3 capas ocultas: 128, 64 y 32	Dropout de 0,2 tras las capas de 128 y 64	Adam, tasa de 0,001, lote de 256, máximo 50 épocas, parada temprana con paciencia de 5	Código d los autor
Zeng (2025)	CNN 1D	2 convoluciones de 32 y 64 filtros de tamaño 3, max-pooling y densa de 64	Dropout de 0,2 antes de la capa densa	Adam, tasa de 0,001, lote de 256, máximo 50 épocas, parada temprana con paciencia de 5	Código d los autor
Zeng (2025)	LSTM	1 capa de 64 unidades	Dropout de 0,2 antes de la salida	Adam, tasa de 0,001, lote de 256, máximo 50 épocas, parada temprana con paciencia de 5	Código d los autor
Abdulhammed et al. (2026)	FFN (tabla de uav-gcs)	2 capas ocultas: 128 y 64	Dropout de 0,3	Adam, 50 épocas, lote de 32	Tabla 7
Abdulhammed et al. (2026)	LSTM (tabla de uav-gcs)	2 capas de 64 y 32 unidades	Dropout de 0,2	Adam, 50 épocas, lote de 32, parada temprana con paciencia de 10	Tabla 8

Referencia	Modelo	Arquitectura	Regularización	Entrenamiento	Fuente del dato
Abdulhammed et al. (2026)	1DCNN (tabla de uav-gcs)	Convoluciones de 64 y 128 filtros de tamaño 3, max-pooling de 2 y densa de 128	Dropout de 0,2 y 0,5	Adam, lote de 64	Tabla 9
Ashraf et al. (2023)	RNN (el lote aparece con dos valores en la misma tabla)	64 unidades; capas no reportadas	Dropout de 0,2	Adam, 20 épocas; lote reportado dos veces, 24 y 864	Tabla 2
Ashraf et al. (2023)	LSTM (el lote aparece con dos valores en la misma tabla)	64 unidades; capas no reportadas	Dropout de 0,2	Adam, 20 épocas; lote reportado dos veces, 24 y 1152	Tabla 2
Ashraf et al. (2023)	Bi-LSTM (el lote aparece con dos valores en la misma tabla)	64 unidades; capas no reportadas	Dropout de 0,2	Adam, 20 épocas; lote reportado dos veces, 24 y 1152	Tabla 2
Zhou et al. (2026)	BPNN	3 capas ocultas: 512, 256 y 128	Dropout de 0,2	No reportado	Texto
Zhou et al. (2026)	Transformer	2 capas, 4 cabezas y dimensión de 128	No reportado	Adam, tasa de 0,001, lote de 64	Texto

Referencia	Modelo	Arquitectura	Regularización	Entrenamiento	Fuente del dato
Hakeem et al. (2025)	MLP (clasificación binaria)	Capa oculta de 64 y entrada de 32	Dropout de 0,2	No reportado	Texto
Hakeem et al. (2025)	CNN (clasificación binaria)	Convolución y max-pooling, sin número de filtros	Dropout de 0,2 y 0,5	No reportado	Texto
Ozsoy et al. (2026)	MLP federado	2 capas ocultas: 64 y 32	Dropout, normalización por lotes y decaimiento de pesos L2	Adam con tasa coseno	Texto
Balgoon (2026)	BiLSTM (componente del modelo propuesto)	2 capas de 128 unidades	Dropout de 0,3	No reportado	Texto
Balgoon (2026)	GNN (componente del modelo propuesto)	3 capas GATv2 de ancho 64, con 8, 4 y 1 cabezas	No reportado	No reportado	Texto
Balgoon (2026)	Autoencoder (componente del modelo propuesto)	No extraída	No reportado	Adam, tasa de 0,001, 150 épocas, lote de 256	Texto
Biswas et al. (2026)	CNN 1D (conjunto isot)	2 bloques Conv1D de tamaño 3, con normalización por lotes y	Dropout de 0,3	No extraído	Texto

Referencia	Modelo	Arquitectura	Regularización	Entrenamiento	Fuente del dato
		max-pooling de 2			
Biswas et al. (2026)	GRU (conjunto isot)	2 capas de 128 y 64 unidades	Dropout de 0,3 y normalización por lotes	No extraído	Texto
Biswas et al. (2026)	RNN (conjunto isot)	2 capas SimpleRNN, la primera de 128 unidades	No extraído	No extraído	Texto
Alsaraireh et al. (2026)	CNN 1D	Capas no extraídas	Dropout de 0,3	Tasa de 0,001, lote de 2048, 20 épocas, parada temprana con paciencia de 5	Tabla 8
Li et al. (2026a)	CNN 1D	Convoluciones 1D con ReLU y pooling, sin número de capas	No reportado	No reportado	Texto
Bakiri et al. (2026)	KAN plana (conjunto uavids-2025)	6039 parámetros, en 23,59 KB	No reportado	Igual a la del modelo propuesto	Tabla 7
Batur Şahin (2026)	Clasificador sin VAE (línea base de la ablación)	3 capas ocultas: 256, 128 y 64	Dropout, sin tasa reportada	Tasa de 0,001 con recocido coseno, lote de 128, 100 épocas	Ecuacion 6 a 9 y sección 4

Interpretación. Solo 12 de los 42 estudios incluidos describen al menos un modelo de referencia, y en los otros 30 el modelo se nombra sin su arquitectura. Además, varios datos no provienen del conjunto principal de cada estudio ni de un modelo de referencia puro: las tablas de Abdulhammed et al. (2026) corresponden a UAV-GCS, y los de Balgoon (2026) son componentes de su modelo propuesto.

Hay que tener en cuenta que el enunciado excluye los estudios sin descripción suficiente de la arquitectura de referencia, y que en esta revisión ese criterio se aplicó exigiendo que el modelo de referencia y sus cifras se pudieran identificar, no que se describiera su arquitectura. Es decir, una aplicación estricta dejaría 12 estudios, y por eso se contrasta la frecuencia con ese subconjunto:

Familia	Estudios que la describen (n = 12)	Estudios que la usan (n = 42)
CNN	6	22
DNN/MLP	6	24
LSTM	3	14
BiLSTM	2	3
RNN	2	5
Autoencoder	1	4
GNN	1	3
GRU	1	5
KAN	1	1
Transformer	1	3

Con esa restricción, CNN y DNN/MLP siguen en los primeros lugares, con 6 y 6 estudios, y LSTM sigue a continuación, con 3, frente a 2 de BiLSTM, que además es una variante más compleja de LSTM, y 2 de RNN. Por lo anterior, se conservan los tres modelos, con la salvedad de que en el subconjunto estricto el margen es de un solo estudio. Esto también respalda tomar la arquitectura del código de los autores (Zeng, 2025), porque es la única que se halló con las tres familias sobre UAVIDS-2025.

Selección de los tres modelos. Dentro de cada familia se toma la variante más simple con arquitectura publicada. Zeng et al. (2025) nombran los tres modelos en su artículo, pero no describen sus capas, que están en el código público de los mismos autores (Zeng, 2025). Como la arquitectura sí está definida, no aplica la búsqueda de arquitectura de la sección 14 del Entregable 2.

Modelo	Arquitectura tomada de	Parámetros	Capas con pesos	Multiplicaciones por muestra
DNN	Zeng (2025)	12 677	4	12 448
LSTM	Zeng (2025)	17 477	2	266 560
CNN 1D	Zeng (2025)	39 493	4	132 928

Los parámetros y las multiplicaciones se calculan para una entrada de 16 atributos y 5 clases, y el código de la sección 4.2.1 verifica el conteo de parámetros.

Verificación de que no son redundantes. Los tres modelos difieren en el sesgo inductivo que incorporan. La DNN combina todos los atributos desde la primera capa, sin suponer ningún orden entre ellos. La CNN 1D busca patrones locales entre atributos vecinos y comparte pesos a lo largo del vector. La LSTM lee los atributos uno a la vez y acumula un estado. Hay que tener en cuenta que el orden de las columnas de una tabla no tiene significado físico, de modo que la CNN y la LSTM imponen una estructura que los datos no garantizan, y justamente por eso la comparación permite medir si ese supuesto ayuda o estorba. Para que sea reproducible, la secuencia sigue el orden de las columnas del archivo.

Equivalencia con los modelos clásicos de referencia. El enunciado asigna a estos modelos el papel que cumple el modelo logístico en clasificación, y la equivalencia se sustenta en tres aspectos. En capacidad de representación, los tres modelos terminan en una capa lineal con softmax:

$$p_\theta(c \mid x) = \frac{\exp\left(w_c^\top \phi_\theta(x)\right)}{\sum_{k=1}^{K} \exp\left(w_k^\top \phi_\theta(x)\right)}$$

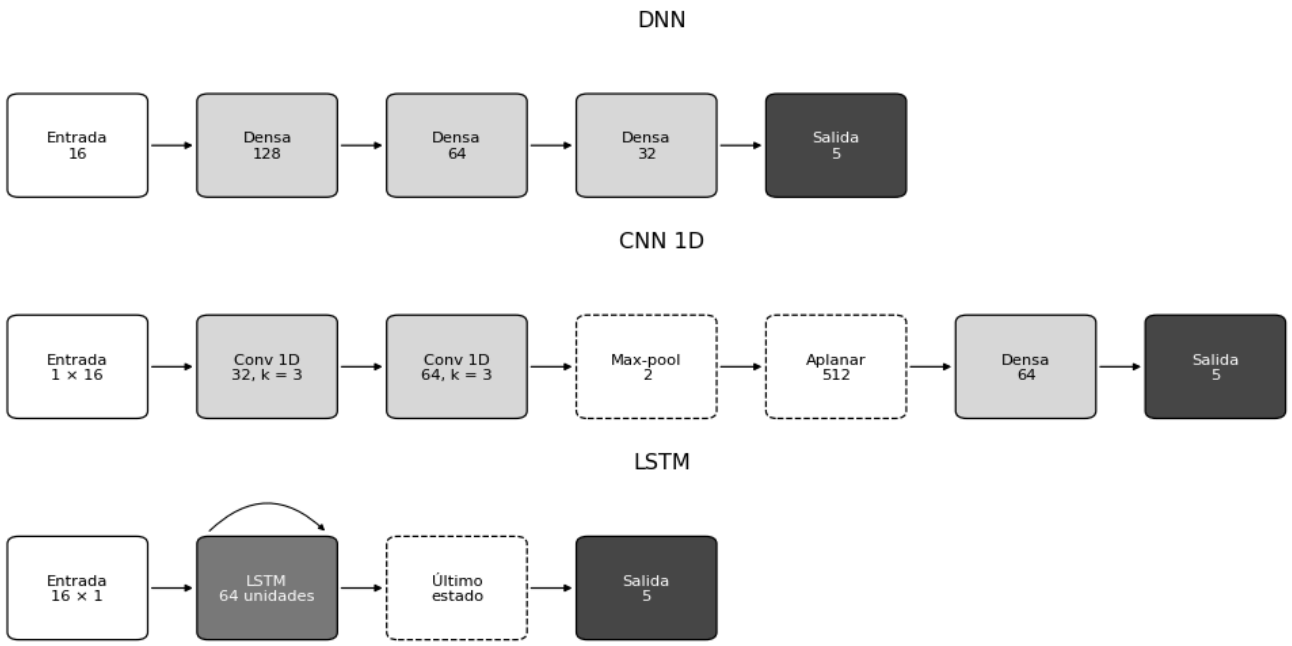
Es decir, son una regresión logística multinomial sobre una representación aprendida $\phi_\theta(x)$, y si ϕ_θ fuera la identidad, el modelo sería exactamente el logístico. En supuestos, el logístico supone

clases separables por un hiperplano en el espacio de entrada, mientras que estos modelos solo lo suponen en el espacio de ϕ_θ. En propósito comparativo, ambos fijan un piso con la misma pérdida, la entropía cruzada: un modelo avanzado que no los supere no justifica su mayor complejidad.

3.3 Análisis detallado por modelo base

Todos los modelos comparten la pérdida, la entropía cruzada ponderada $\mathcal{L}(\theta) = -\frac{1}{n}\sum_i w_{y_i} \log p_\theta(y_i \mid x_i)$, y resuelven la clasificación multiclase de flujos de red de la sección 1. Las referencias completas de cada estudio están en la sección 9, y la configuración de entrenamiento de referencia es la del código de los autores (Zeng, 2025): Adam con tasa de aprendizaje de 0,001, lotes de 256, un máximo de 50 épocas y parada temprana. La figura siguiente resume el flujo de datos de los tres modelos, que después se detalla para cada uno.

▶ **Mostrar el código de la celda**



Interpretación. La figura muestra el flujo de datos de cada modelo, de izquierda a derecha. Los bloques claros son capas con pesos, el de gris medio es la capa recurrente, con su lazo arriba, los punteados no tienen pesos, y los oscuros son la capa de salida. La DNN recibe el vector completo, la CNN 1D lo recorre con filtros de tamaño 3 y la LSTM lo lee como una secuencia de 16 pasos.

Esto significa que los tres modelos difieren en cómo combinan los atributos, y no solo en su número de parámetros: la DNN no supone ningún orden, mientras que la CNN 1D y la LSTM

dependen del orden de las columnas.

Por tal motivo, se conservan estas tres arquitecturas como base, porque sus diferencias responden a supuestos distintos.

Modelo 1: DNN

- **Tipo de arquitectura.** Perceptrón multicapa.
- **Referencias que la respaldan.** Abbas et al. (2026), https://doi.org/10.1109/ojcoms.2026.3698792, Abdulhammed et al. (2026), https://doi.org/10.70102/afts.2026.1836.229, Alissa et al. (2022), https://doi.org/10.3390/drones6100297, Alruwaili y Tunc (2026), https://doi.org/10.48550/arxiv.2607.17025, Arulalan et al. (2023), https://doi.org/10.17762/ijritcc.v11i9.8338, Ashraf et al. (2023), https://doi.org/10.1038/s41598-023-45065-8, Attaullah et al. (2026a), https://doi.org/10.1038/s41598-026-55446-4, Attaullah et al. (2026c), https://doi.org/10.1109/tce.2026.3667039, Attaullah et al. (2026b), https://doi.org/10.1109/tce.2026.3658881, Bakiri et al. (2026), https://doi.org/10.1016/j.sysarc.2026.104038, Batur Şahin (2026), https://doi.org/10.3390/s26020509, Biswas et al. (2026), https://doi.org/10.1038/s41598-026-45063-6, Cengiz et al. (2024), https://doi.org/10.1109/access.2024.3349469, Hadi et al. (2024), https://doi.org/10.1016/j.eswa.2024.124215, Hakeem et al. (2025), https://doi.org/10.3389/fcomp.2025.1545282, Junaid et al. (2026), https://doi.org/10.1016/j.iot.2026.102095, Li et al. (2026b), https://doi.org/10.1109/tcomm.2026.3693173, Ozsoy et al. (2026), https://doi.org/10.47000/tjmcs.1828534, SureshBabu y Geetha (2026), https://indianengineeringjournal.com/index.php/journal/article/view/270, Tan et al. (2019), https://doi.org/10.3390/s19245529, Wang et al. (2026), https://doi.org/10.1038/s41598-026-52524-5, Zarkadis y Douligeris (2026), https://doi.org/10.1109/cits70307.2026.11637173, Zeng et al. (2025), https://doi.org/10.1109/cns66487.2025.11194990, Zhou et al. (2026), https://doi.org/10.3389/fphy.2025.1660104. Las capas se toman del código de los autores (Zeng, 2025).
- **Descripción técnica.** Cada capa oculta aplica $z^{(l)} = \mathrm{Dropout}\big(\mathrm{ReLU}(W^{(l)} z^{(l-1)} + b^{(l)})\big)$, con *dropout* de 0,2. El flujo de datos es 16 → 128 → 64 → 32 → 5, y la capa de salida aplica softmax.

- **Supuestos y sesgo inductivo.** Supone que la clase depende de interacciones globales entre atributos, sin ninguna estructura local, lo cual es coherente con una tabla donde el orden de las columnas no tiene significado.
- **Tipo de problema que aborda.** Clasificación multiclase de flujos de red en los estudios revisados.
- **Datasets y métricas reportadas.** F1 de 93 % en UAVIDS-2025, sin dispersión (Zeng et al., 2025); F1 de 90,66 % en UAVIDS-2025, sin dispersión (Zarkadis y Douligeris, 2026); exactitud de 96,43 ± 0,22 % en T-ITS, con dispersión sobre 10 ejecuciones (Attaullah et al., 2026a), y 91,36 % en KDD Cup 99, sin dispersión (Tan et al., 2019).
- **Fortalezas.** Es el modelo más simple y barato de los tres, fácil de reproducir y el más usado como referencia.
- **Limitaciones.** No aprovecha ninguna estructura de los datos y su capacidad depende del ancho, de modo que necesita regularización y es sensible a la tasa de aprendizaje.
- **Complejidad computacional.** Con $c = (16, 128, 64, 32, 5)$, el número de parámetros es $P = \sum_{l=1}^{4}(c_{l-1}+1)c_l = 12\,677$. El costo por muestra es de orden $\mathcal{O}(\sum_l c_{l-1}c_l)$, unas 12 448 multiplicaciones; la inferencia requiere una pasada y el paso de entrenamiento agrega la retropropagación, del mismo orden. Los estudios no reportan un tiempo de entrenamiento comparable, por lo que se mide en la sección 4.2.1.

Modelo 2: CNN 1D

- **Tipo de arquitectura.** Red convolucional unidimensional poco profunda.
- **Referencias que la respaldan.** Abbas et al. (2026), https://doi.org/10.1109/ojcoms.2026.3698792, Abdulhammed et al. (2026), https://doi.org/10.70102/afts.2026.1836.229, Alharthi (2024), https://doi.org/10.3389/fenrg.2024.1491332, Alruwaili y Tunc (2026), https://doi.org/10.48550/arxiv.2607.17025, Alsaraireh et al. (2026), https://doi.org/10.3390/drones10070503, Alshammari et al. (2024), https://doi.org/10.3934/math.2024348, Arulalan et al. (2023), https://doi.org/10.17762/ijritcc.v11i9.8338, Attaullah et al. (2026c), https://doi.org/10.1109/tce.2026.3667039, Attaullah et al. (2026b), https://doi.org/10.1109/tce.2026.3658881, Bakiri et al. (2026), https://doi.org/10.1016/j.sysarc.2026.104038, Biswas et al. (2026), https://doi.org/10.1038/s41598-026-45063-6, Gharami y Moni (2025), https://doi.org/10.1109/dasc66011.2025.11257309, Hakeem et al. (2025),

https://doi.org/10.3389/fcomp.2025.1545282, Katbar et al. (2024), https://doi.org/10.7546/crabs.2024.08.09, Katib et al. (2025), https://doi.org/10.1016/j.rineng.2025.106838, Li et al. (2026a), https://doi.org/10.1109/jiot.2026.3665688, Menssouri et al. (2024), https://doi.org/10.1109/pimrc59610.2024.10817244, Ozsoy et al. (2026), https://doi.org/10.47000/tjmcs.1828534, Wang et al. (2026), https://doi.org/10.1038/s41598-026-52524-5, Wu et al. (2024), https://doi.org/10.1109/jiot.2024.3360231, Zeng et al. (2025), https://doi.org/10.1109/cns66487.2025.11194990, Zhou et al. (2026), https://doi.org/10.3389/fphy.2025.1660104. Las capas se toman del código de los autores (Zeng, 2025).

- **Descripción técnica.** Cada convolución aplica $z^{(l)} = \mathrm{ReLU}(W^{(l)} * z^{(l-1)} + b^{(l)})$, con filtros de tamaño 3. El flujo de datos es 1 × 16 → 32 filtros → 64 filtros → reducción a 8 posiciones → aplanado de 512 → 64 → 5.
- **Supuestos y sesgo inductivo.** Supone que la información discriminante es local y puede aparecer en varias posiciones. En una tabla, ese supuesto depende del orden de las columnas, que aquí es el del archivo, por lo que se trata de un sesgo discutible.
- **Tipo de problema que aborda.** Clasificación multiclase de flujos de red en los estudios revisados.
- **Datasets y métricas reportadas.** F1 de 93 % en UAVIDS-2025, sin dispersión (Zeng et al., 2025); exactitud de 89,10 % en una de las tablas de Alharthi (2024), sin dispersión; 93 % (Katbar et al., 2024), sin dispersión, y 86,54 % en NSL-KDD, sin dispersión (Alshammari et al., 2024).
- **Fortalezas.** Comparte pesos a lo largo del vector, de modo que tiene pocos parámetros por capa convolucional, y es fácil de reproducir.
- **Limitaciones.** Es sensible al orden de los atributos, y la capa densa posterior al aplanado concentra 32 832 de sus 39 493 parámetros, el 83 %.
- **Complejidad computacional.** El número de parámetros es $P = (3 \cdot 1 + 1) \cdot 32 + (3 \cdot 32 + 1) \cdot 64 + (64 \cdot 8 + 1) \cdot 64 + (64 + 1) \cdot 5 = 39\,493$. El costo por muestra es de orden $\mathcal{O}(\sum_l L\, k\, c_{l-1} c_l)$, con $L = 16$ y $k = 3$, unas 132 928 multiplicaciones. Los estudios no reportan un tiempo de entrenamiento comparable, por lo que se mide en la sección 4.2.1.

Modelo 3: LSTM

- **Tipo de arquitectura.** Red recurrente de una capa (Hochreiter y Schmidhuber, 1997).
- **Referencias que la respaldan.** Abdulhammed et al. (2026), https://doi.org/10.70102/afts.2026.1836.229, Aburasain (2024), https://doi.org/10.1109/access.2024.3359043, Alharthi (2024), https://doi.org/10.3389/fenrg.2024.1491332, Alshammari et al. (2024), https://doi.org/10.3934/math.2024348, Alshamrani y Alghamdi (2026), https://doi.org/10.1016/j.aej.2025.12.065, Ashraf et al. (2023), https://doi.org/10.1038/s41598-023-45065-8, Biswas et al. (2026), https://doi.org/10.1038/s41598-026-45063-6, Hakeem et al. (2025), https://doi.org/10.3389/fcomp.2025.1545282, Katbar et al. (2024), https://doi.org/10.7546/crabs.2024.08.09, Ozsoy et al. (2026), https://doi.org/10.47000/tjmcs.1828534, Wang et al. (2026), https://doi.org/10.1038/s41598-026-52524-5, Wu et al. (2024), https://doi.org/10.1109/jiot.2024.3360231, Zeng et al. (2025), https://doi.org/10.1109/cns66487.2025.11194990, Zhou et al. (2026), https://doi.org/10.3389/fphy.2025.1660104. Las capas se toman del código de los autores (Zeng, 2025).
- **Descripción técnica.** En cada paso t, la celda calcula las compuertas $f_t, i_t, o_t = \sigma(Wh_{t-1} + Vx_t + b)$, actualiza la memoria con $C_t = f_t \odot C_{t-1} + i_t \odot \tanh(W_c h_{t-1} + V_c x_t + b_c)$ y emite $h_t = o_t \odot \tanh(C_t)$. El flujo de datos es una secuencia de 16 pasos de un valor, 64 unidades, último estado y 5 salidas.
- **Supuestos y sesgo inductivo.** Supone que los elementos forman una secuencia con dependencias a lo largo de los pasos. Esto tiene sentido en series de tiempo, pero aquí los pasos son columnas de una tabla y no instantes, de modo que el sesgo no corresponde a la naturaleza de los datos.
- **Tipo de problema que aborda.** Clasificación multiclase de flujos de red en los estudios revisados.
- **Datasets y métricas reportadas.** F1 de 94 % en UAVIDS-2025, sin dispersión (Zeng et al., 2025); F1 de 98,60 % en UAVIDS-2025, sin dispersión para ese modelo (Wang et al., 2026); exactitud de 91,00 % en CICIDS2017, sin dispersión (Ashraf et al., 2023), y 95,91 % en NSL-KDD, sin dispersión (Alshammari et al., 2024).
- **Fortalezas.** Tiene pocos parámetros y gestiona el flujo del gradiente con sus compuertas.

- **Limitaciones.** Es la más costosa de las tres por muestra, porque repite el cálculo en cada paso, depende del orden de las columnas y no se puede paralelizar a lo largo de la secuencia.
- **Complejidad computacional.** Con entrada de dimensión 1 y $h = 64$ unidades, el número de parámetros es $P = 4\,[(1 + h)\,h + 2h] + (h + 1) \cdot 5 = 17\,477$, porque cada una de las cuatro compuertas tiene dos vectores de sesgo. El costo por muestra es de orden $\mathcal{O}(T\,h^2)$, con $T = 16$, unas 266 560 multiplicaciones. Los estudios no reportan un tiempo de entrenamiento comparable, por lo que se mide en la sección 4.2.1.

3.4 Comparación crítica

Comparación estructurada. La tabla resume los tres modelos con los valores de la sección 3.3, sin promediarlos, porque provienen de conjuntos de datos y protocolos distintos.

Criterio	DNN	CNN 1D	LSTM
Desempeño en UAVIDS-2025, repositorio de los autores (F1)	93 %	93 %	94 %
Robustez	Sin dependencia del orden de las columnas	Depende del orden de las columnas	Depende del orden de las columnas
Escalabilidad	Costo lineal en la dimensión de cada capa	Costo lineal en el número de atributos	Costo lineal en el número de pasos, sin paralelizar
Parámetros	12 677	39 493	17 477

Validez de los estudios revisados. Solo 12 de los 42 estudios describen al menos un modelo de referencia, y solo 2 agrupan filas relacionadas al partir los datos; 4 no permiten identificar la partición, 31 no reportan repeticiones, solo 8 reportan dispersión y 6 aplican alguna prueba, de las cuales tres comparan modelos con la prueba t y otra reporta p < 0,01 sin nombrar la prueba en los pasajes revisados (Mohamed et al., 2026), y ninguna sobre particiones agrupadas. Además, la comparabilidad es baja: varios estudios usan conjuntos genéricos, como KDD Cup 99 o CICIDS2017, que no provienen de redes de drones, y dos estudios sobre IoT repiten las

mismas cifras de comparación. Dos resultados merecen una alerta: Abdulhammed et al. (2026) reportan 100 % con LightGBM, y Wang et al. (2026) reportan 99,68 %, en ambos casos sin mencionar la exclusión de `FlowID` ni de las direcciones IP, que la sección 2.1.4 identifica como atajos.

Brecha de desempeño. La literatura reporta que los modelos propuestos superan a estas referencias, como 98,74 % frente a 96,43 % de la DNN (Attaullah et al., 2026a), o 98,18 % frente a 94,60 % de una GNN en UAVIDS-2025 (Balgoon, 2026). Sin embargo, esa brecha se obtiene con particiones aleatorias, y la sección 2.1.3 mostró que cambiar a una partición agrupada mueve el F1 macro en una magnitud parecida o mayor. Por tanto, esta brecha se toma como contexto para interpretar el baseline, y no como una mejora demostrada.

Una auditoría concordante. Bakiri et al. (2026) auditan UAVIDS-2025 de forma independiente, encuentran las mismas 8219 filas repetidas de la sección 2.1.2 y confirman que `FlowID` no identifica la simulación, y por eso parten por nodo de origen. Con esa partición reportan una exactitud de 92,50 % en cinco clases, frente a 0,8607 del baseline de este trabajo. Hay que tener en cuenta que no son cifras comparables de forma directa, porque cambian el modelo, los datos de prueba y lo que se mantiene separado: ellos generalizan a nodos no vistos, y este trabajo, a tramos no vistos del archivo, ya que la sección 2.1.3 muestra que las direcciones de origen no separan a los flujos vecinos. Por tal motivo, esa diferencia se toma como una pregunta abierta sobre la partición, y no como un margen de mejora del modelo.

Vacío en la literatura. De los 42 estudios incluidos, 2 agrupan las filas al partir: Biswas et al. (2026) por sesión y dron en otro conjunto de datos, con cinco semillas y sin prueba estadística, y Bakiri et al. (2026) por nodo en UAVIDS-2025, frente a una KAN plana y a cifras publicadas. Por tanto, ninguno de los 11 que usan UAVIDS-2025 compara modelos base de Deep Learning con una partición que respete la dependencia entre filas y con inferencia estadística sobre repeticiones, al menos en los pasajes revisados. Este trabajo atiende ese vacío, con la salvedad de que 174 estudios elegibles no se leyeron a texto completo.

4. Implementación y evaluación de los tres modelos base

En esta sección se implementan los tres modelos bajo un mismo entorno experimental y se evalúan con el mismo protocolo, en el orden del enunciado: diseño experimental (sección 4.1), implementación (sección 4.2), requisitos experimentales (sección 4.3) y evaluación (sección 4.4).

Lo que ya está guardado se salta, de modo que basta con volver a ejecutar si la corrida se interrumpe.

▶ **Mostrar el código de la celda**

```
Entrada: 16 atributos | clases: 5 | entrenamiento 73,267 | validación 17,802 | pr
```

4.1 Diseño experimental

En esta subsección se define el protocolo antes de entrenar, para que los tres modelos se comparen bajo las mismas condiciones y nada dependa de los resultados.

Elemento	Definición
Variables de entrada	Vector de 16 atributos de tráfico y desempeño, recortado, transformado con $\log(1+x)$ y estandarizado (secciones 2.2.1 y 2.2.5)
Variable objetivo	Una de las 5 clases del espacio $\mathcal{Y}$ (sección 1.1); no hay horizonte de predicción, porque cada flujo se clasifica por sí solo
División del dataset	Entrenamiento, validación y prueba de 64, 16 y 20 %, con semilla 42, y prueba congelada con su suma SHA-256; la estrategia se justifica en la sección 2.1.3
Esquema de validación	Partición fija de validación, agrupada por bloques, para seleccionar hiperparámetros, porque hay decenas de miles de flujos y una validación anidada multiplicaría los entrenamientos sin cambiar la comparación
Preprocesamiento	Eliminación de filas repetidas y transformación con los estadísticos de entrenamiento, sin aumento de datos (secciones 2.1.2 y 2.2.1)
Entrenamiento	Mismo motor, pérdida y optimizador para los tres modelos (sección 4.2.2), con el mismo esfuerzo de ajuste (sección 4.2.3)
Evaluación	Mismas métricas sobre el mismo conjunto de prueba, evaluado una sola vez por corrida con el modelo ya fijado (sección 4.4)

Estudio piloto y dimensionamiento. Antes de la ejecución definitiva, cinco ejecuciones por modelo estiman la variabilidad del F1 macro y fijan el número de ejecuciones. Se ejecuta en la sección 4.3.1 y no aquí, porque necesita los modelos y los hiperparámetros de la sección 4.2.

4.2 Implementación de los tres modelos base

Los modelos son los que resultan de la tabla de frecuencia de la literatura (sección 3.2), y se implementan con PyTorch, cuya versión se registra en la configuración del entorno. Todos siguen las capas del código público de los autores (Zeng, 2025), de modo que no aplica la búsqueda de arquitectura de la sección 14 del Entregable 2. Hay que tener en cuenta que, al contrastar ese código el 4 de noviembre de 2026, con los resultados ya calculados, se detectó

una desviación no prevista en el *dropout*, que la tabla declara y la sección 5.4 evalúa con un análisis de sensibilidad:

Modelo	Origen de la arquitectura	Adaptación a estos datos
DNN	Zeng (2025): perceptrón de 16 → 128 → 64 → 32 → 5, con ReLU y *dropout* de 0,2 tras las capas de 128 y 64	*Dropout* también tras la capa de 32
CNN 1D	Zeng (2025): convoluciones 1D de 32 y 64 filtros de tamaño 3, *max-pooling*, *dropout* de 0,2 y capa densa de 64	Sin *dropout*
LSTM	Zeng (2025): una capa de 64 unidades que lee los 16 atributos como una secuencia de largo 16, con *dropout* de 0,2 antes de la salida	Sin *dropout*

El entrenamiento también difiere del código de los autores, que usa entropía cruzada sin ponderar, paciencia de 5 épocas sobre la pérdida de validación, sin planificador, y una tasa y un lote fijos. Aquí se aplica el protocolo común de la sección 4.2.2, por las razones de la sección 2 y para dar a los tres modelos el mismo esfuerzo de ajuste.

4.2.1 Modelos y costo de entrenamiento

Antes de dimensionar los experimentos se mide la velocidad real de cada modelo en este equipo.

▶ **Mostrar el código de la celda**

	Parámetros	Muestras por segundo	Segundos por época
DNN	12677	33676	2,2
CNN 1D	39493	16629	4,4
LSTM	17477	10091	7,3

	DNN	CNN 1D	LSTM	Total
Búsqueda (6 configuraciones, 15 épocas)	0,06	0,11	0,18	0,35
Piloto (5 semillas, 50 épocas)	0,15	0,31	0,51	0,97
Definitiva (hasta 12 semillas, 50 épocas)	0,37	0,73	1,22	2,32
Total	0,57	1,15	1,91	3,63

Interpretación. LSTM es el modelo más costoso, con 7,3 segundos por época en CPU, y el total en el peor caso, con todas las corridas hasta el máximo de épocas, es de 3,6 horas.

Esto significa que es un límite superior, porque la parada temprana lo reduce y el piloto fija después el número real de semillas, y que el costo no restringe el número de ejecuciones que pida el estudio piloto.

Por lo anterior, se conserva este presupuesto, y cada corrida se guarda al terminar, de modo que una interrupción no obligue a repetir lo ya hecho.

4.2.2 Motor de entrenamiento e hiperparámetros

Todos los modelos se entrenan con la misma función: entropía cruzada ponderada, Adam (Kingma y Ba, 2015), como en el código de los autores (Zeng, 2025), reducción de la tasa de aprendizaje por 10 si la pérdida de validación no mejora en 2 épocas y parada temprana sobre el F1 macro de validación. El conjunto de prueba solo se evalúa en la fase definitiva, una vez por corrida, con el modelo ya fijado. Se fijan las semillas de Python, NumPy y PyTorch, sin determinismo estricto, porque no se usa aleatoriedad del dispositivo más allá de la de PyTorch en CPU. La tabla que sigue a la celda reúne estas definiciones.

▶ **Mostrar el código de la celda**

Definición	Valor
Capas y unidades	Las de la sección 3.3
Función de pérdida	Entropía cruzada ponderada
Optimizador	Adam
Tasa de aprendizaje	Se ajusta en la sección 4.2.3
Planificador	Reducción por 10 si la pérdida de validación no mejora en 2 épocas
Decaimiento de pesos	Se ajusta en la sección 4.2.3
Tamaño de lote	Se ajusta en la sección 4.2.3
Dropout	0,2 en la DNN; las otras dos arquitecturas no lo usan aquí (sección 4.2)
Máximo de épocas	50
Parada temprana	Paciencia de 6 épocas sobre el F1 macro de validación
Registro de cada corrida	Curvas, parámetros, semilla, hiperparámetros, tiempo y equipo

4.2.3 Ajuste de hiperparámetros con el mismo esfuerzo

Para que el baseline no quede subestimado, los tres modelos reciben exactamente el mismo presupuesto: las mismas configuraciones del mismo espacio, sobre todo el conjunto de entrenamiento y con un máximo de 15 épocas. La arquitectura no se ajusta, porque viene definida por la literatura.

▶ **Mostrar el código de la celda**

	Modelo	lr	weight_decay	batch	F1 macro de validación	Minutos
0	DNN	0,001	0,0001	128	0,9283	0,9067
1	DNN	0,0001	0	256	0,9018	0,4917
2	DNN	0,0001	0	128	0,9184	0,9317
3	DNN	0,0005	0	128	0,933	1,1517
4	DNN	0,0001	0,0001	256	0,9012	0,5717
5	DNN	0,001	0,0001	256	0,9297	0,5633
6	CNN 1D	0,001	0,0001	128	0,9298	2,755
7	CNN 1D	0,0001	0	256	0,9236	1,3717
8	CNN 1D	0,0001	0	128	0,9293	0,9867
9	CNN 1D	0,0005	0	128	0,9226	0,9533
10	CNN 1D	0,0001	0,0001	256	0,9225	0,6517
11	CNN 1D	0,001	0,0001	256	0,9206	1,5233
12	LSTM	0,001	0,0001	128	0,873	2,065
13	LSTM	0,0001	0	256	0,8076	1,1467
14	LSTM	0,0001	0	128	0,8375	2,6683
15	LSTM	0,0005	0	128	0,876	2,8217
16	LSTM	0,0001	0,0001	256	0,8033	1,3117
17	LSTM	0,001	0,0001	256	0,8706	1,2667

	lr	weight_decay	batch
CNN 1D	0,001	0,0001	128
DNN	0,0005	0	128
LSTM	0,0005	0	128

	max	min	Rango
Modelo			
CNN 1D	0,9298	0,9206	0,0092
DNN	0,933	0,9012	0,0318
LSTM	0,876	0,8033	0,0727

Interpretación. La tabla resume, para cada modelo, el mejor y el peor F1 macro de validación entre las mismas 6 configuraciones. LSTM es el más sensible a los hiperparámetros, con un rango de 0,073. Es decir, sin ajuste su desempeño podría subestimarse en esa magnitud, lo que justifica dar a todos el mismo esfuerzo de optimización.

Hay que tener en cuenta que esta búsqueda usa una sola semilla y 15 épocas, de modo que ordena configuraciones, pero sus valores no estiman el desempeño final. Por ello, ninguna conclusión sobre qué modelo es mejor se apoya en esta tabla.

4.3 Requisitos experimentales

En esta subsección se fijan las repeticiones y se ejecutan las corridas. Las semillas se controlan en cada corrida, sin determinismo estricto (sección 4.2.2). Cada modelo se ejecuta con semillas independientes y documentadas, y el mínimo de seis ejecuciones se ajusta con el piloto. La media, la desviación estándar y los intervalos de confianza se reportan en la sección 5.1. Cada corrida guarda en la carpeta de resultados su configuración, su semilla, su tiempo y el equipo usado, y las versiones de *software* se registran en la configuración del entorno. El entrenamiento usa solo CPU, de modo que no aplica el registro de GPU.

4.3.1 Estudio piloto y dimensionamiento

Cinco ejecuciones por modelo, con semillas distintas y evaluadas sobre la partición de validación, estiman la variabilidad del F1 macro. Con ella se calcula el número de ejecuciones necesario para detectar δ con potencia de 0,80 y α = 0,05, mediante $R \approx (z_{1-\alpha/2} + z_{1-\beta})^2 \hat{\sigma}_d^2 / \delta^2$. Esto se registra en un preregistro fechado, antes de la fase definitiva.

El tope de ejecuciones solo acota el cómputo, y es alto porque cada corrida es barata en CPU. Si el cálculo lo supera, se usa el tope y la diferencia mínima detectable que resulte se declara como limitación explícita, tal como lo admite el enunciado.

▶ **Mostrar el código de la celda**

Modelo	Media	Desviación estándar	Coeficiente de variación (%)
CNN 1D	0,9281	0,00389	0,4196
DNN	0,92862	0,0029	0,31207
LSTM	0,88189	0,00824	0,93487

	Par	Diferencia media	σ̂_d	δ/σ̂_d	Shapiro-Wilk, p	R necesario
0	CNN 1D - DNN	-0,00053	0,00675	1,48061	0,80595	4
1	CNN 1D - LSTM	0,0462	0,00834	1,19878	0,87405	6
2	DNN - LSTM	0,04673	0,00891	1,12265	0,28783	7


```
{
 "fecha": "2026-11-03 23:16",
 "ejecuciones_calculadas": 7,
 "tope_por_presupuesto": 30,
 "limitado_por_tope": false,
 "ejecuciones_definitivas": 7,
 "diferencia_minima_detectable": 0.009432
}
```


Interpretación. El preregistro, fechado el 2026-11-03 23:16, fija 7 ejecuciones por modelo. El cálculo con la mayor $\hat{\sigma}_d$ entre los tres pares pedía 7.

En ese sentido, con estas ejecuciones la diferencia mínima detectable, con potencia de 0,80, es de 0,0094, que no supera el margen δ = 0,01. Esto significa que el diseño tiene potencia suficiente para detectar una diferencia relevante, con la variabilidad que el piloto midió en validación; la sección 5.3 repite el cálculo con la variabilidad de la prueba.

Por tal motivo, las comparaciones finales se acompañan siempre de esta diferencia mínima detectable y del tamaño de efecto, y no solo del valor p.

4.3.2 Ejecución definitiva

Se ejecuta el número de corridas fijado en el preregistro, con las mismas semillas, partición y pipeline para los tres modelos. Es la única fase que toca el conjunto de prueba, y sus probabilidades se guardan para el análisis estadístico.

▶ **Mostrar el código de la celda**

```
Fase definitiva completa: 7 semillas
Corridas guardadas: 54
```

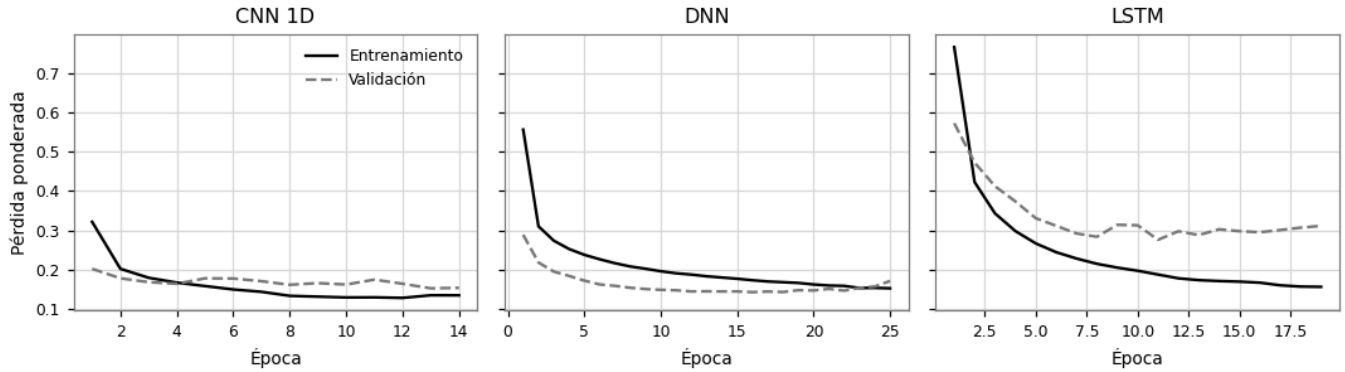
4.4 Evaluación

En esta subsección se calculan, para cada corrida, las métricas declaradas a priori sobre el conjunto de prueba, y se revisa el comportamiento de los modelos: curvas de aprendizaje, desempeño por clase y por nivel de ruido, calibración y errores. Esto es primordial porque el promedio global no muestra en qué condiciones falla cada modelo, y justamente esas condiciones definen el margen que le queda a un modelo avanzado.

▶ **Mostrar el código de la celda**

```
Corridas analizadas: 21 | Semillas pareadas: 7 | Observaciones de prueba: 22883
```

▶ **Mostrar el código de la celda**



Modelo	F1 de entrenamiento	F1 de validación	F1 de prueba	Épocas	Brecha entrenamiento - validación	Brecha validación - prueba
CNN 1D	0,9292	0,9241	0,8549	10,5714	0,0051	0,0691
DNN	0,9287	0,9272	0,8548	17,1429	0,0015	0,0725
LSTM	0,9137	0,8771	0,8423	15	0,0367	0,0348

Interpretación. Las curvas promedian la pérdida de las 7 semillas de cada modelo, y hay que leerlas con una salvedad: la de entrenamiento se acumula durante la época, con *dropout* activo, mientras que la de validación se calcula al cerrarla. La mayor brecha de F1 macro entre entrenamiento y validación es la de LSTM, con 0,037, y las corridas se detienen en promedio entre 10,6 y 17,1 épocas de un máximo de 50.

Lo anterior indica que esa brecha mide cuánto se ajusta cada modelo a particularidades del entrenamiento que no se trasladan a datos nuevos, de modo que una brecha alta apunta a sobreajuste y una brecha baja con F1 bajo apuntaría a subajuste. Así mismo, la diferencia entre validación y prueba llega a 0,072 en valor absoluto. Como supera $\delta = 0{,}01$, el F1 de validación no sirve como estimación final: puede ser optimista porque sobre él se eligieron la época y los hiperparámetros, y además los bloques de prueba pueden ser más difíciles que los de validación, sin que esta tabla permita separar ambas causas.

Por lo anterior, se conserva la parada temprana sobre el F1 macro de validación y las comparaciones de la sección 5 se hacen solo con el F1 de prueba.

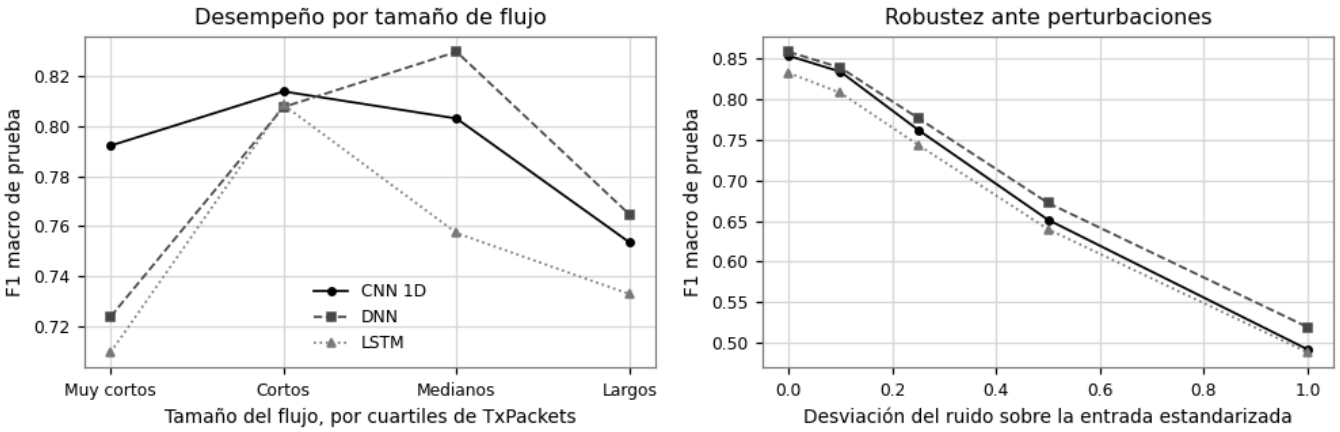
▶ **Mostrar el código de la celda**

Modelo	CNN 1D	DNN	LSTM
Blackhole	0,8334	0,8257	0,824
Flooding	0,8017	0,8075	0,7824
Normal	0,9688	0,9739	0,9569
Sybil	0,8517	0,856	0,8302
Wormhole	0,8192	0,8108	0,8178

	Precision	Recall	F1
Blackhole	0,8451	0,8223	0,8334
Flooding	0,7209	0,9059	0,8017
Normal	0,9785	0,9596	0,9688
Sybil	0,953	0,771	0,8517
Wormhole	0,783	0,8592	0,8192

Modelo	CNN 1D	DNN	LSTM
Muy cortos	0,7921	0,724	0,7098
Cortos	0,8137	0,8075	0,8083
Medianos	0,8029	0,8295	0,7573
Largos	0,7536	0,7647	0,733

	CNN 1D	DNN	LSTM
σ = 0.0	0,8537	0,8586	0,8326
σ = 0.1	0,834	0,839	0,808
σ = 0.25	0,7622	0,7767	0,7437
σ = 0.5	0,6515	0,6723	0,6399
σ = 1.0	0,4918	0,5193	0,488

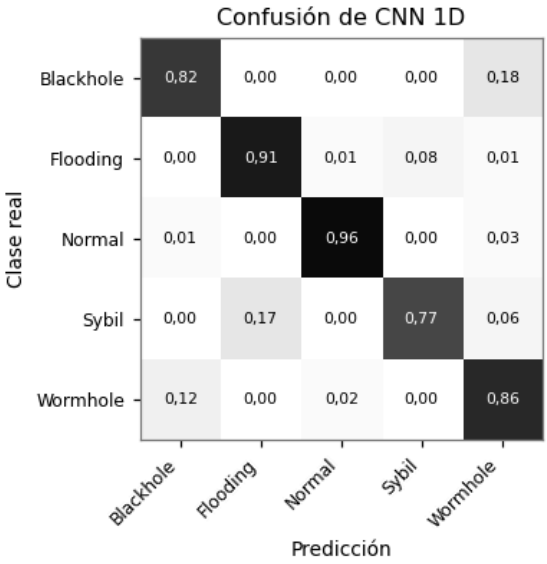


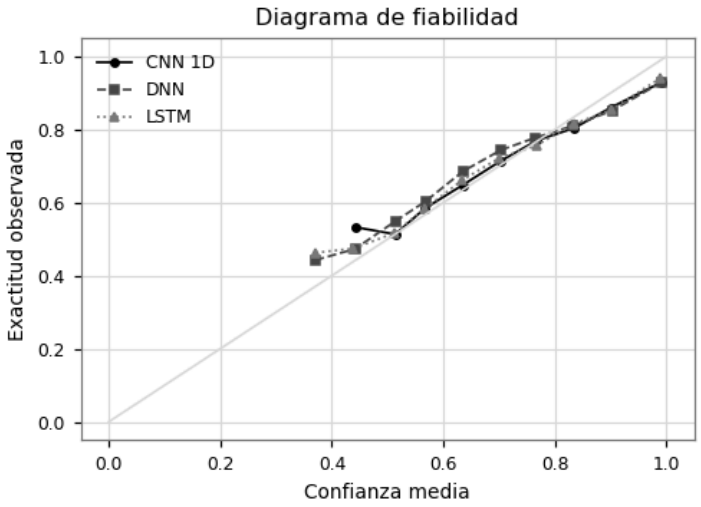
Interpretación. Para CNN 1D, la clase más difícil es Flooding, con F1 de 0,802, y la mayor diferencia entre precision y recall aparece en Flooding, con 0,721 y 0,906, lo que muestra hacia qué tipo de error se inclina el modelo en esa clase. Por tamaño de flujo, su F1 macro va de 0,754 en los flujos largos a 0,814 en los cortos. Con un ruido gaussiano de σ = 0,5 sobre la entrada estandarizada, el F1 macro baja 0,202 en CNN 1D, la mayor caída de los tres modelos.

En ese sentido, la comparación global oculta diferencias entre tipos de flujo, de modo que un promedio no basta para decidir dónde conviene mejorar. Hay que tener en cuenta que esta robustez se mide con los pesos de la primera semilla y con un ruido simulado, por lo que orienta la lectura, pero no reemplaza una prueba con tráfico real.

Por esta razón, lo indicado es exigir a cualquier modelo avanzado que reporte el resultado por clase y por tamaño de flujo junto con el global, y juzgar su aporte principalmente donde el baseline todavía tiene margen.

▶ **Mostrar el código de la celda**



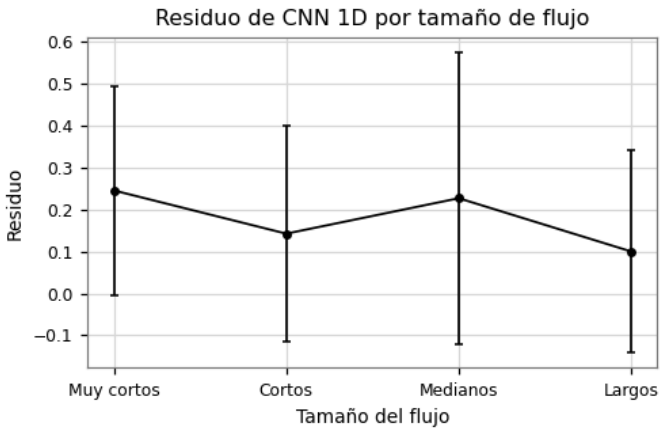


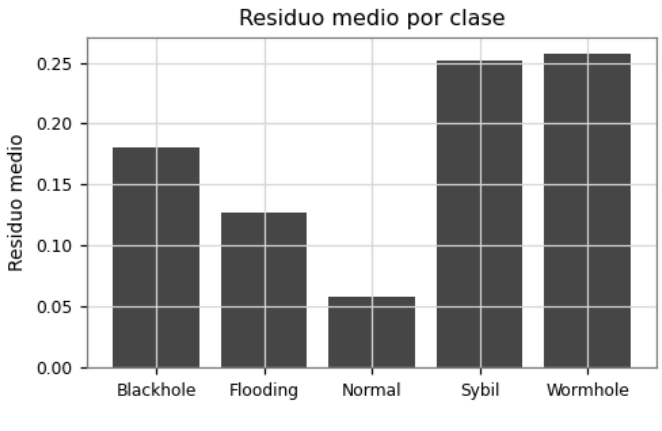
Interpretación. La matriz acumula las 7 semillas de CNN 1D y se normaliza por fila, de modo que la diagonal es la exhaustividad de cada clase. La confusión más frecuente es la de Blackhole predicha como Wormhole, en el 17,6 % de sus muestras. En calibración, el error esperado $ECE = \sum_b \frac{n_b}{n} |\mathrm{acc}(b) - \mathrm{conf}(b)|$ va de 0,040 a 0,052, y el mayor corresponde a DNN.

Esto significa que el error no se reparte al azar, sino que se concentra en pares concretos, lo que orienta dónde buscar la mejora. Así mismo, un ECE alto indica que las probabilidades del modelo no se pueden leer como frecuencias, aunque la clase predicha sea correcta, y esto importa porque el Entregable 3 lleva el modelo a un tablero donde se muestra esa confianza.

En consecuencia, se decide reportar el ECE junto con el F1 macro y dejar la recalibración como una decisión posterior, ajustada solo con validación, sin que altere el baseline de este entregable.

▶ **Mostrar el código de la celda**





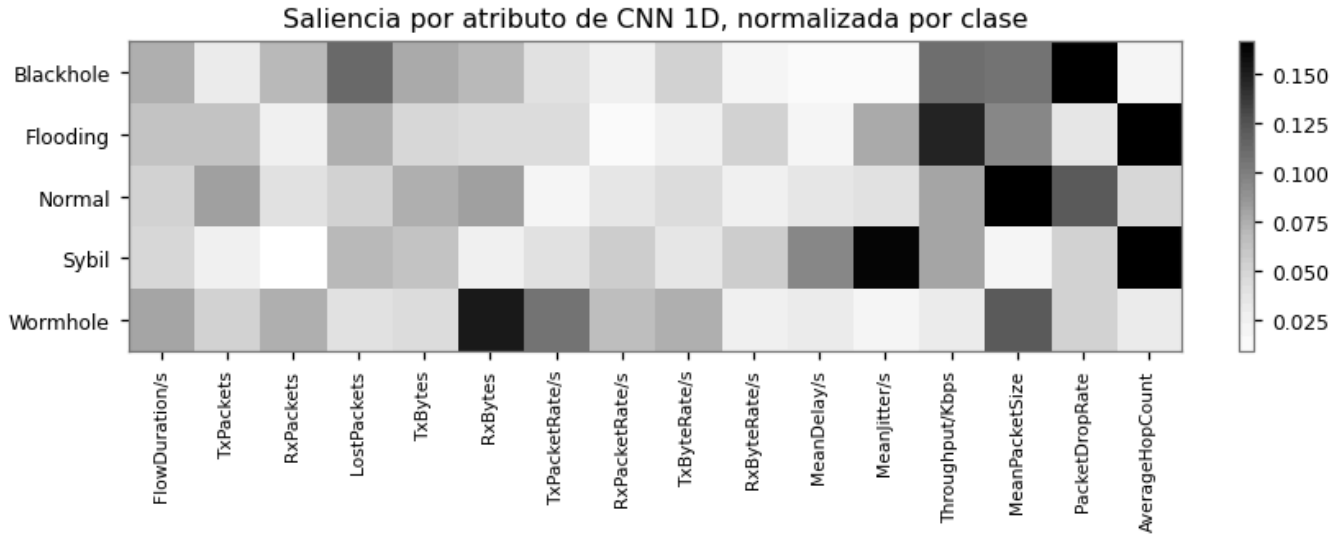
Interpretación. En clasificación, el residuo de una muestra se define como $1 - p_\theta(y_i \mid x_i)$, la probabilidad que el modelo no asignó a su clase real, promediada entre semillas. Para CNN 1D,

el residuo medio va de 0,100 a 0,245 entre rangos de tamaño de flujo, con un ρ de Spearman de -0,259 frente a `TxPackets` ($p < 0{,}001$), y la prueba de Levene entre rangos da $p < 0{,}001$. Por clase, el residuo medio va de 0,058 a 0,258, y el mayor corresponde a Wormhole.

Esto significa que el residuo tiene estructura, porque depende del tamaño del flujo y no se comporta como un error aleatorio. Así mismo, la dispersión cambia entre rangos, es decir, hay heterocedasticidad, de modo que un promedio global describe mal a cada rango. Hay que tener en cuenta que la diferencia entre clases mide un sesgo que persiste pese a la ponderación de la pérdida.

Por tal motivo, se reportan los resultados por tamaño de flujo y por clase, y no solo el promedio global.

▶ **Mostrar el código de la celda**



	Blackhole, acierto	Blackhole predicho como Wormhole	Wormhole, acierto	|fallo - acierto|
MeanPacketSize	-1,38	0,59	0,59	1,97
AverageHopCount	-0,51	1,07	1,21	1,58
LostPackets	0,09	-0,63	-0,63	0,72
TxPackets	-0,46	-1,09	-1,09	0,63
MeanJitter/s	-0,02	-0,6	0,57	0,58
RxBytes	-0,59	-1,14	-0,96	0,55

Interpretación. El mapa muestra, para CNN 1D con la semilla 2001, el valor medio de $|\partial f_c / \partial x_j|$ sobre los aciertos de cada clase. El atributo con más saliencia es `PacketDropRate` en Blackhole y `RxBytes` en Wormhole. La tabla contrasta, en desviaciones estándar frente a la

media de entrenamiento, los 946 flujos de Blackhole predichos como Wormhole, que es el error más frecuente, con los aciertos de ambas clases; la mayor diferencia está en `MeanPacketSize`.

Es decir, los errores se concentran en flujos cuyos atributos se parecen más a los de la otra clase, lo cual es coherente con el ruido de etiquetas y el solapamiento de la sección 2.1.2. Hay que tener en cuenta que la saliencia es una derivada local, y que orienta la lectura, pero no demuestra por sí sola la ausencia de atajos.

Con base en esto, se propone contrastar esta inspección con el análisis de atajos de la sección 2.1.4, y mantener esa auditoría en los modelos avanzados del Entregable 2.

5. Benchmark de los modelos base y definición del baseline

En esta sección se comparan los tres modelos, se fija el baseline y se calcula el margen de mejora que debe superar un modelo avanzado, con pruebas de hipótesis formales. El orden es el siguiente: primero la tabla comparativa y el ranking, luego la definición del baseline y del umbral, después la verificación de supuestos y, finalmente, las pruebas. El nivel de significancia es $\alpha = 0{,}05$, declarado en la sección 1 antes de ver cualquier resultado. Las observaciones pareadas son las semillas, que comparten partición y conjunto de prueba en los tres modelos.

5.1 Comparación tabular

▶ **Mostrar el código de la celda**

Modelo	CNN 1D	DNN	LSTM
F1 macro	0,8549 ± 0,0091 [0,8466; 0,8633]	0,8548 ± 0,0110 [0,8446; 0,8649]	0,8423 ± 0,0135 [0,8298; 0,8547]
Accuracy	0,8607 ± 0,0088 [0,8526; 0,8689]	0,8604 ± 0,0111 [0,8502; 0,8707]	0,8499 ± 0,0134 [0,8375; 0,8623]
Balanced accuracy	0,8636 ± 0,0081 [0,8562; 0,8711]	0,8617 ± 0,0113 [0,8513; 0,8722]	0,8501 ± 0,0119 [0,8391; 0,8610]
Precision macro	0,8561 ± 0,0078 [0,8489; 0,8633]	0,8545 ± 0,0118 [0,8437; 0,8654]	0,8438 ± 0,0172 [0,8279; 0,8597]
Recall macro	0,8636 ± 0,0081 [0,8562; 0,8711]	0,8617 ± 0,0113 [0,8513; 0,8722]	0,8501 ± 0,0119 [0,8391; 0,8610]
F1 ponderado	0,8619 ± 0,0086 [0,8540; 0,8698]	0,8610 ± 0,0111 [0,8507; 0,8712]	0,8503 ± 0,0133 [0,8380; 0,8626]
Precision ponderada	0,8711 ± 0,0076 [0,8641; 0,8782]	0,8672 ± 0,0120 [0,8561; 0,8783]	0,8587 ± 0,0160 [0,8439; 0,8735]
Recall ponderado	0,8607 ± 0,0088 [0,8526; 0,8689]	0,8604 ± 0,0111 [0,8502; 0,8707]	0,8499 ± 0,0134 [0,8375; 0,8623]
AUC-ROC macro	0,9637 ± 0,0010 [0,9627; 0,9646]	0,9602 ± 0,0050 [0,9556; 0,9648]	0,9728 ± 0,0048 [0,9684; 0,9773]
AUC-PR macro	0,8511 ± 0,0066 [0,8450; 0,8571]	0,8336 ± 0,0104 [0,8239; 0,8432]	0,8791 ± 0,0235 [0,8574; 0,9008]
MCC	0,8254 ± 0,0106 [0,8156; 0,8352]	0,8244 ± 0,0143 [0,8112; 0,8376]	0,8117 ± 0,0176 [0,7954; 0,8279]
ECE	0,0482 ± 0,0051 [0,0435; 0,0529]	0,0520 ± 0,0079 [0,0447; 0,0593]	0,0402 ± 0,0090 [0,0318; 0,0485]
Parámetros	39 493	12 677	17 477
Minutos de entrenamiento	1,5 ± 0,3	0,6 ± 0,2	2,3 ± 0,3
ms por muestra	0,029	0,001	0,010

Interpretación. La tabla reporta media ± desviación estándar sobre 7 semillas y, entre corchetes, el intervalo de confianza al 95 % calculado con la t de Student, $\bar{x} \pm t_{0{,}975;\,n-1}\, s/\sqrt{n}$. En la métrica primaria, CNN 1D obtiene 0,8549, DNN 0,8548 y LSTM 0,8423, con desviaciones entre semillas de 0,0091 a 0,0135. El orden cambia con la métrica: LSTM obtiene el mayor AUC-ROC macro, 0,9728, y su AUC-PR macro es de 0,8791, con un ECE de 0,0402.

Hay que tener en cuenta que esa dispersión mide la inicialización y el orden de los lotes, no la selección de los datos, porque todas las corridas comparten la partición. Por eso los intervalos no deben leerse como la incertidumbre total de generalización, que se aborda con el remuestreo por bloques y con la validación cruzada repetida en la sección 5.4.

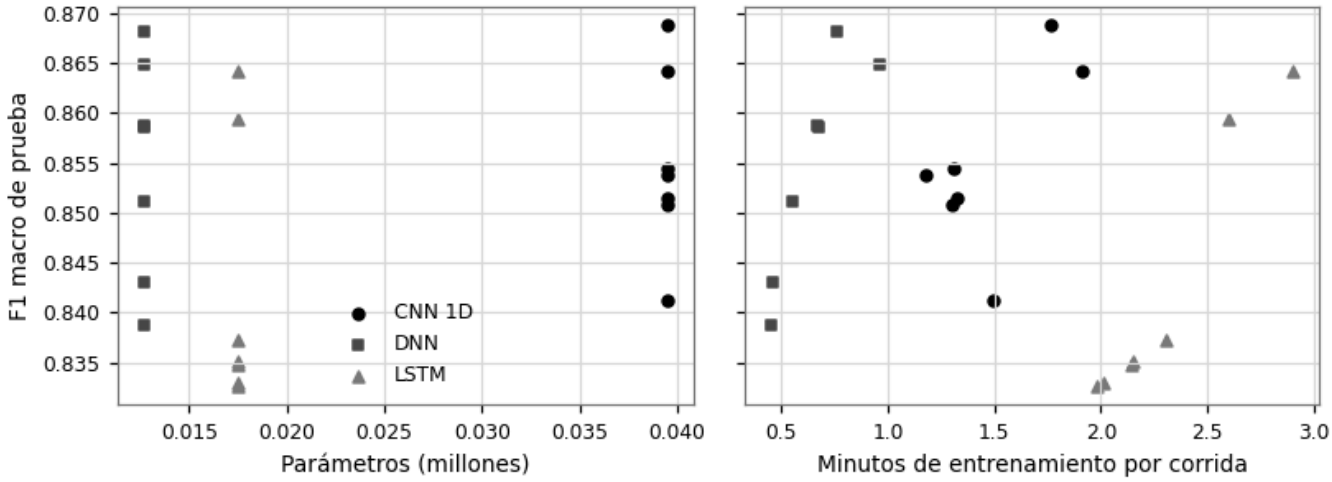
Por tal motivo, se opta por usar esta tabla para describir el desempeño y por apoyar toda afirmación comparativa en las pruebas pareadas de la sección 5.4, no en los promedios.

5.2 Ranking de los modelos base

Se ordenan los tres modelos por la métrica primaria y se contrasta ese orden con su costo: número de parámetros, tiempo de entrenamiento y tiempo de inferencia. El propósito es determinar si la mayor complejidad se traduce en mejor desempeño.

▶ **Mostrar el código de la celda**

Modelo	Posición	F1 macro	Parámetros (M)	Minutos	ms por muestra
CNN 1D	1	0,8549	0,0395	1,4674	0,0287
DNN	2	0,8548	0,0127	0,6455	0,0007
LSTM	3	0,8423	0,0175	2,299	0,01



Interpretación. El orden por F1 macro es CNN 1D, DNN, LSTM. El modelo con más parámetros es CNN 1D, con 0,04 millones, y el más lento de entrenar es LSTM, con 2,3 minutos por corrida. El mejor modelo es también el más grande, aunque supera a DNN por solo 0,0002. Frente a DNN, CNN 1D usa 3,1 veces los parámetros, 2,3 veces el tiempo de entrenamiento y 41 veces el tiempo de inferencia por muestra.

Lo anterior indica que, entre estos tres modelos, más parámetros no se traducen de forma directa en mejor desempeño: LSTM tiene más parámetros que DNN y un F1 macro menor. Sin embargo, con tres modelos esa relación es solo una descripción, porque no hay grados de libertad para estimarla, y además mezcla capacidad con sesgo inductivo.

Por ello, se recomienda leer el *trade-off* junto con la significancia y la relevancia práctica de la sección 5.4: una diferencia pequeña no justifica por sí sola el costo de un modelo mayor.

5.3 Definición del baseline y margen de mejora disponible

Este es el objetivo central del entregable. Se identifica el mejor modelo, se cuantifican las diferencias absolutas y relativas con los demás, con la d de Cohen (1988) como tamaño de efecto, y se define el umbral de mejora. La mejora mínima detectable sale de la potencia exacta de la prueba t pareada con α = 0,05 y potencia de 0,80, y el umbral combina esa cifra con el margen de relevancia δ = 0,01 declarado a priori:
$\text{umbral} = \overline{F1}_{\text{baseline}} + \max(\delta, \text{mejora mínima detectable})$.

▶ **Mostrar el código de la celda**

	Comparación	Diferencia absoluta	Diferencia relativa (%)	σ̂_d	δ/σ̂_d	d de Cohen pareada	Mínima detectable	Potencia para δ
0	CNN 1D - DNN	0,0002	0,0208	0,0078	1,2826	0,0228	0,0099	0,8059
1	CNN 1D - LSTM	0,0127	1,5039	0,0177	0,564	0,7144	0,0226	0,2429

Modelo	Partición	F1	Promedio
CNN 1D, este trabajo	Agrupada por bloques	0,8549	Macro
DNN, este trabajo	Agrupada por bloques	0,8548	Macro
LSTM, este trabajo	Agrupada por bloques	0,8423	Macro
Regresión logística, este trabajo (contexto)	Agrupada por bloques	0,7661	Macro
Boosting, este trabajo (contexto)	Agrupada por bloques	0,8748	Macro
DNN, Zeng et al. (2025)	Aleatoria estratificada	0,9300	Según lo publicado
CNN, Zeng et al. (2025)	Aleatoria estratificada	0,9300	Según lo publicado
LSTM, Zeng et al. (2025)	Aleatoria estratificada	0,9400	Según lo publicado
MLP, Zarkadis y Douligeris (2026)	Aleatoria estratificada	0,9066	Según lo publicado
RealMLP, Zarkadis y Douligeris (2026)	Aleatoria estratificada	0,9428	Según lo publicado

Interpretación. El baseline es CNN 1D, con F1 macro de 0,8549 ± 0,0091 e IC 95 % de [0,8466; 0,8633]. Con 7 semillas y la mayor $\hat{\sigma}_d$ de sus comparaciones, 0,0177, la mejora mínima detectable es de 0,0226, que supera el margen δ = 0,01. Como referencias de contexto, en este mismo conjunto de prueba la regresión logística obtiene 0,7661, de modo que el peor modelo base la supera en 0,0762, y el boosting, sin ajuste de hiperparámetros, obtiene 0,8748, por encima del baseline y por debajo del umbral que se define enseguida. La tabla agrega los valores publicados, que hay que leer con dos salvedades: usan partición aleatoria, y la sección 2.1.3 mostró que esa partición sube el F1 macro de referencia en unas 0,069, de modo que una diferencia puede deberse a la partición y no al modelo; además, se transcriben como F1 tal como los publican sus autores, sin que en todos los casos se pueda confirmar que el promedio sea macro.

Esto quiere decir que un modelo avanzado debe alcanzar un F1 macro medio de al menos 0,8775 para superar el baseline de forma distinguible y relevante. Como la mínima detectable supera δ, el umbral queda fijado por la variabilidad medida y no por la relevancia práctica, lo que se declara como limitación del diseño. Con esa misma $\hat{\sigma}_d$, detectar δ con potencia de 0,80 exigiría 27 semillas pareadas. Así mismo, la ventaja sobre el modelo lineal responde la pregunta que dejó abierta la sección 2.2.3: en prueba, la no linealidad sí aporta más que δ.

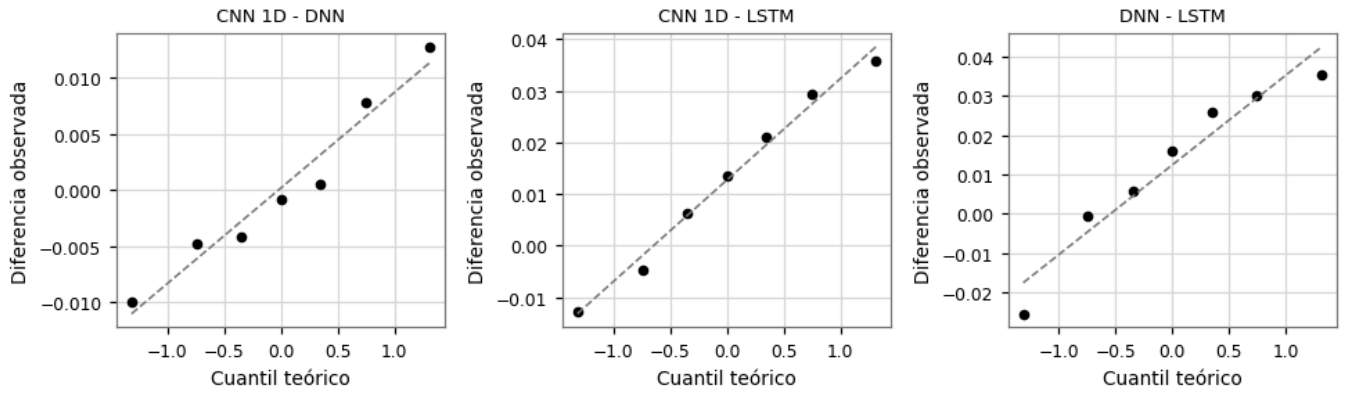
En consecuencia, se decide heredar en el Entregable 2 este umbral y la $\hat{\sigma}_d$ medida como punto de partida para dimensionar las semillas, y conservar el archivo `baseline.json` como referencia. El margen hasta el F1 macro perfecto es de 0,1451. Como cota de referencia no hay acuerdo entre anotadores ni desempeño humano, porque la etiqueta proviene del escenario; la única señal medible de ruido irreducible son las 547 filas, el 0,4 % del archivo, en grupos de casi

duplicados con más de una clase (sección 2.1.2). Esa cota es parcial, porque solo observa los flujos casi idénticos, de modo que no permite afirmar qué parte del margen es alcanzable.

5.4 Análisis estadístico y pruebas de hipótesis

Antes de elegir la prueba se verifican los supuestos: normalidad de las diferencias pareadas con Shapiro-Wilk y gráficos Q-Q (Shapiro y Wilk, 1965), y homogeneidad de varianzas con la prueba de Brown-Forsythe. Si la normalidad no se rechaza, la prueba de cada par es la t pareada; de lo contrario, Wilcoxon (1945). Esta regla se fija antes de ver los resultados, para evitar elegir la prueba según convenga, y en todos los casos se reportan ambas.

▶ **Mostrar el código de la celda**



	Par	Shapiro-Wilk, W	Shapiro-Wilk, p	Prueba elegida
0	CNN 1D - DNN	0,9551	0,7754	t pareada
1	CNN 1D - LSTM	0,9703	0,9006	t pareada
2	DNN - LSTM	0,9326	0,5734	t pareada

Interpretación. Shapiro-Wilk no rechaza la normalidad en ninguno de los pares, y los gráficos Q-Q permiten verificar si las diferencias se apartan de la recta. La prueba de Brown-Forsythe da p = 0,865 para la igualdad de varianzas entre modelos.

Hay que tener en cuenta que con 7 observaciones estas pruebas tienen poca potencia, de modo que no rechazar la normalidad no la demuestra. Además, las semillas comparten el conjunto de prueba, por lo que la independencia se cumple solo respecto a la inicialización y al orden de los lotes, y no respecto a la muestra de datos.

Por lo anterior, se aplica la regla declarada para cada par y se reportan siempre las dos pruebas, además del *bootstrap* pareado, de modo que la conclusión no dependa de un único supuesto.

▶ **Mostrar el código de la celda**

Par	CNN 1D - DNN	CNN 1D - LSTM	DNN - LSTM
Diferencia media	0,00018	0,01267	0,01249
IC 95 % t	[-0,0070; 0,0074]	[-0,0037; 0,0291]	[-0,0071; 0,0321]
IC 95 % bootstrap	[-0,0049; 0,0057]	[0,0005; 0,0247]	[-0,0030; 0,0257]
Prueba elegida	t pareada	t pareada	t pareada
p sin ajustar	0,95379	0,10764	0,16962
p Holm	0,95379	0,32291	0,33923
p t	0,95379	0,10764	0,16962
p t Holm	0,95379	0,32291	0,33923
p Wilcoxon	0,93750	0,10938	0,15625
p Wilcoxon Holm	0,93750	0,32812	0,32812
d de Cohen pareada	0,023	0,714	0,59
IC 95 % de d	[-1,25; 0,80]	[0,04; 2,40]	[-0,13; 2,73]
r biserial	-0,071	0,714	0,643
δ de Cliff	-0,02	0,551	0,592
P. de superioridad	0,429	0,714	0,714
Potencia para δ	0,806	0,243	0,185
Mínima detectable	0,00992	0,02256	0,02694
Potencia de Wilcoxon para δ	0,768	0,22	0,168
Mínima detectable de Wilcoxon	0,0105	0,02388	0,02851
p TOST	0,00787	0,64777	0,61685
Potencia de TOST	0,817	0,058	0,021

Prueba	Estadístico	p	Tamaño de efecto	Potencia para δ	Mínima detectable
ANOVA de medidas repetidas (Greenhouse-Geisser)	2,6900	0,14306	η^2 parcial = 0,310	0,173	0,0276
Friedman	2,0000	0,36788	W de Kendall = 0,143	0,186	0,0258

Par	Diferencia de rangos	p Nemenyi	Potencia para δ	Mínima detectable
CNN 1D - DNN	0,1429	0,96139	0,142	0,0262
CNN 1D - LSTM	0,5714	0,53334	0,142	0,0262
DNN - LSTM	0,7143	0,37498	0,142	0,0262

	Suma de cuadrados	Porcentaje
Modelo	0,00074	24,27107
Semilla	0,00066	21,59715
Residuo	0,00165	54,13178

Interpretación. Tras la corrección de Holm (1979), ningún par difiere de forma significativa. La ANOVA de medidas repetidas da F = 2,69 con p = 0,143 tras la corrección de Greenhouse-Geisser, y la prueba de Friedman (1937) da χ^2 = 2,00 con p = 0,368 y una W de Kendall de 0,143; la distancia crítica de Nemenyi es de 1,25 rangos (Demšar, 2006). En la descomposición de la varianza, el modelo explica el 24,3 %, la semilla el 21,6 % y el residuo el 54,1 %. En potencia, con el residuo medido y suponiendo que dos modelos difieren en δ y el tercero queda en el punto medio, que es la configuración menos favorable, la ANOVA alcanza 0,17 y su diferencia mínima detectable es de 0,0276; Friedman, 0,19 y 0,0258, y Nemenyi, 0,14 y 0,0262, estas dos últimas por simulación. Entre los pares, Wilcoxon alcanza de 0,17 a 0,77 para δ, con una mínima detectable de 0,0105 a 0,0285, y TOST, si la diferencia real fuera nula, de 0,02 a 0,82.

Es decir, hay que distinguir dos preguntas. En significancia estadística, no hay evidencia de que algún par difiera. En relevancia práctica, la diferencia media de CNN 1D - LSTM y DNN - LSTM supera δ = 0,01, pero sin significancia y con una potencia que va de 0,18 a 0,24, de modo que el resultado en esos pares es no concluyente. La equivalencia se evalúa aparte con TOST (Schuirmann, 1987): se declara equivalente, dentro de ±δ, el par CNN 1D - DNN, conclusión que se mantiene al multiplicar el valor p por las tres comparaciones. Así mismo, con 7 pares el valor p mínimo de Wilcoxon es 0,0156, de modo que esta prueba sí puede superar el nivel más exigente de Holm, α/3 = 0,0167. Además, con esas potencias, el no rechazo de las pruebas globales no descarta una diferencia de magnitud δ, y donde TOST tiene potencia baja, como en los pares con LSTM, tampoco podría declararse equivalencia aunque existiera.

Por tanto, corresponde apoyar la decisión en la diferencia media con su intervalo y en el tamaño de efecto, y no solo en el valor p. Una ausencia de significancia no se interpreta como equivalencia.

▶ **Mostrar el código de la celda**

Par	Diferencia media de accuracy	Discordantes (media)	p de McNemar (mediana)	p mínimo	p máximo	Semillas con p Holm < 0,05	Potencia para δ	Mínima detectable
CNN 1D - DNN	0,0003	1003,5714	6,94e-06	3,09e-18	0,80453	5 de 7	1,000	0,0038
CNN 1D - LSTM	0,0109	1680,8571	2,71e-12	4,44e-79	0,00038	7 de 7	1,000	0,0051
DNN - LSTM	0,0106	1695,8571	2,17e-31	1,63e-74	0,32314	6 de 7	1,000	0,0051

Par	Diferencia media de AUC por clase	Clase × semilla con p Holm < 0,05	p DeLong (mediana)	Potencia para δ	Mínima detectable
CNN 1D - DNN	0,0035	30 de 35	8,11e-20	1,000	0,0010
CNN 1D - LSTM	-0,0092	32 de 35	4,05e-28	1,000	0,0019
DNN - LSTM	-0,0127	31 de 35	5,55e-34	1,000	0,0021

Métrica	Par	Diferencia media	IC 95 % por filas	IC 95 % por bloques	Mínima detectable por filas	Mínima detectable por bloques
F1 macro	CNN 1D - DNN	0,0002	[-0,0014; 0,0017]	[-0,0082; 0,0076]	0,0022	0,0116
F1 macro	CNN 1D - LSTM	0,0126	[0,0104; 0,0147]	[-0,0011; 0,0270]	0,0032	0,0207
F1 macro	DNN - LSTM	0,0124	[0,0100; 0,0149]	[-0,0058; 0,0326]	0,0035	0,0279
Accuracy	CNN 1D - DNN	0,0003	[-0,0012; 0,0018]	[-0,0077; 0,0084]	0,0022	0,0113
Accuracy	CNN 1D - LSTM	0,0108	[0,0087; 0,0128]	[-0,0038; 0,0239]	0,0030	0,0202
Accuracy	DNN - LSTM	0,0105	[0,0082; 0,0127]	[-0,0091; 0,0291]	0,0033	0,0278
AUC macro	CNN 1D - DNN	0,0035	[0,0032; 0,0038]	[-0,0001; 0,0093]	0,0004	0,0070
AUC macro	CNN 1D - LSTM	-0,0092	[-0,0103; -0,0080]	[-0,0345; 0,0043]	0,0016	0,0299
AUC macro	DNN - LSTM	-0,0127	[-0,0139; -0,0114]	[-0,0429; 0,0036]	0,0018	0,0364

Interpretación. Estas pruebas trabajan con cada observación de prueba, no con cada semilla. McNemar (1947) compara las tasas de error mediante los casos discordantes, $\chi^2 = (|n_{01} - n_{10}| - 1)^2/(n_{01} + n_{10})$, con su versión exacta si hay menos de 25, y DeLong et al. (1988) compara las curvas ROC correlacionadas. Como el problema es multiclase, DeLong se aplica uno contra el resto, por clase, con corrección de Holm dentro de cada semilla. Como cada semilla entrega una prueba, se reporta la mediana y el rango de los valores p. El *bootstrap* estratificado por clase (Efron y Tibshirani, 1993), con 1000 remuestreos de las filas de prueba, da para el baseline un IC 95 % de [0,8508; 0,8588], y el intervalo de la diferencia excluye el cero en CNN 1D - LSTM y DNN - LSTM. Como las filas de un bloque no son independientes, el remuestreo se repite con los 24 bloques de prueba, dentro de su clase dominante, y el IC 95 % del baseline pasa a [0,7661; 0,9182], y ningún intervalo de diferencia excluye el cero. La última tabla agrega, con los mismos remuestreos, el intervalo de la diferencia media de accuracy y de AUC macro, y con los bloques ninguno de esos intervalos excluye el cero.

Hay que tener en cuenta que, con unas 22 883 observaciones, McNemar y DeLong detectan diferencias muy pequeñas: con potencia de 0,80, la mínima detectable va de 0,0038 a 0,0051 en accuracy, y de 0,0010 a 0,0021 en el AUC de una clase, de modo que un valor p bajo aquí no implica relevancia práctica. Como δ se declaró sobre el F1 macro, en estas dos métricas la potencia para δ, prácticamente de 1, solo sirve como referencia de escala. Además, ambas pruebas y el *bootstrap* por filas suponen observaciones independientes, algo que la sección 2.1.3 descarta, de modo que sus valores p y sus intervalos resultan optimistas: al respetar los bloques, la mínima detectable en F1 macro sube de un rango de 0,0023 a 0,0035 por filas a uno de 0,0116 a 0,0279 por bloques. Por eso, el remuestreo por bloques es el que mejor mide lo que las semillas no miden, cuánto cambiaría la diferencia con otra muestra de bloques, aunque con tan pocos bloques su intervalo es solo aproximado.

Así las cosas, se decide reportar McNemar y DeLong como pide el enunciado, pero solo como apoyo, sin basar en ellas ninguna afirmación de superioridad, y juzgar la relevancia siempre contra δ = 0,01.

Validación cruzada repetida

Las semillas sobre una partición fija no miden la variabilidad que proviene de la selección de los datos. Por eso, como recomienda el enunciado, se agrega una validación cruzada repetida 5 × 5, agrupada por bloques y solo con entrenamiento y validación, de modo que la prueba sigue congelada. En cada pliegue, la transformación y los pesos de clase se ajustan con la parte de entrenamiento, y un 20 % de esa parte se usa para la parada temprana. Las diferencias pareadas se prueban con la corrección de varianza de Nadeau y Bengio (2003), $t = \bar{d} \,/\, \sqrt{\left(\frac{1}{kr} + \frac{n_2}{n_1}\right)\hat{\sigma}^2_{d'}}$ porque los pliegues comparten datos de entrenamiento y la prueba sin corregir sería demasiado optimista.

▶ **Mostrar el código de la celda**

Modelo	F1 macro medio	Desviación
CNN 1D	0,8888	0,0382
DNN	0,8811	0,0417
LSTM	0,8649	0,0491

Par	Diferencia media	IC 95 % corregido	t corregida	p sin ajustar	P. de superioridad	Potencia para δ	Mínima detectable	p Holm
CNN 1D - DNN	0,0077	[-0,0082; 0,0236]	1,001	0,32697	0,76	0,238	0,0225	0,65394
CNN 1D - LSTM	0,0239	[-0,0058; 0,0536]	1,664	0,10919	0,84	0,103	0,0420	0,32756
DNN - LSTM	0,0162	[-0,0179; 0,0503]	0,979	0,33723	0,68	0,089	0,0483	0,65394

Interpretación. Con 25 pliegues, el F1 macro medio va de 0,8649 a 0,8888, y la desviación entre pliegues llega a 0,0491, mayor que la que producen las semillas sobre una partición fija, porque aquí también cambian los datos. Con la corrección de Nadeau y Bengio (2003), ningún par difiere de forma significativa tras el ajuste de Holm. La potencia para detectar δ va de 0,09 a 0,24, y la diferencia mínima detectable, de 0,0225 a 0,0483.

Esto significa que la incertidumbre por la selección de los bloques pesa más que la de la inicialización. En ese sentido, una diferencia que parece clara entre semillas puede no sostenerse cuando cambia el conjunto de datos, y por eso esta prueba es la más exigente del estudio. Con esa mínima detectable, el no rechazo no descarta diferencias mayores que δ. Hay que tener en cuenta que el estadístico corregido es conservador cuando los pliegues se solapan mucho.

Por tal motivo, se toma esta prueba como la referencia para afirmar que un modelo base supera a otro.

Análisis de sensibilidad del *dropout*

La sección 4.2 declara que el *dropout* de los tres modelos difiere del código de los autores (Zeng, 2025). Para saber si esa desviación cambia las conclusiones, se entrenan de nuevo los tres modelos con el *dropout* original, con las mismas semillas, la misma partición y los mismos hiperparámetros, y se comparan por pares con las corridas definitivas. Este análisis se decidió después de conocer los resultados, de modo que no reemplaza al baseline preregistrado y solo mide cuánto depende de esa diferencia. Además, los hiperparámetros se eligieron sin este *dropout*, por lo que no están ajustados para estas variantes.

▶ **Mostrar el código de la celda**

Modelo	F1 macro definitivo	F1 macro con dropout original	Desviación	Diferencia media	IC 95 % t	p sin ajustar	p Holm	d de Cohen pareada	Potenci para δ
CNN 1D	0,8549	0,8610	0,0082	0,0061	[-0,0059; 0,0181]	0,26215	0,78645	0,47	0,404
DNN	0,8548	0,8582	0,0111	0,0035	[-0,0070; 0,0139]	0,45022	0,90045	0,31	0,501
LSTM	0,8423	0,8357	0,0263	-0,0066	[-0,0285; 0,0154]	0,49023	0,90045	-0,28	0,157

Par con dropout original	Diferencia media	IC 95 % t	p sin ajustar	p Holm	d de Cohen pareada	Potencia para δ	p TOST
CNN 1D - DNN	0,0028	[-0,0046; 0,0102]	0,39333	0,39333	0,35	0,783	0,02762
CNN 1D - LSTM	0,0253	[0,0038; 0,0469]	0,02817	0,08451	1,09	0,161	0,93382
DNN - LSTM	0,0225	[-0,0004; 0,0455]	0,05299	0,10599	0,91	0,148	0,88518

Interpretación. Con el *dropout* del código de los autores, el F1 macro de prueba es de 0,8610 en CNN 1D, 0,8582 en DNN y 0,8357 en LSTM. El mayor cambio frente a las corridas definitivas es el de LSTM, con una diferencia media de -0,0066. Tras la corrección de Holm, ningún modelo cambia de forma significativa, y ninguna diferencia alcanza δ = 0,01. Entre las variantes, ningún par difiere de forma significativa.

Esto significa que el orden de los modelos se mantiene. Hay que tener en cuenta que no rechazar no demuestra equivalencia: TOST no la respalda en ningún modelo, y la mínima detectable llega a 0,0302. Además, este análisis usa la misma partición y hiperparámetros elegidos sin ese *dropout*.

Por tal motivo, se conserva el baseline preregistrado y se declara la desviación como una amenaza a la validez interna.

5.5 Discusión crítica

Esta subsección responde las preguntas del enunciado con la evidencia de las secciones 4.4 y 5.1 a 5.4. El resultado central es que el estudio fija un baseline y un umbral, pero no permite ordenar con certeza a los tres modelos, porque dos de ellos son equivalentes en esta partición y el tercero queda en un resultado no concluyente.

Qué modelo funciona mejor y por qué. Por F1 macro, CNN 1D (0,8549) y DNN (0,8548) quedan empatadas, y LSTM queda por detrás (0,8423). CNN 1D es la primera y por eso es el baseline, como indica el enunciado, pero su ventaja de 0,0002 no tiene peso alguno: gana en 3 de las 7 semillas, y TOST declara equivalentes a los dos modelos dentro de ±0,01 (p = 0,0079). En sesgo inductivo, la convolución y la LSTM suponen que la posición en el vector tiene significado, mientras que aquí el orden es solo el de las columnas del archivo (sección 2.2.2). Que CNN 1D iguale a DNN con 3,1 veces sus parámetros sugiere que la localidad no aporta sobre las conexiones densas, y que LSTM quede atrás es coherente con que deba resumir en un estado de 64 unidades una secuencia de 16 pasos sin sentido temporal. En capacidad y regularización, LSTM es además el modelo más sensible a los hiperparámetros, con un rango de 0,073 en la búsqueda; su configuración elegida no tiene *weight decay* ni, por la desviación declarada en la sección 4.2, *dropout*, y muestra la mayor brecha entre entrenamiento y validación, 0,037 frente a 0,005 en CNN 1D y 0,001 en DNN. El análisis de sensibilidad de la sección 5.4 tampoco respalda que la falta de *dropout* explique el rezago de LSTM: con él, su F1 macro baja a 0,8357 y su desviación sube a 0,0263, y la distancia con CNN 1D crece a 0,0253. Hay que tener en cuenta que estas explicaciones son plausibles, pero no están demostradas, porque el diseño compara tres arquitecturas completas y no aísla cada factor.

El orden, además, depende de la métrica. LSTM, la última por F1 macro, obtiene el mayor AUC-ROC macro (0,9728, frente a 0,9637 en CNN 1D y 0,9602 en DNN), el mayor AUC-PR macro (0,8791, frente a 0,8511 y 0,8336) y el menor ECE (0,0402, frente a 0,0482 y 0,0520). Esto sugiere que ordena mejor las probabilidades, aunque decide peor al tomar la clase más probable, de modo que parte de su desventaja podría estar en la decisión y no en la discriminación.

Qué modelo generaliza mejor. Ninguno se distingue de forma concluyente. La brecha entre entrenamiento y prueba es casi la misma en los tres: 0,074 en CNN 1D y en DNN, y 0,071 en LSTM. En la validación repetida, CNN 1D tiene la mayor media y la menor dispersión (0,8888 ± 0,0382), seguida de DNN (0,8811 ± 0,0417) y de LSTM (0,8649 ± 0,0491), pero ningún par difiere tras el ajuste de Holm (p = 0,654, 0,328 y 0,654). Frente a un ruido gaussiano de σ = 0,5, con los pesos de la primera semilla, DNN es la que menos cae, 0,186, frente a 0,193 en LSTM y 0,202 en CNN 1D.

Lo que sí queda claro es que el F1 absoluto depende mucho de qué bloques se evalúan. CNN 1D y DNN pasan de 0,924 y 0,927 en validación a 0,855 en prueba, y esa caída no se explica solo por haber elegido la época y los hiperparámetros sobre validación, porque la regresión logística, que no elige nada sobre esa partición, cae todavía más, de 0,8803 a 0,7661. El boosting, en cambio, solo cae de 0,8875 a 0,8748. Una explicación posible, que este diseño no permite comprobar, es que los modelos que extrapolan sufren más con el desplazamiento de covariables de la sección 2.1.4, mientras que los árboles saturan fuera del rango visto. Así mismo, el remuestreo de los 24 bloques de prueba lleva el IC 95 % del baseline a [0,7661; 0,9182], y en la validación repetida el F1 macro de CNN 1D va de 0,8300 a 0,9649 entre pliegues. En ese sentido, el 0,8549 solo es comparable con otros modelos evaluados sobre esta misma partición.

Si las diferencias son significativas y relevantes. No hay evidencia estadística sólida de diferencias. La prueba t pareada da valores p ajustados por Holm de 0,954, 0,323 y 0,339; la ANOVA de medidas repetidas da p = 0,143; Friedman, p = 0,368, y la validación repetida con la corrección de Nadeau y Bengio (2003) tampoco rechaza. El *bootstrap* pareado por semillas apenas excluye el cero en CNN 1D - LSTM, con [0,0005; 0,0247], pero sin ajuste por comparaciones múltiples y con solo 7 pares. McNemar, DeLong y el *bootstrap* por filas sí rechazan, y ninguno sostiene una afirmación de superioridad, porque suponen filas independientes: con el remuestreo por bloques, ningún intervalo excluye el cero.

En relevancia práctica, la situación cambia según el par. Entre CNN 1D y DNN, la diferencia de 0,0002 queda muy por debajo de δ = 0,01, con un intervalo por bloques de [-0,0082; 0,0076], contenido en ±δ. Sin embargo, esa equivalencia vale para esta partición, porque en la validación repetida, donde cambian los datos de entrenamiento, el intervalo de la diferencia es de [-0,0082; 0,0236] y no cabe en ±δ. En los dos pares con LSTM, la diferencia media, de 0,0127 y 0,0125, supera δ, pero sus intervalos t, [-0,0037; 0,0291] y [-0,0071; 0,0321], incluyen el cero, la potencia para δ es de solo 0,24 y 0,18, y TOST tampoco permite declararlos equivalentes (p = 0,648 y 0,617), con una potencia de solo 0,06 y 0,02 aunque la diferencia real fuera nula. Es decir, en esos pares el resultado es no concluyente: LSTM puede ser peor en una magnitud relevante, pero este diseño no lo demuestra.

Si existe un *trade-off* entre desempeño y costo. Sí, y favorece al modelo más simple. DNN iguala a CNN 1D con un tercio de sus parámetros (12 677 frente a 39 493), menos de la mitad del tiempo de entrenamiento (0,65 frente a 1,47 minutos por corrida) y una inferencia unas 41 veces más rápida (0,0007 frente a 0,0287 ms por muestra). LSTM es la más costosa de entrenar, con 2,30 minutos, sin que ese costo se refleje en el F1 macro. Por tal motivo, aunque CNN 1D sea el baseline por la regla del enunciado, DNN queda como la alternativa recomendada

cuando importe el costo, por ejemplo, en un despliegue a bordo, teniendo en cuenta que estos tiempos se midieron en un solo equipo con CPU. Así mismo, el boosting, sin ajuste de hiperparámetros, supera al baseline en 0,0199 en este mismo conjunto de prueba, lo que indica que estas arquitecturas de Deep Learning no tienen aquí una ventaja ganada frente a un método clásico para datos tabulares.

Baseline y umbral de mejora. El baseline es CNN 1D, con F1 macro de 0,8549 ± 0,0091 e IC 95 % de [0,8466; 0,8633] entre semillas. Para ser una mejora distinguible y relevante, un modelo avanzado debe alcanzar un F1 macro medio de al menos 0,8775, es decir, 0,0226 por encima del baseline, evaluado con esta misma partición y con semillas pareadas. Ese umbral lo fija la mínima detectable, que con 7 semillas y $\hat{\sigma}_d$ = 0,0177 supera a δ = 0,01; con unas 27 semillas, el umbral podría bajar hasta cerca de 0,8649, y entonces lo fijaría la relevancia práctica. El margen teórico hasta un F1 perfecto es de 0,1451.

Margen que sugiere la literatura y su credibilidad. Los valores de referencia publicados sobre UAVIDS-2025 que se comparan aquí van de 0,9066 a 0,9428, entre 0,052 y 0,088 por encima del baseline, y otros estudios reportan más, como 0,962 (Mohamed et al., 2026) o 0,986 (Wang et al., 2026). Esa brecha no es creíble como margen de mejora real, porque no compara lo mismo: esos estudios parten de forma aleatoria, lo que en la sección 2.1.3 sube el F1 macro de referencia en 0,0689, y Zarkadis y Douligeris (2026) codifican las direcciones IP, que aquí se excluyen por funcionar como atajo. Además, esos estudios no siempre declaran si su F1 es macro o ponderado, aunque en este conjunto de datos esa diferencia pesa poco: el F1 ponderado del baseline es de 0,8619 y el macro de 0,8549, una distancia de 0,007, muy inferior a la que produce la partición, porque las clases están casi balanceadas. Al descontar solo el efecto de la partición, la distancia frente a la CNN de Zeng et al. (2025), de 0,0751, queda en 0,0062, por debajo de δ, aunque ese descuento es aproximado, porque se midió con el boosting en validación. La evidencia más creíble es interna: el boosting sin ajuste queda 0,0199 por encima del baseline, a 0,0027 del umbral. Esto significa que el margen con respaldo hoy es del orden de 0,02, similar al umbral e inferior a la desviación de cerca de 0,04 entre pliegues, de modo que un modelo mejor puede no distinguirse si no se aumentan las semillas.

Amenazas a la validez.

- **Validez interna.** El archivo no identifica la simulación de origen, y la partición por bloques reduce esa fuga, pero no la elimina: `FlowID`, que no se usa como entrada, conserva un F1 macro de 0,859 con partición agrupada, porque los tramos de una sola clase son más largos que el bloque (sección 2.1.4), y el 1,8 % de las filas de prueba tiene un casi duplicado en entrenamiento. Si los atributos de tráfico delatan el tramo del archivo, el baseline sigue

siendo optimista. La optimización desigual se evita con el mismo presupuesto de ajuste para los tres modelos (sección 4.2.3), aunque la búsqueda usa una sola semilla. Además, el *dropout* de los tres modelos difiere del código de los autores (sección 4.2), una desviación que se detectó con los resultados ya calculados. Con el *dropout* original, el orden se mantiene y ninguna diferencia media alcanza δ: es de 0,0061 en CNN 1D, 0,0035 en DNN y -0,0066 en LSTM, sin significancia. Sin embargo, sus intervalos todavía admiten diferencias mayores que δ, de modo que la desviación no se puede descartar como irrelevante. La prueba no se usó para elegir hiperparámetros, y ni los modelos ni la partición cambiaron después de verla.

- **Validez externa.** Los flujos provienen de un solo simulador y de una sola configuración de radio y enrutamiento, sin drones reales. Por tanto, los resultados no se extienden a otros enjambres ni a capturas en campo, y los atributos de tráfico pueden reflejar parámetros de cada escenario y no solo el ataque (sección 2.1.4).
- **Validez de construcción.** El F1 macro pondera por igual las cinco clases, aunque la proporción de clases la definió quien construyó el archivo. La etiqueta corresponde al escenario y no a cada flujo, de modo que parte del error puede ser ruido de etiquetas (sección 2.1.2). Además, el orden de los modelos cambia entre el F1 macro y el AUC, de modo que la conclusión depende de la métrica; por esa razón, la métrica primaria se declaró a priori y se complementa con los resultados por clase, por tamaño de flujo y con las demás métricas declaradas.

Limitaciones del estudio. Se evalúa una sola partición de un solo conjunto de datos simulado, y las semillas solo miden la inicialización y el orden de los lotes. El piloto estimó la variabilidad en validación, con $\hat{\sigma}_d$ de 0,0068 a 0,0089, y en prueba resultó mayor, de 0,0078 a 0,0212, por lo que las 7 ejecuciones preregistradas solo dieron potencia suficiente para el par CNN 1D - DNN. Además, 174 de los 228 estudios elegibles, entre ellos 40 de los 68 Q1, no se leyeron a texto completo, y la cadena original omitió sinónimos de método y de tarea: la verificación de cobertura halló 23 registros elegibles más, de los cuales se leyeron los 5 Q1 y quedan 18 sin leer (sección 3.1), y de los evaluados se revisaron las tablas y los pasajes de comparación y protocolo, no el texto íntegro.

Recomendaciones futuras. Para el Entregable 2 se recomienda conservar `particion.npz`, el umbral de `baseline.json` y el mismo protocolo, usar el *dropout* del código de los autores, comparando entonces contra las corridas de `resultados/sensibilidad/`, y volver a ajustar sus hiperparámetros, y aumentar las semillas hasta unas 27 o, al menos, reportar la mínima detectable que resulte. Así mismo, conviene incluir arquitecturas pensadas para datos tabulares, que no supongan un orden entre atributos, y exigirles el resultado por clase, porque el mayor

espacio de mejora está en separar los ataques entre sí: en CNN 1D, Normal ya alcanza un F1 de 0,9688, mientras que los cuatro ataques van de 0,8017 a 0,8517, con Blackhole predicho como Wormhole en el 17,6 % de sus muestras. También conviene revisar la decisión de LSTM con calibración sobre validación y evaluar el efecto de los pesos de clase, porque en Flooding, la clase con mayor peso (1,936), el recall de 0,9059 contrasta con una precision de 0,7209. Por último, para la validez externa hace falta obtener de los autores el identificador de simulación o probar el modelo con otro conjunto de datos de drones.

6. Guía de procedimiento para clasificación con datos tabulares

El enunciado indica cómo proceder según el tipo de problema. La tabla contrasta cada punto de su guía para clasificación tabular con lo que se hace en este trabajo, y señala dónde se desarrolla.

Aspecto	Guía del enunciado	Aplicación en este trabajo	Dónde
Partición	Estratificada por clase; agrupada si hay varios registros por entidad; validación cruzada anidada si el tamaño es reducido	Estratificada por clase y agrupada por bloques de filas; sin validación anidada, porque hay decenas de miles de flujos	Secciones 2.1.3 y 4.1
Preprocesamiento	Imputación, escalado y codificación ajustados solo en entrenamiento; *embeddings* para categóricas de alta cardinalidad	No hace falta imputar; transformación y estandarización ajustadas solo en entrenamiento; las categóricas se excluyen, porque funcionan como atajos	Secciones 2.1.2, 2.1.4 y 2.2.1
Modelos base típicos	MLP superficial, MLP profundo regularizado y red residual tabular	Los que resultan de la tabla de frecuencia de la literatura, como exige el enunciado	Sección 3.2
Pérdida	Entropía cruzada, ponderada o *focal* ante desbalance	Entropía cruzada ponderada	Secciones 2.2.5 y 4.2.2
Evaluación	F1 macro, AUC-ROC, AUC-PR, MCC y calibración	Las cinco, además de las métricas declaradas en la sección 1.1	Secciones 4.4 y 5.1
Inferencia	DeLong, McNemar y Friedman con *post-hoc*	Las tres, con Nemenyi y Holm como *post-hoc*	Sección 5.4
Referencia de contexto	Regresión logística, para dimensionar el margen	Regresión logística con regularización L2, además de un boosting	Secciones 2.2.3 y 5.3

7. Limitaciones inferenciales de trabajar con tres modelos

En esta sección se declaran los límites de lo que este diseño permite concluir, para leer los resultados con honestidad. El propósito del entregable es fijar el baseline y su variabilidad, no obtener significancia a toda costa.

Potencia con tres modelos. Con pocas ejecuciones, varias pruebas tienen muy poca potencia. La prueba de Friedman con tres modelos es una de ellas, y su *post-hoc* de Nemenyi es conservador. Con n pares, el valor p exacto bilateral más pequeño de Wilcoxon es $2/2^n$: con 5 pares es 0,0625, de modo que nunca podría rechazar H_0 a α = 0,05, y por esto se usan al menos 6 ejecuciones. Con las tres comparaciones entre pares, además, el nivel más exigente de la corrección de Holm es α/3 ≈ 0,0167, lo que con 6 pares ($2/2^6$ = 0,03125) tampoco deja rechazar en la primera comparación; con las 7 ejecuciones de este diseño ($2/2^7$ = 0,0156) ya es posible. Una ausencia de significancia no demuestra equivalencia.

Diseño de repetición. La validación cruzada repetida 5 × 5 con la corrección de varianza de Nadeau y Bengio (2003) da más observaciones pareadas y mide también la selección de los datos, y por ser viable en CPU, con unas 2 horas en este equipo, se aplica en la sección 5.4, además de las semillas repetidas sobre una partición fija. Hay que tener en cuenta que las ejecuciones con semillas comparten el conjunto de prueba, de modo que no son estrictamente independientes, y que su variabilidad corresponde a la inicialización y al orden de los lotes, no a la selección de los datos.

Qué se hace al respecto. Las comparaciones se complementan con la prueba t pareada cuando se cumple la normalidad y con el *bootstrap* pareado. Se reporta la diferencia mínima detectable con las ejecuciones disponibles (secciones 4.3.1 y 5.3) y, para cada prueba de hipótesis, junto con su potencia para δ, además de los tamaños de efecto con su intervalo y la probabilidad de superioridad (sección 5.4). Para afirmar equivalencia entre dos modelos se usa TOST con el margen δ, y no la ausencia de significancia (sección 5.4).

Qué ocurrió en este estudio. El preregistro fijó 7 ejecuciones con la variabilidad que el piloto midió en validación. En prueba, la variabilidad fue mayor, de modo que la potencia para δ es de 0,81 en CNN 1D - DNN, pero de solo 0,24 y 0,18 en los pares con LSTM, con diferencias mínimas detectables de 0,0226 y 0,0269. Friedman (p = 0,368) y Nemenyi no rechazan, con una potencia para δ de solo 0,19 y 0,14, y ese resultado no se interpreta como equivalencia: la equivalencia del par CNN 1D - DNN proviene de TOST, y los pares con LSTM quedan como no

concluyentes, porque en ellos TOST tiene una potencia de solo 0,06 y 0,02. En la validación repetida, la potencia para δ va de 0,09 a 0,24. McNemar y DeLong, en cambio, tienen una potencia cercana a 1, pero a costa de suponer filas independientes: al respetar los bloques, la mínima detectable en F1 macro es de 0,0116 a 0,0279. Detectar δ con la mayor $\hat{\sigma}_d$ de las comparaciones del baseline exigiría 27 semillas.

8. Conclusiones

Este entregable fija, con evidencia estadística, el punto de partida del proyecto: la detección de intrusiones en redes de drones con UAVIDS-2025, clasificando cada flujo como tráfico normal o como uno de cuatro ataques, con el F1 macro como métrica primaria y con δ = 0,01 y α = 0,05 declarados a priori.

El baseline es CNN 1D, con un F1 macro de 0,8549 ± 0,0091 e IC 95 % de [0,8466; 0,8633], y el umbral de mejora es de 0,8775. Un modelo avanzado debe alcanzar ese valor, con esta misma partición y con semillas pareadas, para ser una mejora distinguible y relevante. El umbral lo fija la variabilidad medida, porque la mejora mínima detectable, de 0,0226, supera a δ, lo que se declara como una limitación del diseño.

Los tres modelos base no se pueden ordenar con certeza. Ninguna prueba que respete la dependencia entre filas detecta diferencias. CNN 1D y DNN son equivalentes dentro de ±δ en esta partición, según TOST (p = 0,0079), y DNN lo logra con un tercio de los parámetros, menos de la mitad del tiempo de entrenamiento y una inferencia unas 41 veces más rápida, de modo que es la alternativa recomendada cuando importe el costo. Los pares con LSTM quedan como no concluyentes, con una potencia de 0,24 y 0,18, y aunque LSTM es la última por F1 macro, obtiene los mejores AUC-ROC, AUC-PR y ECE.

La forma de partir los datos pesa más que la arquitectura. La partición aleatoria sube el F1 macro de referencia en 0,0689, casi siete veces δ y más de cinco veces la mayor diferencia entre modelos base, de 0,0127. Esto explica buena parte de la distancia con los valores de referencia publicados, de 0,9066 a 0,9428, de modo que esa brecha no es un margen de mejora creíble.

El margen con respaldo es estrecho. El margen teórico hasta un F1 perfecto es de 0,1451, pero el boosting sin ajuste, con 0,8748, sugiere que la mejora con respaldo hoy es del orden de 0,02, cercana al umbral. Además, el F1 absoluto depende mucho de los bloques evaluados, porque el IC 95 % por bloques del baseline va de 0,7661 a 0,9182. Por eso, lo que se hereda al Entregable 2 son las comparaciones pareadas sobre esta partición, y no el valor absoluto.

Los resultados tienen límites claros. Los datos son simulados, de un solo simulador y sin identificador de simulación; la partición por bloques reduce la fuga, pero no la elimina, como muestra `FlowID`, que conserva un F1 macro de 0,859, y 174 de los 228 estudios elegibles de la revisión, más 18 de la verificación de cobertura, no se leyeron a texto completo. Además, el *dropout* de los modelos difiere del código de los autores; al corregirlo en un análisis de sensibilidad, el orden se mantiene, el F1 macro de CNN 1D pasa a 0,8610, por debajo del umbral, y ninguna diferencia media alcanza δ. En consecuencia, estas cifras describen el comportamiento dentro de UAVIDS-2025 y no se extienden a enjambres reales.

Para el Entregable 2 se heredan `particion.npz`, `declaracion_a_priori.json`, `preregistro.json` y `baseline.json`, con la recomendación de subir a unas 27 semillas para que el umbral lo fije la relevancia práctica y no la variabilidad.

9. Bibliografía

- Abbas, S., Fayaz, M., Ullah, A., et al. (2026). A Federated Learning-Based Hybrid Architecture for Multi-Class Attack Detection in UAVs Communications. *IEEE Open Journal of the Communications Society, 7*, 6121–6135. https://doi.org/10.1109/ojcoms.2026.3698792
- Abdulhammed, R., Al-Dabag, M. L. A., Najeeb, S. M. M., et al. (2026). Assessing learning-based models for intrusion detection in uav networks. *Archives for Technical Sciences*, 229. https://doi.org/10.70102/afts.2026.1836.229
- Aburasain, R. Y. (2024). Enhanced Black Widow Optimization With Hybrid Deep Learning Enabled Intrusion Detection in Internet of Things-Based Smart Farming. *IEEE Access, 12*, 16621–16631. https://doi.org/10.1109/access.2024.3359043
- Alharthi, R. S. (2024). Enhancing unmanned aerial vehicle and smart grid communication security using a ConvLSTM model for intrusion detection. *Frontiers in Energy Research, 12*. https://doi.org/10.3389/fenrg.2024.1491332
- Alissa, K. A., Alotaibi, S. S., Alrayes, F. S., et al. (2022). Crystal Structure Optimization with Deep-Autoencoder-Based Intrusion Detection for Secure Internet of Drones Environment. *Drones, 6*(10), 297. https://doi.org/10.3390/drones6100297
- Alotaibi, S. S., Sayed, A. E., Elhameed, E. S. A., et al. (2024). Enhancing Security in IoT-Assisted UAV Networks Using Adaptive Mongoose Optimization Algorithm With Deep Learning. *IEEE Access, 12*, 63768–63776. https://doi.org/10.1109/access.2024.3392618
- Alruwaili, F. J. y Tunc, C. (2026). Federated Lightweight Intrusion Detection in Drone Swarms with Knowledge Distillation. *arXiv (Cornell University)*.

https://doi.org/10.48550/arxiv.2607.17025

- Alsaraireh, S., Al-Fayoumi, M. A. y Alnabhan, M. (2026). Dual-Impact Feature Selection for Adversarially Robust, Functionality-Preserving UAV Intrusion Detection. *Drones, 10*(7), 503. https://doi.org/10.3390/drones10070503
- Alshammari, S. M., Alganmi, N. A., Ba-Aoum, M. H., et al. (2024). Hybrid arithmetic optimization algorithm with deep learning model for secure Unmanned Aerial Vehicle networks. *AIMS Mathematics, 9*(3), 7131–7151. https://doi.org/10.3934/math.2024348
- Alshamrani, A. y Alghamdi, A. M. (2026). Zero-shot attack detection in UAV networks using foundation models. *Alexandria Engineering Journal, 136*, 105–124. https://doi.org/10.1016/j.aej.2025.12.065
- Arulalan, V., Balamurugan, G. y Premanand, V. (2023). A Deep CNN Framework for UAV Intrusion Detection in Intelligent Systems. *International Journal on Recent and Innovation Trends in Computing and Communication, 11*(9), 227–236. https://doi.org/10.17762/ijritcc.v11i9.8338
- Ashraf, S. N., Manickam, S., Zia, S. S., et al. (2023). IoT empowered smart cybersecurity framework for intrusion detection in internet of drones. *Scientific Reports, 13*(1), 18422. https://doi.org/10.1038/s41598-023-45065-8
- Attaullah, H. M., Basheer, S., Ehsan, M., et al. (2026c). Multi-Agent Federated Edge Learning for UAV-IDS in Smart City IoT Environment. *IEEE Transactions on Consumer Electronics, 72*(2), 3988–3995. https://doi.org/10.1109/tce.2026.3667039
- Attaullah, H. M., Ehsan, M., Basheer, S., et al. (2026b). GenFed-IDS: A Lightweight Federated Generative AI Framework for UAV Anomaly Detection in Rescue Operations. *IEEE Transactions on Consumer Electronics, 72*(3), 7662–7669. https://doi.org/10.1109/tce.2026.3658881
- Attaullah, H. M., Khan, I. U., Alam, M. M., et al. (2026a). BPBiLSTM-IDS: a lightweight intrusion detection framework for cyber-physical UAV networks. *Scientific Reports, 16*(1). https://doi.org/10.1038/s41598-026-55446-4
- Babu, S. A., Ranganath, A., Goswami, M. M., et al. (2024). Modified Marine Predators Algorithm With Deep Learning-Driven Security Solution for IoT-Assisted UAV Networks. *IEEE Access, 12*, 54991–54998. https://doi.org/10.1109/access.2024.3386570
- Bakiri, A., Fenanir, S. y Goudjil, L. (2026). H-MicroKAN: An energy-aware hierarchical Kolmogorov–Arnold Network for TinyML UAV intrusion detection with conditional inference. *Journal of Systems Architecture, 181*, 104038. https://doi.org/10.1016/j.sysarc.2026.104038

- Balgoon, J. O. (2026). Edge-Aware GATv2-BiLSTM Framework for Spatiotemporal Intrusion Detection in Autonomous UAV Command-and-Control Networks. *International Journal of Intelligent Systems and Applications in Engineering*. https://ijisae.org/index.php/IJISAE/article/view/8549
- Batur Şahin, C. (2026). Quantum-Resilient Federated Learning for Multi-Layer Cyber Anomaly Detection in UAV Systems. *Sensors, 26*(2), 509. https://doi.org/10.3390/s26020509
- Biswas, T. K., Zannat, A., Ishtiaq, W., et al. (2026). A novel unified lightweight temporal-spatial transformer approach for intrusion detection in drone networks. *Scientific Reports, 16*(1). https://doi.org/10.1038/s41598-026-45063-6
- Cengiz, K., Lipsa, S., Dash, R. K., et al. (2024). A Novel Intrusion Detection System Based on Artificial Neural Network and Genetic Algorithm With a New Dimensionality Reduction Technique for UAV Communication. *IEEE Access, 12*, 4925–4937. https://doi.org/10.1109/access.2024.3349469
- Cohen, J. (1988). *Statistical power analysis for the behavioral sciences* (2.ª ed.). Lawrence Erlbaum Associates.
- DeLong, E. R., DeLong, D. M. y Clarke-Pearson, D. L. (1988). Comparing the areas under two or more correlated receiver operating characteristic curves: A nonparametric approach. *Biometrics, 44*(3), 837–845.
- Demšar, J. (2006). Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research, 7*, 1–30.
- Efron, B. y Tibshirani, R. J. (1993). *An introduction to the bootstrap*. Chapman & Hall.
- Friedman, M. (1937). The use of ranks to avoid the assumption of normality implicit in the analysis of variance. *Journal of the American Statistical Association, 32*(200), 675–701.
- Geirhos, R., Jacobsen, J.-H., Michaelis, C., Zemel, R., Brendel, W., Bethge, M. y Wichmann, F. A. (2020). Shortcut learning in deep neural networks. *Nature Machine Intelligence, 2*(11), 665–673.
- Gharami, K. y Moni, S. S. (2025). An Efficient Privacy-preserving Intrusion Detection Scheme for UAV Swarm Networks. En *IEEE/AIAA Digital Avionics Systems Conference (DASC)* (pp. 1–10). https://doi.org/10.1109/dasc66011.2025.11257309
- Hadi, H. J., Cao, Y., Li, S., et al. (2024). Real-time fusion multi-tier DNN-based collaborative IDPS with complementary features for secure UAV-enabled 6G networks. *Expert Systems with Applications, 252*, 124215. https://doi.org/10.1016/j.eswa.2024.124215
- Hadi, H. J., Khan, M. K. y Ahmad, N. (2026). A Real-Time Multi-Tier Machine Learning Intrusion Detection Framework for Securing Ground Control Station–UAV Communications.

IEEE Open Journal of the Communications Society, 7, 3881–3900. https://doi.org/10.1109/ojcoms.2026.3683883

- Hakeem, A., Sabir, M., Alhebshi, R. M., et al. (2025). Integrating artificial intelligence for improved security of IoT-drones through cyber-physical attack detection. *Frontiers in Computer Science, 7*. https://doi.org/10.3389/fcomp.2025.1545282
- Hamad, A. H., Hussein, N. K. y Abdulghani, A. M. (2025). A Deep Learning Paradigm for Intrusion Detection in Unmanned Aerial Vehicle Networks Using Extended LSTM. *International journal of intelligent engineering and systems, 18*(4), 507–523. https://doi.org/10.22266/ijies2025.0531.33
- Hochreiter, S. y Schmidhuber, J. (1997). Long short-term memory. *Neural Computation, 9*(8), 1735–1780. https://doi.org/10.1162/neco.1997.9.8.1735
- Holm, S. (1979). A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics, 6*(2), 65–70.
- Junaid, , Algarni, F., Khan, M. I., et al. (2026). An AI-driven framework for intrusion detection and predictive threat modeling in UAV ecosystem. *Internet of Things, 40*, 102095. https://doi.org/10.1016/j.iot.2026.102095
- Katbar, N. M., Hakeem, A., Ahmed, I., et al. (2024). Radial Basis Function Neural Networks (RBFNN) for Secure Intrusion Detection in the Internet of Drones (IoD). *Proceedings of the Bulgarian Academy of Sciences, 77*(8). https://doi.org/10.7546/crabs.2024.08.09
- Katib, I. A., Ragab, M., Sharaf, S. A., et al. (2025). Integrating convolutional stacked long short-term memory with bio-inspired algorithm for secure unmanned aerial vehicle communication in SDN environment. *Results in Engineering, 27*, 106838. https://doi.org/10.1016/j.rineng.2025.106838
- Kingma, D. P. y Ba, J. (2015). Adam: A method for stochastic optimization. En *International Conference on Learning Representations*. https://arxiv.org/abs/1412.6980
- Li, M., Jin, Y., Ji, W., et al. (2026a). Enhancing UAV Intrusion Detection Against Adversarial Samples Based on AdvGAN and Decision Boundary Reconstruction. *IEEE Internet of Things Journal, 13*(10), 20705–20717. https://doi.org/10.1109/jiot.2026.3665688
- Li, X., Zhang, T., Wang, J., et al. (2026b). Enhance UAV Network Resilience by Malicious Traffic Detection: A Twin Graph Encoder Approach. *IEEE Transactions on Communications, 74*, 9138–9153. https://doi.org/10.1109/tcomm.2026.3693173
- Lopez-Paz, D. y Oquab, M. (2017). Revisiting classifier two-sample tests. En *International Conference on Learning Representations*.
- McNemar, Q. (1947). Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika, 12*(2), 153–157.

- Menssouri, S., Delamou, M., Ibrahimi, K., et al. (2024). Enhanced Intrusion Detection System for Multiclass Classification in UAV Networks. En *IEEE International Symposium on Personal, Indoor and Mobile Radio Communications (PIMRC)* (pp. 1–6). https://doi.org/10.1109/pimrc59610.2024.10817244
- Mohamed, M., Mohamed, D. y Alosman, K. (2026). Spatio-temporal graph neural networks for zero-day intrusion detection in low-altitude autonomous UAV swarms. *Ad Hoc Networks, 191*, 104251. https://doi.org/10.1016/j.adhoc.2026.104251
- Nadeau, C. y Bengio, Y. (2003). Inference for the generalization error. *Machine Learning, 52*(3), 239–281.
- Negm, N., Alamro, H., Allafi, R., et al. (2024). Tasmanian devil optimization with deep autoencoder for intrusion detection in IoT assisted unmanned aerial vehicle networks. *Ain Shams Engineering Journal, 15*(11), 102943. https://doi.org/10.1016/j.asej.2024.102943
- Ozsoy, D. B., Ozcan, A. y Al-Hubaishi, M. (2026). Federated Learning for Intrusion Detection in UAV Networks. *Turkish Journal of Mathematics and Computer Science, 18*(2), 481–503. https://doi.org/10.47000/tjmcs.1828534
- Page, M. J., McKenzie, J. E., Bossuyt, P. M., et al. (2021). The PRISMA 2020 statement: An updated guideline for reporting systematic reviews. *BMJ, 372*, n71. https://doi.org/10.1136/bmj.n71
- Schuirmann, D. J. (1987). A comparison of the two one-sided tests procedure and the power approach for assessing the equivalence of average bioavailability. *Journal of Pharmacokinetics and Biopharmaceutics, 15*(6), 657–680.
- SCImago (2026). *SJR: SCImago Journal & Country Rank* [Conjunto de datos]. https://www.scimagojr.com
- Shapiro, S. S. y Wilk, M. B. (1965). An analysis of variance test for normality (complete samples). *Biometrika, 52*(3/4), 591–611.
- SureshBabu, P. y Geetha, M. (2026). Deep Reinforced Adaptive Graph Optimization Network for Intelligent Intrusion Detection in UAV Communication Networks. *Indian Engineering Journal*. https://indianengineeringjournal.com/index.php/journal/article/view/270
- Tan, X., Su, S., Zuo, Z., et al. (2019). Intrusion Detection of UAVs Based on the Deep Belief Network Optimized by PSO. *Sensors, 19*(24), 5529. https://doi.org/10.3390/s19245529
- Wang, J., Alsubait, T., Mufleh, A. S. S., et al. (2026). Residual-aware lightweight deep learning framework for high-fidelity intrusion detection in UAV swarm networks. *Scientific Reports, 16*(1). https://doi.org/10.1038/s41598-026-52524-5
- Wilcoxon, F. (1945). Individual comparisons by ranking methods. *Biometrics Bulletin, 1*(6), 80–83.

- Wu, Y., Yang, L., Zhang, L., et al. (2024). Intrusion Detection for Unmanned Aerial Vehicles Security: A Tiny Machine Learning Model. *IEEE Internet of Things Journal, 11*(12), 20970–20982. https://doi.org/10.1109/jiot.2024.3360231
- Zarkadis, I.-C. y Douligeris, C. (2026). XAI and Statistical Analysis for Reliable Intrusion Detection in the UAVIDS-2025 Dataset: From Tree to Hybrid and Tabular DNN Ensembles. En *International Conference on Computer, Information and Telecommunication Systems* (pp. 1–8). https://doi.org/10.1109/cits70307.2026.11637173
- Zeng, Q. (2025). *UAV–IDS2025* [Código fuente]. GitHub. https://github.com/QingliZeng/UAV–IDS2025
- Zeng, Q., Bashir, A. y Nait-Abdesselam, F. (2025). *UAVIDS-2025* [Conjunto de datos]. Zenodo. https://doi.org/10.5281/zenodo.15336998
- Zeng, Q., Bashir, A. y Naït-Abdesselam, F. (2025). UAVIDS-2025: A Benchmark Dataset for Intrusion Detection in UAV Networks Using Machine Learning Techniques. En *IEEE Conference on Communications and Network Security* (pp. 1–9). https://doi.org/10.1109/cns66487.2025.11194990
- Zhou, W., Li, B. y Zeng, X. (2026). Intrusion detection model of UAV system based on machine learning and neural network. *Frontiers in Physics, 13*. https://doi.org/10.3389/fphy.2025.1660104